DEPSTAR

Devang Patel Institute of Advance Technology and Research

(A Constitute Institute of CHARUSAT)

Certificate

This is to certify that

Mr./~~Mrs.~~ AARAV HITESHKUMAR PATEL

of CE-2 *Class,*

ID. No. 25DCE072 *has satisfactorily completed*

his/~~her~~ term work in ITUE203 (WDF) *for*

the ending in October 2026 /2027

Date : 21/10/26

Sign. of Faculty

Head of Department

Scanned with OKEN Scanner

INDEX

ITUE203 - Web Development Fundamentals (WDF)

Pract. No.	Practical Name	GitHub Repository	Page No.
1	Project Initiation: Scope, Sitemap, Wireframe and GitHub Repo	https://github.com/Aarav280907/WEB	3
2	Static HTML5 Skeletons for StudentHub Pages	https://github.com/Aarav280907/WEB	9
3	Responsive Layouts with CSS Grid and Flexbox	https://github.com/Aarav280907/WEB/tree/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css	36
4	Dynamic UI Components with JavaScript	https://github.com/Aarav280907/WEB/tree/f711ece9a5c0a3d9163db60f7684acf5837073a8/Practical(1-4)(Studenthub)/javascript	47
5	Registration Form with JavaScript Validation	https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/javascript	55
6	Fetch API: JSON Data with Search, Filter, Sort and Pagination	https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-6	63
7	PHP Form Processing (POST) with CSV/JSON Storage	https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-7	72
8	MySQL Schema, ER Model and PDO/MySQLi Connection	https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-8	77
9	Secure User Registration with MySQLi	https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-9	81
10	Login/Logout, Sessions and Role-Based Access	https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-10	86
11	Student Management (CRUD) with PHP and MySQL	https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-11	94
12	Event Management (CRUD) with Poster Upload	https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-12	101
13	PHP JSON API and AJAX (Fetch) CRUD	https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-13	109
14	Admin Dashboard with Audit Log	https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-14	115
15	Integration, Testing and Local Deployment	https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-15	121

PRACTICAL-1

AIM : Initiate the semester-long StudentHub portal by identifying the problem scope, user roles, key modules, navigation flow, and minimum 10 pages. Create a sitemap, low-fidelity wireframe, project folder structure, README file, and GitHub repository.

SOLUTION:

GITHUB REPO : https://github.com/Aarav280907/WEB

Requirement Analysis :

- Project Name : Student Hub Portal
- Objective : To develop a web portal that allows students to access academic information, study materials, notices, assignments, and announcements from a single platform.
- Target Users : Students , Teachers
- Functions Requirements :

Functional Requirements :

1. Login / Sign up
2. Username & Password
3. Dashboard
4. My Attendance
5. Time table
6. Fees Detail
7. Student Profile
8. Announcement & Notices
9. Meterial
10. Submit Assignments
11. Exam Timetable and Results
12. Contact Faculty And Admin

Non-Functional Requirements:

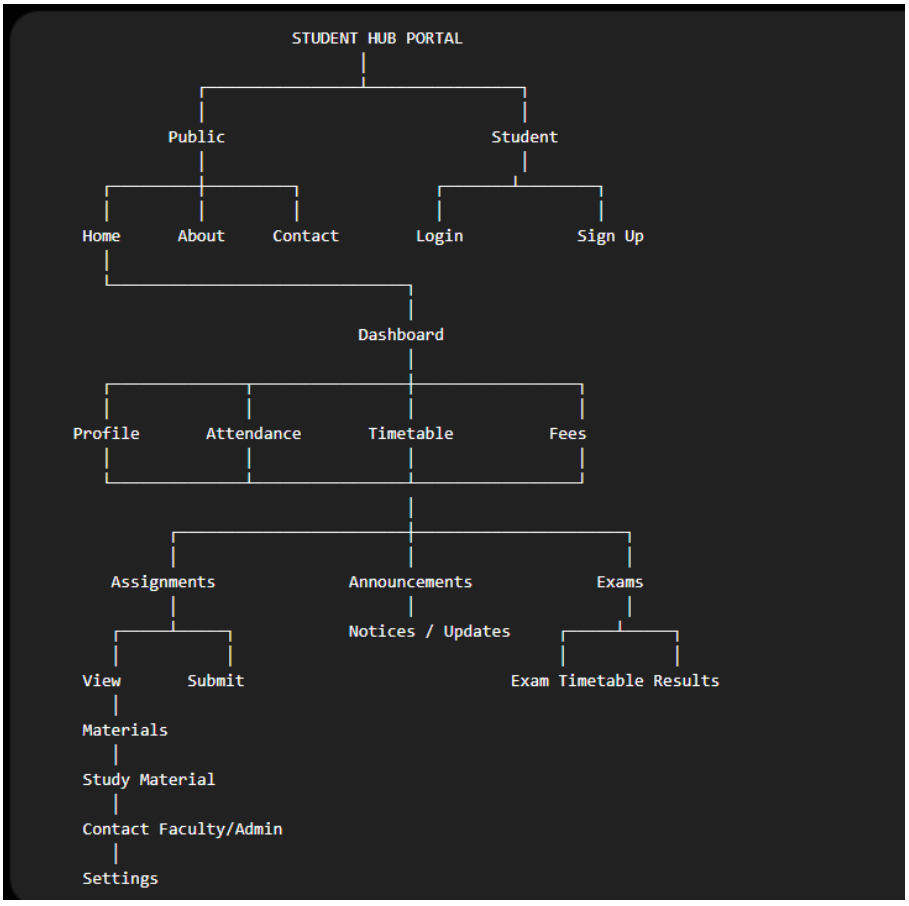
- Responsive design for mobile and desktop
- Secure login with authentication

- Fast page loading
- Easy-to-use interface
- Reliable system with minimal downtime
- Data privacy and security

Software Requirements:

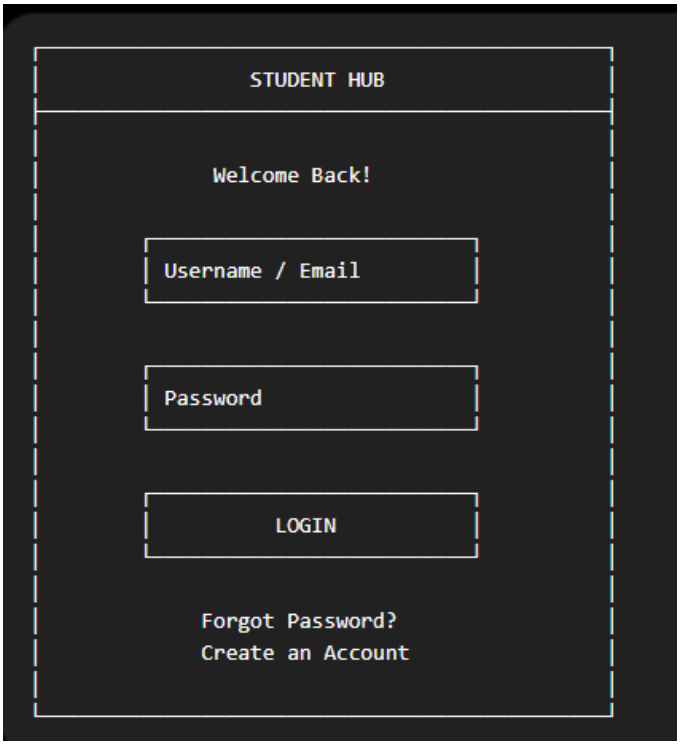
- HTML
- CSS
- JavaScript
- PHP/Node.js (or any backend technology)
- MySQL Database
- VS Code
- Google Chrome

SITEMAP:

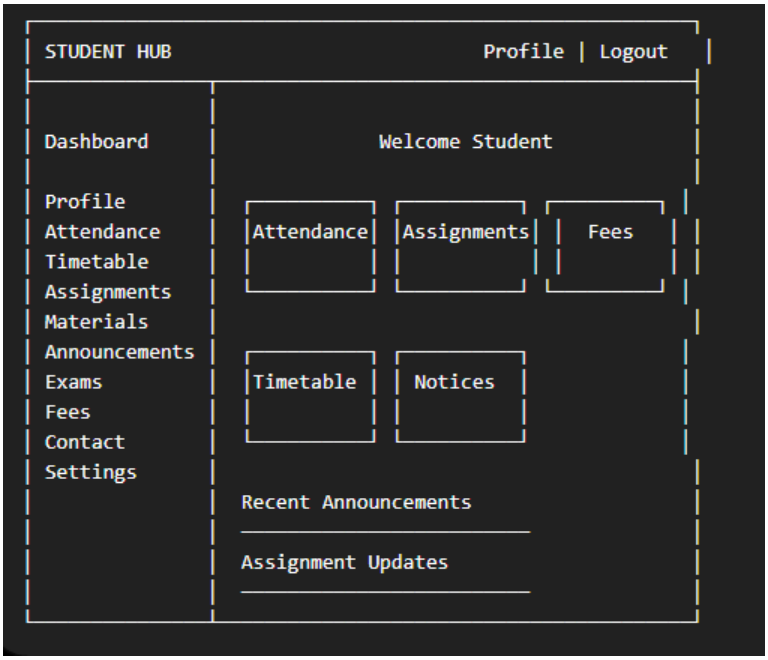


WIREFRAME :

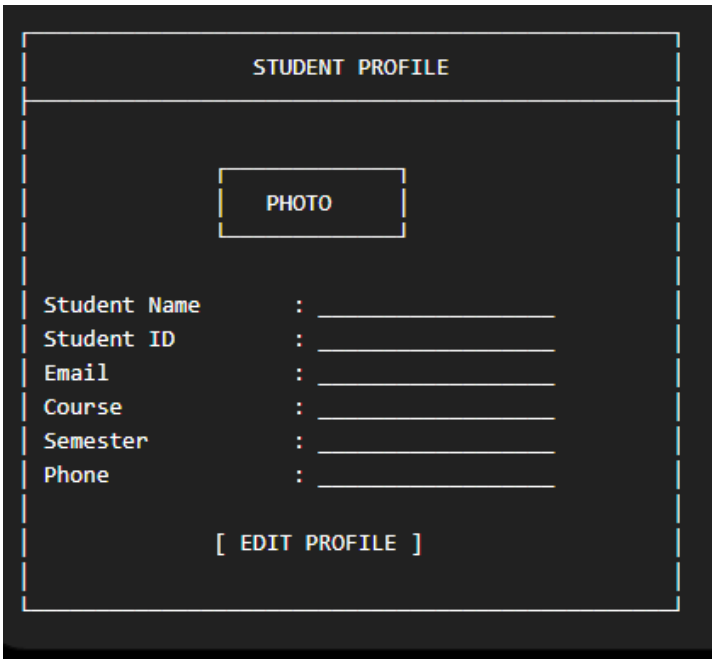
LOGIN :



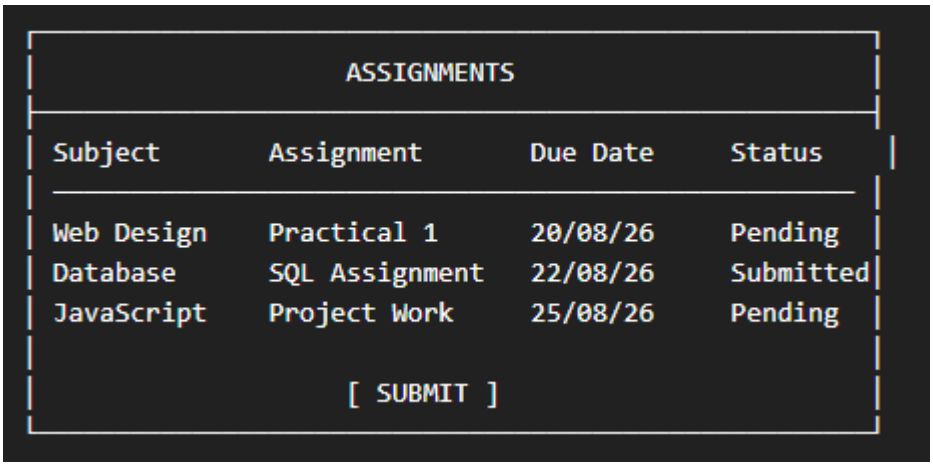
DASHBOARD:



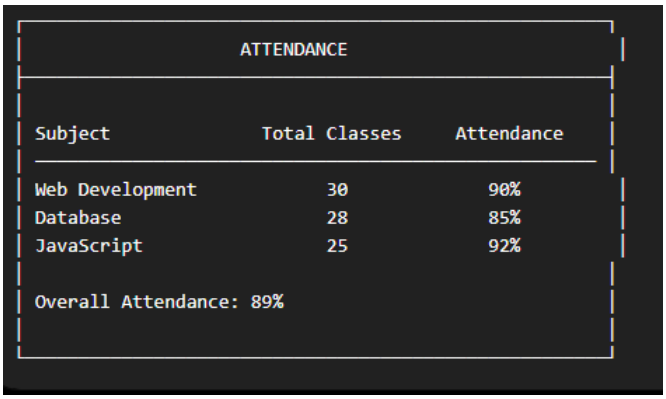
STUDENT PROFILE :



ASSIGNMENT :



ATTENDENCE :



TIMETABLE :

TIME TABLE					
Day	9-10	10-11	11-12	12-1	2-3
Monday	Web	DB	Java	Break	Maths
Tuesday	Java	Web	DB	Break	English
Wed.	Maths	Java	Web	Break	DB

KEY QUESTIONS :

1. What is URL and parts of URL?

➔ A URL (Uniform Resource Locator) is the address of a resource on the internet. It tells a web browser where to find a webpage or file and how to access it. The main parts of a URL are:

1. **Protocol (Scheme)**
2. **Domain Name (Host)**
3. **Path**
4. **Parameters**
5. **Anchor**

2. How does HTML file processed in web browser?

➔ When you open an HTML file in a browser, it is processed roughly as follows:

1. **Request / Loading**
2. **Parsing HTML**
3. **Processing CSS and JavaScript**
4. **Rendering the Page**

3. How will page navigation flow managed among all HTML pages?

➔ Page navigation is managed using hyperlinks (<a> tags) in HTML. Each link contains the URL or filename of another HTML page.

Home → Login → Dashboard → Profile / Attendance / Timetable / Assignments / Fees / Exams → Logout

4.How will GitHub commits be maintained after each practical?

➔ GitHub commits are maintained after completing each practical by saving the changes in the Git repository with a meaningful commit message. The basic process is:

1. **Make changes to the project**
2. **Test the changes**

3. **Add the changed files**
4. **Create a commit with a meaningful message**
5. **Push the commit to GitHub**

PRACTICAL-2

AIM : Develop static HTML5 skeletons for at least 10 StudentHub pages such as Home, About, Register, Login, Dashboard, Events, Profile, Contact, Admin, FAQ, and Feedback. Use semantic tags and accessibility-friendly structure.

INDEX.HTML :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>StudentHub</title>

</head>
<body>
  <header class="pagelist">
    <img src="./studenthub.png" class="studenthub" alt="StudentHub Logo">
    <h1>Welcome to StudentHub</h1>
    <nav>
      <a href="./login.html">Login</a>
      <a href="./signup.html">Sign Up</a>
    </nav>
  </header>

  <main>
    <p>Your one-stop portal for attendance, timetables, fees, announcements,
assignments, and more.</p>
  </main>

  <footer>
    <p>&copy; 2026 Student Hub Portal</p>
  </footer>
</body>
</html>
```

PREVIEW:



Welcome to StudentHub

Login Sign Up

Your one-stop portal for attendance, timetables, fees, announcements, assignments, and more.

© 2026 Student Hub Portal

LOGIN.HTML :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login - Student Hub Portal</title>
</head>
<body>
  <header>
    <h1>Student Hub Portal</h1>
  </header>

  <main>
    <div class="auth-container">
    <h2>Login</h2>
    <form action="./home.html" method="post">


        <label for="username">Username:</label>
        <input type="text" id="username" name="username" required><br><br>

        <label for="password">Password:</label>
        <input type="password" id="password" name="password" required><br><br>
```

```
        <label>
          <input type="checkbox" name="remember"> Remember Me
        </label><br><br>

        <input type="submit" value="Login">
       </form>
      <div class="auth-links">
        <p><a href="./signup.html">Don't have an account? Sign up here</a></p>
        <p><a href="./forgot-password.html">Forgot Password?</a></p>
      </div>
    </div>
  </main>

  <footer>
    <p>&copy; 2026 Student Hub Portal</p>
  </footer>
</body>
</html>
```

PREVIEW :

Student Hub Portal

Login

Username:

Password:

Remember Me

Login

Don't have an account? Sign up here

Forgot Password?

© 2026 Student Hub Portal

SIGN-UP.HTML :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sign Up - Student Hub Portal</title>
  <!-- <link rel="stylesheet" type="text/css" href="class.css"> -->
</head>
<body>
  <header>
    <h1>Student Hub Portal</h1>
  </header>

  <main>
    <div class="auth-container">
    <h2>Create Your Account</h2>
    <form action="./home.html" method="post">
      <fieldset>
        <legend>Sign Up</legend>

        <label for="username">Username:</label>
        <input type="text" id="username" name="username" required><br><br>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required><br><br>

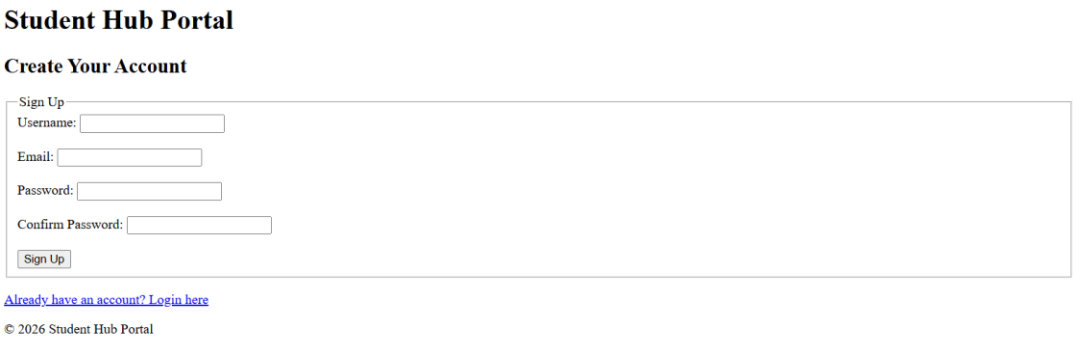
        <label for="password">Password:</label>
        <input type="password" id="password" name="password" required
autocomplete="new-password"><br><br>

        <label for="confirm-password">Confirm Password:</label>
        <input type="password" id="confirm-password" name="confirm-password"
required><br><br>

        <input type="submit" value="Sign Up">
      </fieldset>
    </form>
    <div class="auth-links">
    <p><a href="./login.html">Already have an account? Login here</a></p>
    </div>
    </div>
  </main>

  <footer>
    <p>&copy; 2026 Student Hub Portal</p>
  </footer>
</body>
</html>
```

PREVIEW :



HOME :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Home - Student Hub Portal</title>
</head>
<body>
  <header>
    <h1>Student Hub Portal</h1>
    <nav>

      <a href="./profile.html">Profile</a>
      <a href="./dashboard.html">Dashboard</a>
      <a href="./contact.html">Contact Us</a>
      <a href="./aboutus.html">About Us</a>
      <a href="./settings.html">Settings</a>
      <a href="./index.html">Logout</a>

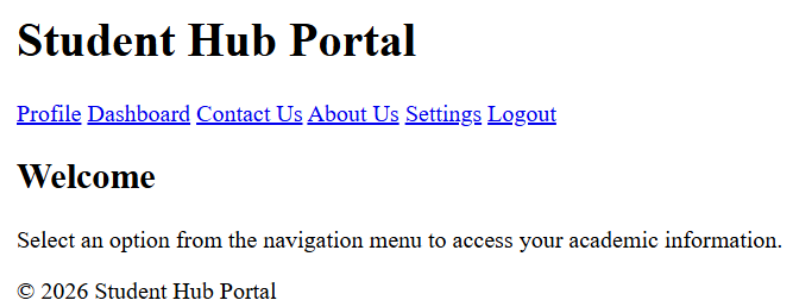
    </nav>
  </header>

  <main>
    <h2>Welcome</h2>
    <p>Select an option from the navigation menu to access your academic
information.</p>
  </main>

  <footer>
    <p>&copy; 2026 Student Hub Portal</p>
  </footer>
```

```
</body>
</html>
```

PREVIEW:



PROFILE.HTML :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Profile - Student Hub Portal</title>
</head>
<body>
  <header>
    <h1>Student Hub Portal</h1>
    <nav>
      <a href="./home.html">Home</a>
      <a href="./dashboard.html">Dashboard</a>
      <a href="./contact.html">Contact Us</a>
      <a href="./settings.html">Settings</a>
      <a href="./index.html">Logout</a>
    </nav>
  </header>

  <main>
    <h2>My Profile</h2>
    <section class="profile-info">
      <img src="./default-profile.png" alt="Profile Picture" class="profile-pic">
      <p><strong>Name:</strong> AARAV HITESHKUMAR PATEL</p>
      <p><strong>Roll Number:</strong> 25DCE072</p>
      <p><strong>Course:</strong> B.Tech Computer Engineering</p>
      <p><strong>Semester:</strong> 3</p>
      <p><strong>Email:</strong> 25dce072@charusat.edu.in</p>
```

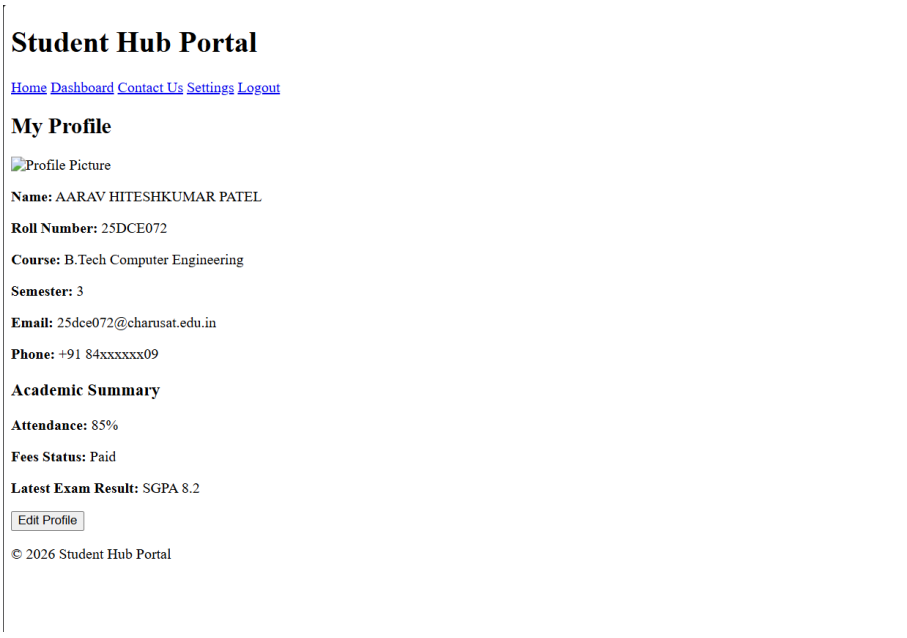
```
      <p><strong>Phone:</strong> +91 84xxxxxx09</p>
    </section>

    <section class="academic-summary">
      <h3>Academic Summary</h3>
      <p><strong>Attendance:</strong> 85%</p>
      <p><strong>Fees Status:</strong> Paid</p>
      <p><strong>Latest Exam Result:</strong> SGPA 8.2</p>
    </section>

    <button onclick="window.location.href='./edit-profile.html'">Edit
Profile</button>
  </main>

  <footer>
    <p>&copy; 2026 Student Hub Portal</p>
  </footer>
</body>
</html>
```

PREVIEW :



DASHBOARD.HTML :

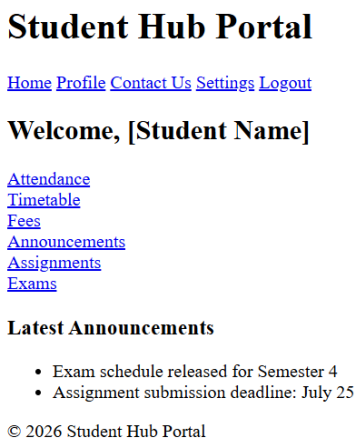
```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard - Student Hub Portal</title>
</head>
<body>
  <header>
    <h1>Student Hub Portal</h1>
    <nav>
      <a href="./home.html">Home</a>
      <a href="./profile.html">Profile</a>
      <a href="./contact.html">Contact Us</a>
      <a href="./settings.html">Settings</a>
      <a href="./index.html">Logout</a>
    </nav>
  </header>

  <main>
    <h2>Welcome, [Student Name]</h2>
    <section class="dashboard-cards">
      <div class="card"><a href="./attendance.html">Attendance</a></div>
      <div class="card"><a href="./timetable.html">Timetable</a></div>
      <div class="card"><a href="./fees.html">Fees</a></div>
      <div class="card"><a href="./announcement.html">Announcements</a></div>
      <div class="card"><a href="./assignment.html">Assignments</a></div>
      <div class="card"><a href="./exam.html">Exams</a></div>
    </section>

    <section class="notifications">
      <h3>Latest Announcements</h3>
      <ul>
        <li>Exam schedule released for Semester 4</li>
        <li>Assignment submission deadline: July 25</li>
      </ul>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Student Hub Portal</p>
  </footer>
</body>
</html>
```

PREVIEW :



Student Hub Portal

Home Profile Contact Us Settings Logout

Welcome, [Student Name]

Attendance
Timetable
Fees
Announcements
Assignments
Exams

Latest Announcements

- Exam schedule released for Semester 4
- Assignment submission deadline: July 25

© 2026 Student Hub Portal

ATTENDANCE.HTML :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Attendance - Student Hub Portal</title>
</head>
<body>
  <header>
    <h1>Student Hub Portal</h1>
    <nav>
      <a href="./home.html">Home</a>
      <a href="./dashboard.html">Dashboard</a>
      <a href="./profile.html">Profile</a>
      <a href="./contact.html">Contact Us</a>
      <a href="./settings.html">Settings</a>
      <a href="./index.html">Logout</a>
    </nav>
  </header>

  <main>
    <h2>Attendance Record</h2>
    <section class="attendance-summary">
      <p><strong>Overall Attendance:</strong> 85%</p>
      <div class="progress-bar">
        <div class="progress" style="width:85%"></div>
```

```
    </div>
  </section>

  <section class="attendance-details">
    <h3>Subject-wise Attendance</h3>
    <table>
      <thead>
        <tr>
          <th>Subject</th>
          <th>Total Classes</th>
          <th>Attended</th>
          <th>Percentage</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>Web Development</td>
          <td>40</td>
          <td>34</td>
          <td>85%</td>
        </tr>
                  <tr>
          <td>Data Structures</td>
          <td>35</td>
          <td>30</td>
          <td>86%</td>
        </tr>
        <tr>
          <td>Database Management</td>
          <td>28</td>
          <td>24</td>
          <td>85%</td>
        </tr>
        <tr>
          <td>Operating Systems</td>
          <td>32</td>
          <td>26</td>
          <td>81%</td>
        </tr>
        <tr>
          <td>Computer Networks</td>
          <td>30</td>
          <td>27</td>
          <td>90%</td>
        </tr>
      </tbody>
    </table>
  </section>
</main>
```

```
  <footer>
    <p>&copy; 2026 Student Hub Portal. All rights reserved.</p>
  </footer>
</body>
</html>
```

PREVIEW :

Student Hub Portal

Home Dashboard Profile Contact Us Settings Logout

Attendance Record

Overall Attendance: 85%

Subject-wise Attendance

Subject	Total Classes	Attended	Percentage
Web Development	40	34	85%
Data Structures	35	30	86%
Database Management	28	24	85%
Operating Systems	32	26	81%
Computer Networks	30	27	90%

© 2026 Student Hub Portal. All rights reserved.

TIMETABLE.HTML :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Timetable - Student Hub Portal</title>
</head>
<body>
  <header>
    <h1>Student Hub Portal</h1>
    <nav>
      <a href="./home.html">Home</a>
      <a href="./dashboard.html">Dashboard</a>
      <a href="./profile.html">Profile</a>
      <a href="./contact.html">Contact Us</a>
```

```
    <a href="./settings.html">Settings</a>
    <a href="./index.html">Logout</a>
  </nav>
</header>

<main>
  <h2>Weekly Timetable</h2>
  <table class="timetable">
    <thead>
      <tr>
        <th>Day</th>
        <th>9:00 - 10:00</th>
        <th>10:00 - 11:00</th>
        <th>11:00 - 12:00</th>
        <th>1:00 - 2:00</th>
        <th>2:00 - 3:00</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Monday</td>
        <td>Web Development</td>
        <td>Data Structures</td>
        <td>Database Systems</td>
        <td>Maths</td>
        <td>Elective</td>
      </tr>
      <tr>
        <td>Tuesday</td>
        <td>Operating Systems</td>
        <td>Computer Networks</td>
        <td>Web Development</td>
        <td>Elective</td>
        <td>Maths</td>
      </tr>
      <tr>
        <td>Wednesday</td>
        <td>Data Structures</td>
        <td>Database Systems</td>
        <td>Operating Systems</td>
        <td>Computer Networks</td>
        <td>Elective</td>
      </tr>
      <tr>
        <td>Thursday</td>
        <td>Web Development</td>
        <td>Maths</td>
        <td>Database Systems</td>
        <td>Operating Systems</td>
```

```
          <td>Computer Networks</td>
        </tr>
        <tr>
          <td>Friday</td>
          <td>Elective</td>
          <td>Web Development</td>
          <td>Data Structures</td>
          <td>Database Systems</td>
          <td>Maths</td>
        </tr>
        <tr>
          <td>Saturday</td>
          <td>Computer Networks</td>
          <td>Operating Systems</td>
          <td>Elective</td>
          <td>Web Development</td>
          <td>Seminar</td>
        </tr>
      </tbody>
    </table>
  </main>

  <footer>
    <p>&copy; 2026 Student Hub Portal. All rights reserved.</p>
  </footer>
</body>
</html>
```

PREVIEW :

Student Hub Portal

Home Dashboard Profile Contact Us Settings Logout

Weekly Timetable

Day	9:00 - 10:00	10:00 - 11:00	11:00 - 12:00	1:00 - 2:00	2:00 - 3:00
Monday	Web Development	Data Structures	Database Systems	Maths	Elective
Tuesday	Operating Systems	Computer Networks	Web Development	Elective	Maths
Wednesday	Data Structures	Database Systems	Operating Systems	Computer Networks	Elective
Thursday	Web Development	Maths	Database Systems	Operating Systems	Computer Networks
Friday	Elective	Web Development	Data Structures	Database Systems	Maths
Saturday	Computer Networks	Operating Systems	Elective	Web Development	Seminar

© 2026 Student Hub Portal. All rights reserved.

FEES.HTML :

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Fees - Student Hub Portal</title>
</head>
<body>
  <header>
    <h1>Student Hub Portal</h1>
    <nav>
      <a href="./home.html">Home</a>
      <a href="./dashboard.html">Dashboard</a>
      <a href="./profile.html">Profile</a>
      <a href="./contact.html">Contact Us</a>
      <a href="./settings.html">Settings</a>
      <a href="./index.html">Logout</a>
    </nav>
  </header>

  <main>
    <h2>Fees Details</h2>
    <section class="fees-summary">
      <p><strong>Total Fees:</strong> ₹70,000</p>
      <p><strong>Paid:</strong> ₹40,000</p>
      <p><strong>Balance:</strong> ₹30,000</p>
      <p><strong>Next Due Date:</strong> 30 August 2026</p>
    </section>
```

```
    <section class="payment-history">
      <h3>Payment History</h3>
      <table>
        <thead>
          <tr>
            <th>Date</th>
            <th>Amount</th>
            <th>Mode</th>
            <th>Status</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>01 Jan 2026</td>
            <td>₹40,000</td>
            <td>Online</td>
            <td>Paid</td>
          </tr>
          <tr>
            <td>30 Aug 2026</td>
            <td>₹30,000</td>
            <td>Offline</td>
            <td>Due</td>
          </tr>
        </tbody>
      </table>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Student Hub Portal. All rights reserved.</p>
  </footer>
</body>
</html>
```

PREVIEW :

Student Hub Portal

Home Dashboard Profile Contact Us Settings Logout

Fees Details

Total Fees: ₹70,000

Paid: ₹40,000

Balance: ₹30,000

Next Due Date: 30 August 2026

Payment History

Date	Amount	Mode	Status
01 Jan 2026	₹40,000	Online	Paid
30 Aug 2026	₹30,000	Offline	Due

© 2026 Student Hub Portal. All rights reserved.

ANNOUNCEMENT.HTML :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Announcements</title>

</head>
<body>

  <header>
    <h1>Student Hub Portal</h1>
    <nav>
      <a href="./home.html">Home</a>
      <a href="./dashboard.html">Dashboard</a>
      <a href="./profile.html">Profile</a>
      <a href="./contact.html">Contact Us</a>
      <a href="./settings.html">Settings</a>
      <a href="./index.html">Logout</a>
    </nav>
  </header>
    <div class="auth messanger">
        <h2>Announcements</h2>
        <div class="announcement-box">
            <h3>Announcement 1</h3>
```

```
            <p>This is first announcement.</p>
        </div>
        <div class="announcement-box">
            <h3>Announcement 2</h3>
            <p>This is second announcement.</p>
        </div>
        <div class="announcement-box">
            <h3>Announcement 3</h3>
            <p>This is third announcement.</p>
        </div>
        <div class="announcement-box">
            <h3>Announcement 4</h3>
            <p>This is fourth announcement.</p>
        </div>
    </div>
    <footer>
    <h2>Contact Us</h2>
    <p>You can reach us at:
      <a href="mailto:25dce072@charusat.edu.in">25dce072@charusat.edu.in</a>
    </p>
  </footer>
</body>
</html>
```

PREVIEW :

Student Hub Portal

Home Dashboard Profile Contact Us Settings Logout

Announcements

Announcement 1

This is first announcement.

Announcement 2

This is second announcement.

Announcement 3

This is third announcement.

Announcement 4

This is fourth announcement.

Contact Us

You can reach us at: 25dce072@charusat.edu.in

ASSIGNMENTS.HTML :

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Assignment</title>
</head>
<body>
    <header>
    <h1>Student Hub Portal</h1>
    <nav>
      <a href="./home.html">Home</a>
      <a href="./dashboard.html">Dashboard</a>
      <a href="./profile.html">Profile</a>
      <a href="./contact.html">Contact Us</a>
      <a href="./settings.html">Settings</a>
      <a href="./index.html">Logout</a>
    </nav>
  </header>
    <div class="content">
    <h1>Welcome to My Assignment</h1>
    <p>This is a sample webpage created for practice. It includes basic HTML elements
styled with CSS.</p>

    <h2>Topics Covered</h2>
    <ul>
        <li>HTML Structure</li>
        <li>CSS Styling</li>
        <li>Lists and Buttons</li>
    </ul>

    <button>Click Me</button>
    </div>
  <footer>
    <h2>Contact Us</h2>
    <p>You can reach us at:
      <a href="mailto:25dce072@charusat.edu.in">25dce072@charusat.edu.in</a>
    </p>
  </footer>
</body>
</html>
```

PREVIEW :

Student Hub Portal

Home Dashboard Profile Contact Us Settings Logout

Welcome to My Assignment

This is a sample webpage created for practice. It includes basic HTML elements styled with CSS.

Topics Covered

- HTML Structure
- CSS Styling
- Lists and Buttons

Click Me

Contact Us

You can reach us at: 25dce072@charusat.edu.in

EXAMS.HTML :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exams - Student Hub Portal</title>
</head>
<body>
  <header>
    <h1>Student Hub Portal</h1>
    <nav>
      <a href="./home.html">Home</a>
      <a href="./dashboard.html">Dashboard</a>
      <a href="./profile.html">Profile</a>
      <a href="./contact.html">Contact Us</a>
      <a href="./settings.html">Settings</a>
      <a href="./index.html">Logout</a>
    </nav>
  </header>

  <div class="exam-container">
    <h2>Upcoming Exams</h2>
    <table>
      <tr>
        <th>Subject</th>
        <th>Date</th>
```

```
        <th>Time</th>
      </tr>
      <tr>
        <td>Discrete Mathematics</td>
        <td>10 August 2026</td>
        <td>10:00 AM</td>
      </tr>
      <tr>
        <td>Fundamentals of Data Structure and Algorithm</td>
        <td>12 August 2026</td>
        <td>2:00 PM</td>
      </tr>
      <tr>
        <td>Fundamentals of Computer Networks</td>
        <td>14 August 2026</td>
        <td>9:00 AM</td>
      </tr>
      <tr>
        <td>Web Development Frameworks</td>
        <td>16 August 2026</td>
        <td>11:00 AM</td>
      </tr>
      <tr>
        <td>Object Oriented Programming</td>
        <td>18 August 2026</td>
        <td>1:00 PM</td>
      </tr>
    </table>
  </div>

  <footer>
    <p>&copy; 2026 Student Hub Portal. All rights reserved.</p>
  </footer>
</body>
</html>
```

PREVIEW :

Student Hub Portal

Home Dashboard Profile Contact Us Settings Logout

Upcoming Exams

Subject	Date	Time
Discrete Mathematics	10 August 2026	10:00 AM
Fundamentals of Data Structure and Algorithm	12 August 2026	2:00 PM
Fundamentals of Computer Networks	14 August 2026	9:00 AM
Web Development Frameworks	16 August 2026	11:00 AM
Object Oriented Programming	18 August 2026	1:00 PM

© 2026 Student Hub Portal. All rights reserved.

CONTACT.HTML :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contact Us - Student Hub Portal</title>
</head>
<body>
  <header>
    <h1>Student Hub Portal</h1>
    <nav>
      <a href="./home.html">Home</a>
      <a href="./dashboard.html">Dashboard</a>
      <a href="./profile.html">Profile</a>
      <a href="./aboutus.html">About Us</a>
      <a href="./settings.html">Settings</a>
      <a href="./index.html">Logout</a>
    </nav>
  </header>

  <main>
    <h2>Contact Us</h2>
    <section class="contact-info">
      <p><strong>Email:</strong> studenthub@university.edu</p>
      <p><strong>Phone:</strong> +91 98765 43210</p>
      <p><strong>Address:</strong> Student Hub Office, University Campus, Petlad,
Gujarat, India</p>
    </section>

    <section class="contact-form">
```

```
      <h3>Send us a message</h3>
      <form>
        <label for="name">Name:</label><br>
        <input type="text" id="name" name="name" required><br><br>

        <label for="email">Email:</label><br>
        <input type="email" id="email" name="email" required><br><br>

        <label for="message">Message:</label><br>
        <textarea id="message" name="message" rows="5" required></textarea><br><br>

        <button type="submit">Submit</button>
      </form>
    </section>
  </main>
  <footer>
    <p>&copy; 2026 Student Hub Portal. All rights reserved.</p>
  </footer>
</body>
</html>
```

PREVIEW :

Student Hub Portal

Home Dashboard Profile About Us Settings Logout

Contact Us

Email: studenthub@university.edu

Phone: +91 98765 43210

Address: Student Hub Office, University Campus, Petlad, Gujarat, India

Send us a message

Name:

Email:

Message:

Submit

© 2026 Student Hub Portal. All rights reserved.

SETTINGS.HTML :

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Settings - Student Hub Portal</title>
</head>
<body>
  <header>
    <h1>Student Hub Portal</h1>
    <nav>
      <a href="./home.html">Home</a>
      <a href="./dashboard.html">Dashboard</a>
      <a href="./profile.html">Profile</a>
      <a href="./contact.html">Contact Us</a>
      <a href="./aboutus.html">About Us</a>
      <a href="./index.html">Logout</a>
    </nav>
  </header>

  <main>
    <h2>Settings</h2>

    <section class="account-settings">
      <h3>Account Settings</h3>
      <ul>
        <li><a href="./change-password.html">Change Password</a></li>
        <li><a href="./update-email.html">Update Email/Phone</a></li>
        <li><a href="./edit-profile.html">Edit Profile Picture</a></li>
      </ul>
    </section>

    <section class="preferences">
      <h3>Preferences</h3>
      <form>
        <label for="theme">Theme:</label>
        <select id="theme" name="theme">
          <option value="light">Light</option>
          <option value="dark">Dark</option>
        </select><br><br>

        <label for="language">Language:</label>
        <select id="language" name="language">
          <option value="en">English</option>
          <option value="hi">Hindi</option>
        </select><br><br>

        <label>
          <input type="checkbox" name="notifications" checked> Enable Notifications
        </label>
      </form>
```

```
    </section>

  </main>

  <footer>
    <p>&copy; 2026 Student Hub Portal</p>
  </footer>
</body>
</html>
```

PREVIEW :

Student Hub Portal

Home Dashboard Profile Contact Us About Us Logout

Settings

Account Settings

- Change Password
- Update Email/Phone
- Edit Profile Picture

Preferences

Theme: Light

Language: English

Enable Notifications

© 2026 Student Hub Portal

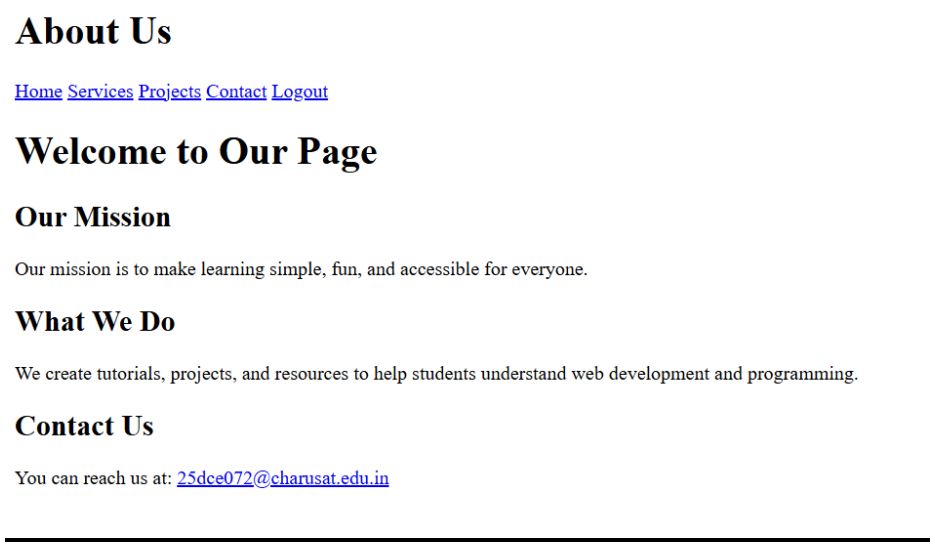
ABOUTUS.HTML :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>About Us</title>
</head>
<body>
  <header>
    <h1>About Us</h1>
    <nav>
      <a href="./home.html">Home</a>
      <a href="./services.html">Services</a>
```

```
      <a href="./projects.html">Projects</a>
      <a href="./contact.html">Contact</a>
      <a href="./index.html">Logout</a>
    </nav>
  </header>

  <div class="content">
    <h1>Welcome to Our Page</h1>
    <section>
      <h2>Our Mission</h2>
      <p>Our mission is to make learning simple, fun, and accessible for
everyone.</p>
    </section>
    <section>
      <h2>What We Do</h2>
      <p>We create tutorials, projects, and resources to help students understand web
development and programming.</p>
    </section>
  </div>
  <footer>
    <h2>Contact Us</h2>
    <p>You can reach us at:
      <a href="mailto:25dce072@charusat.edu.in">25dce072@charusat.edu.in</a>
    </p>
  </footer>
</body>
</html>
```

PREVIEW :



KEY QUESTIONS :

Q1. Are semantic tags such as header, nav, main, section, article, aside, and footer used properly?

➔ Yes, semantic HTML tags are used properly in most of the pages. The project uses tags such as <header>, <nav>, <main>, <section>, and <footer> to organize the page content.

1. <header> – Used for the page heading/logo area.
2. <nav> – Used for navigation links between pages.
3. <main> – Used for the main content of the webpage.
4. <section> – Used to divide content into meaningful sections.
5. <footer> – Used for footer information.
6. <article> and <aside> – These are not required on every page and are not widely used in the current project.

➔ Overall, the semantic structure is appropriate and can be improved further by using <article> and <aside> wherever their specific content structure requires them.

Q2. Are form labels, headings, image alt text, and navigation links accessible?

➔ Yes, the project includes important accessibility features.

1. **Form Labels** – Form fields such as name, email, username, password, and message have <label> elements associated with the input fields.
2. **Headings** – Headings such as <h1>, <h2>, and <h3> are used to organize the content.
3. **Image Alt Text** – Images include alt attributes, for example:

<img src="./studenthub.png" alt="StudentHub Logo">

1. **Navigation Links –** Navigation is provided using clear <a> links to move between different HTML pages.

➔ Therefore, the project follows basic accessibility practices, although the accessibility can be improved further by checking keyboard navigation, focus indicators, and meaningful link names.

Q3. Is the page structure consistent across all pages?

➔ Yes, the page structure is generally consistent across the Student Hub pages.

Most pages follow a common structure:

HEADER
↓
NAVIGATION
↓
MAIN CONTENT
↓
SECTIONS
↓
FOOTER

The common navigation contains links such as **Home, Dashboard, Profile, Contact, Settings, and Logout**. The pages also use the same CSS file (class.css) for common styling.

Therefore, the overall page structure and navigation are consistent, which makes the website easier for users to understand and navigate.

PRACTICAL-3

AIM: Design responsive layouts for Home, About, Registration, Dashboard, and Events pages using CSS Grid and Flexbox. Optional to use CSS framework such as Bootstrap or Tailwind. Ensure mobile-first responsiveness.

SOURCE CODE :

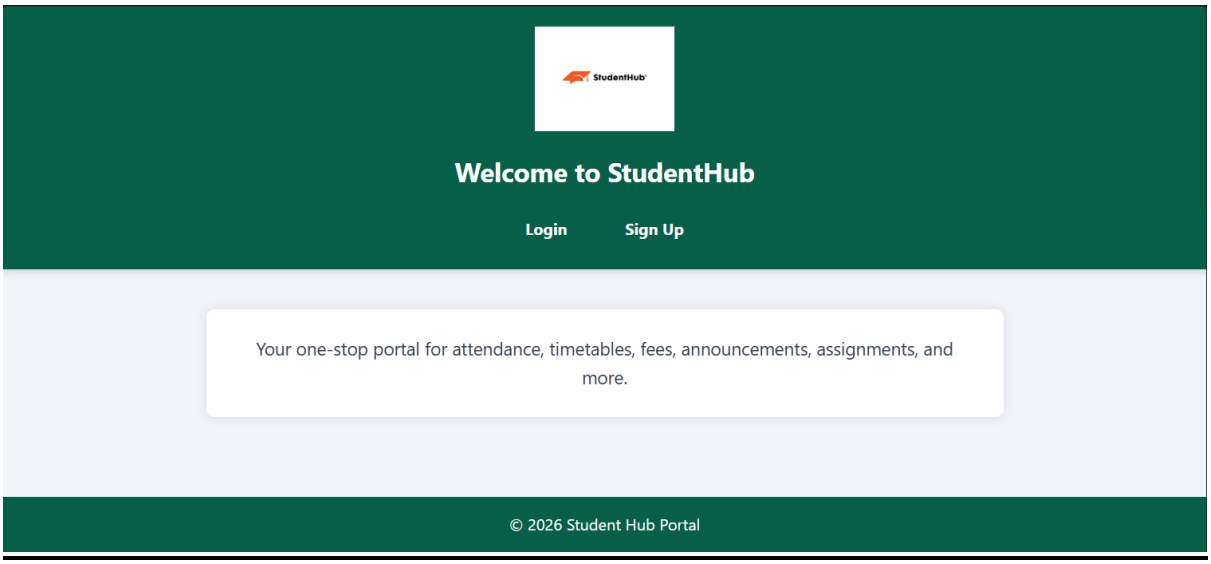
COMMON.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/common.css

INDEX.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/index.css

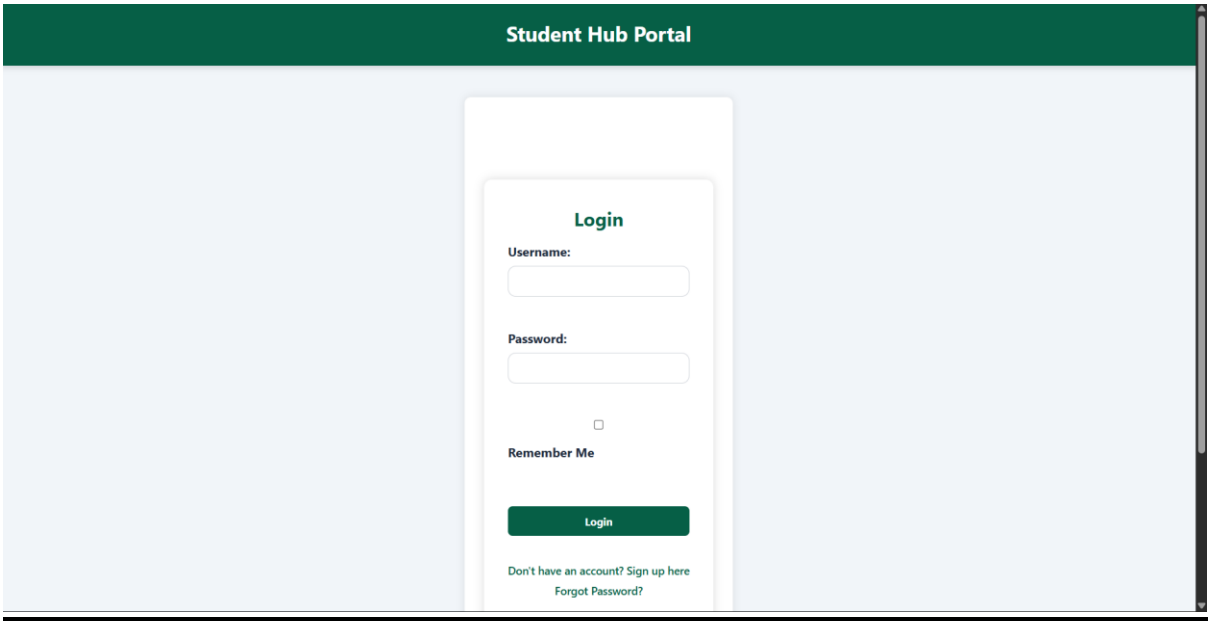
PREVIEW :



LOGIN.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/login.css

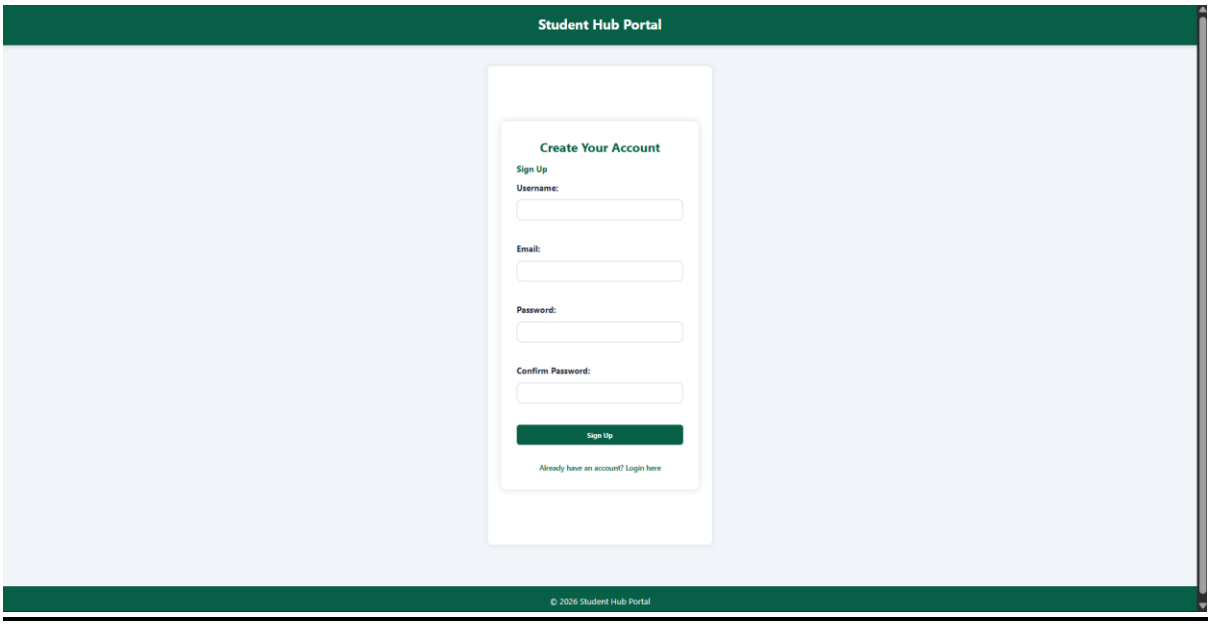
PREVIEW :



SIGNUP.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/signup.css

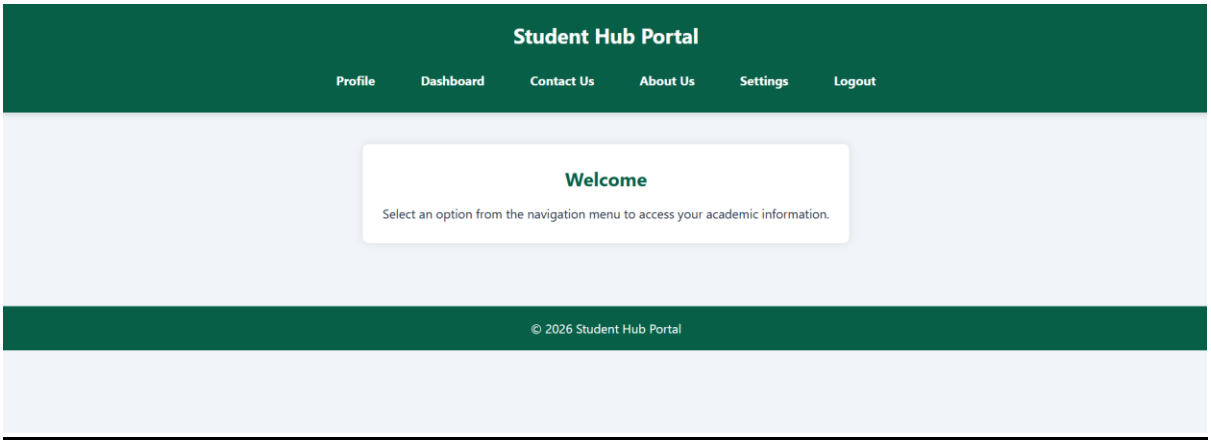
PREVIEW :



HOME.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/home.css

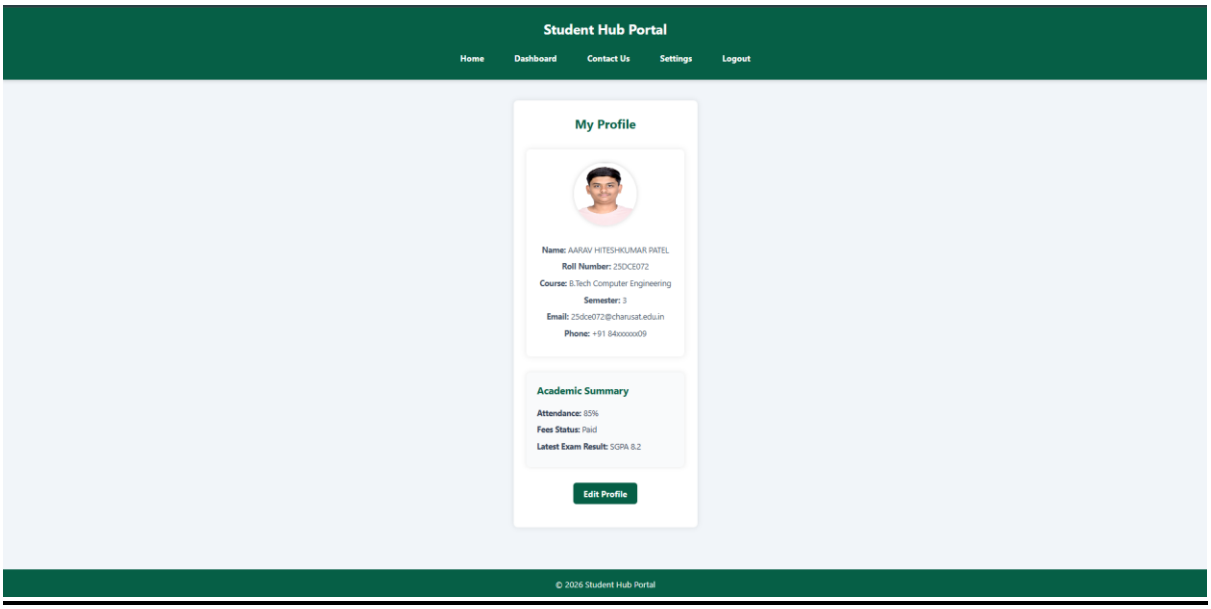
PREVIEW :



PROFILE.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/profile.css

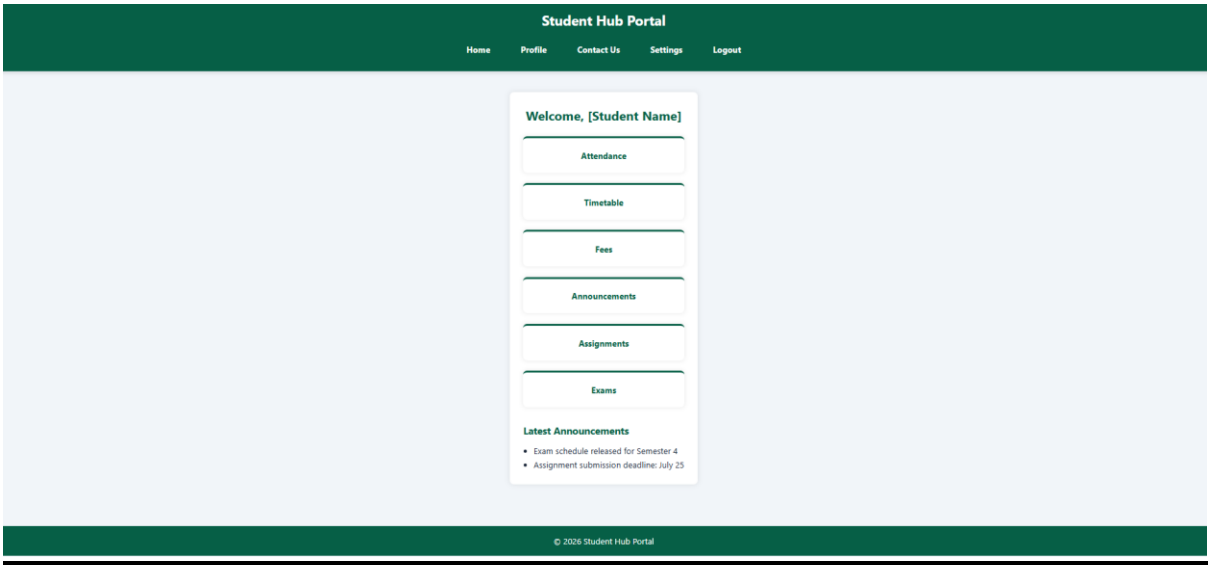
PREVIEW :



DASHBOARD.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/dashboard.css

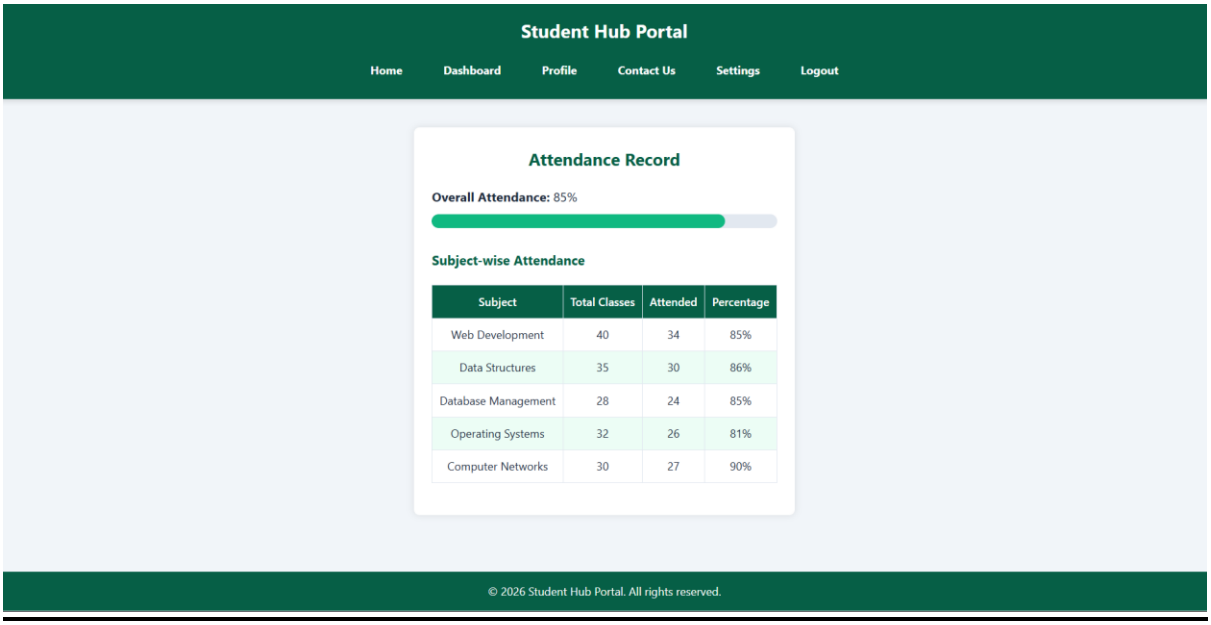
PREVIEW :



ATTENDENCE.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/attendance.css

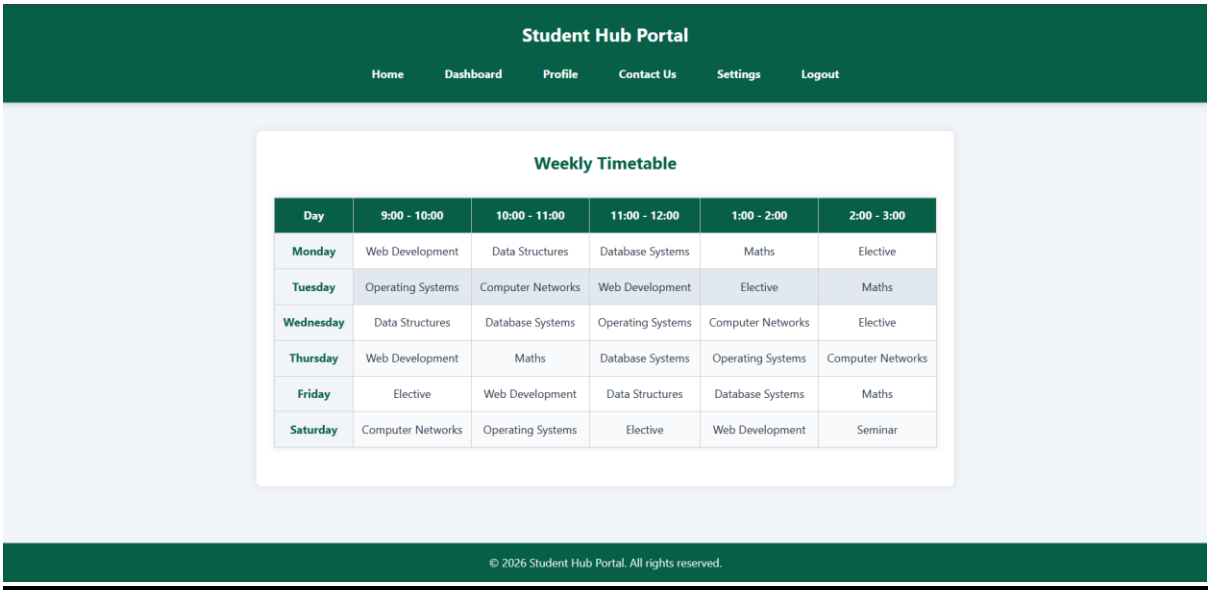
PREVIEW :



TIMETABLE.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/timetable.css

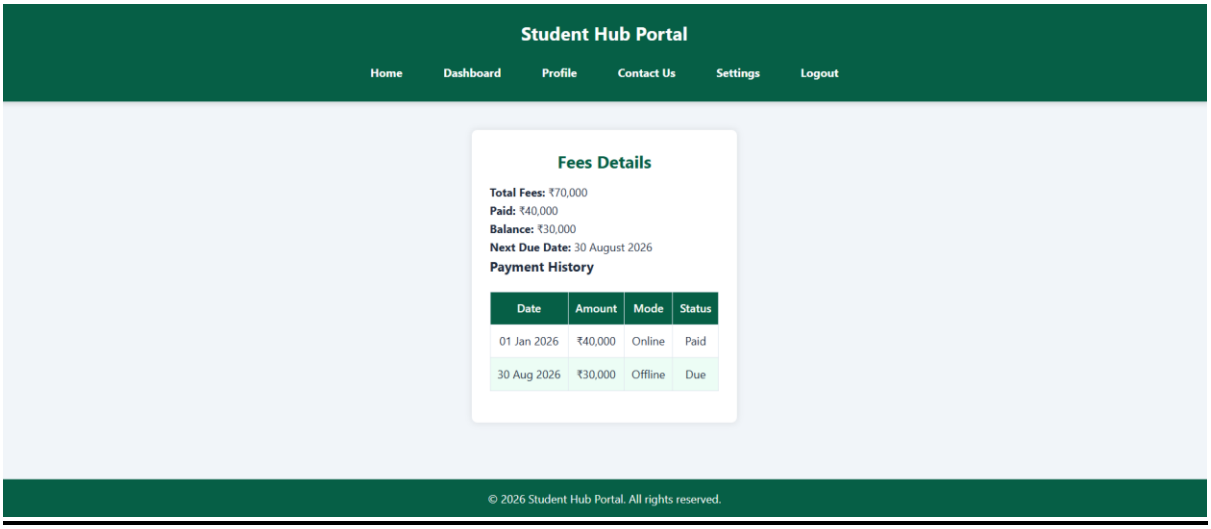
PREVIEW :



FEES.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/fees.css

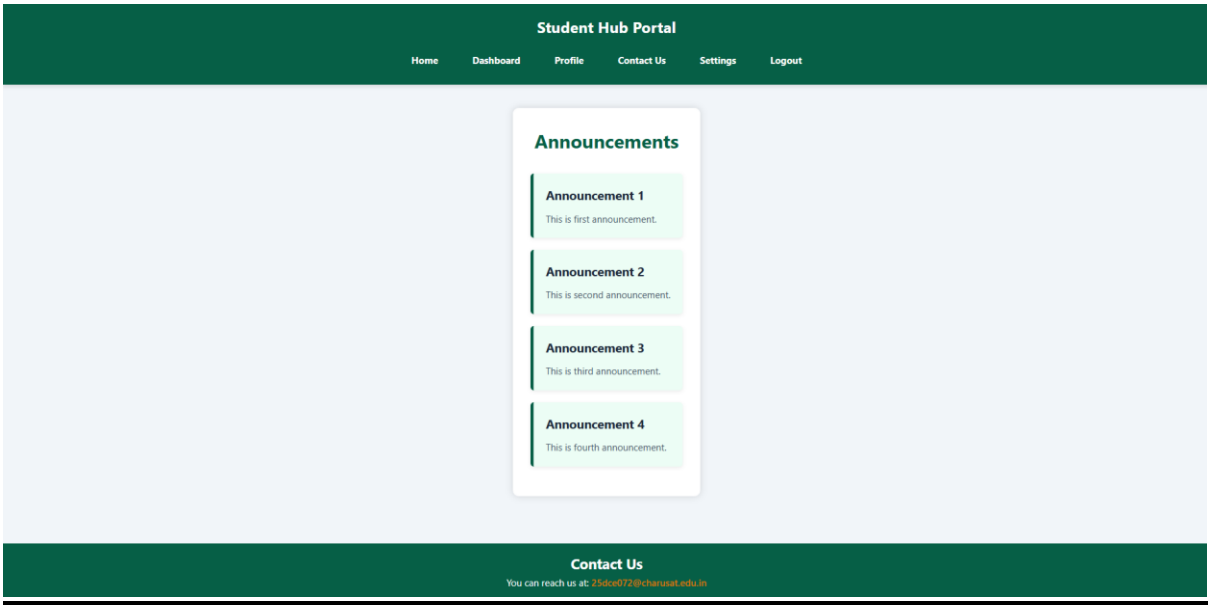
PREVIEW :



ANNOUNCEMENT.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/announcement.css

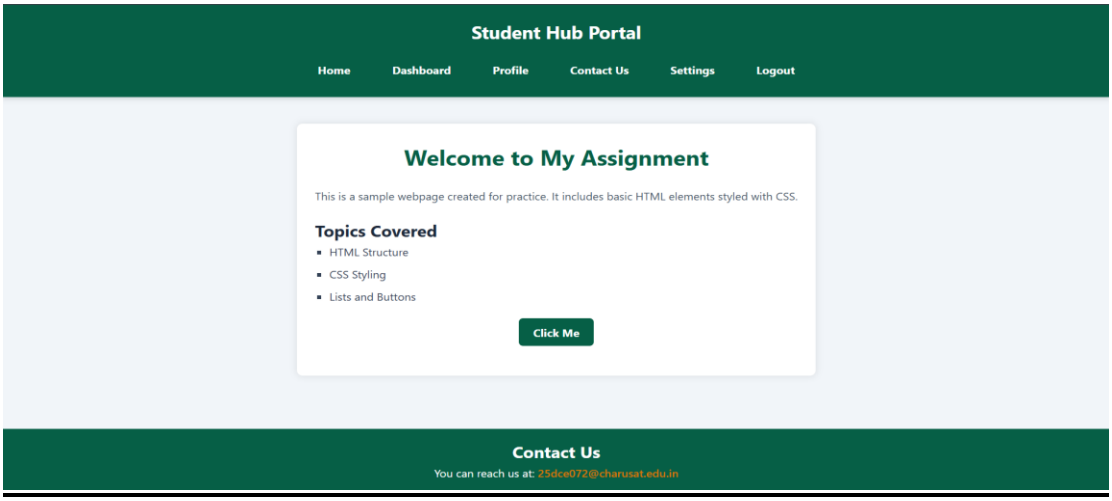
PREVIEW :



ASSIGNMENT.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/assignment.css

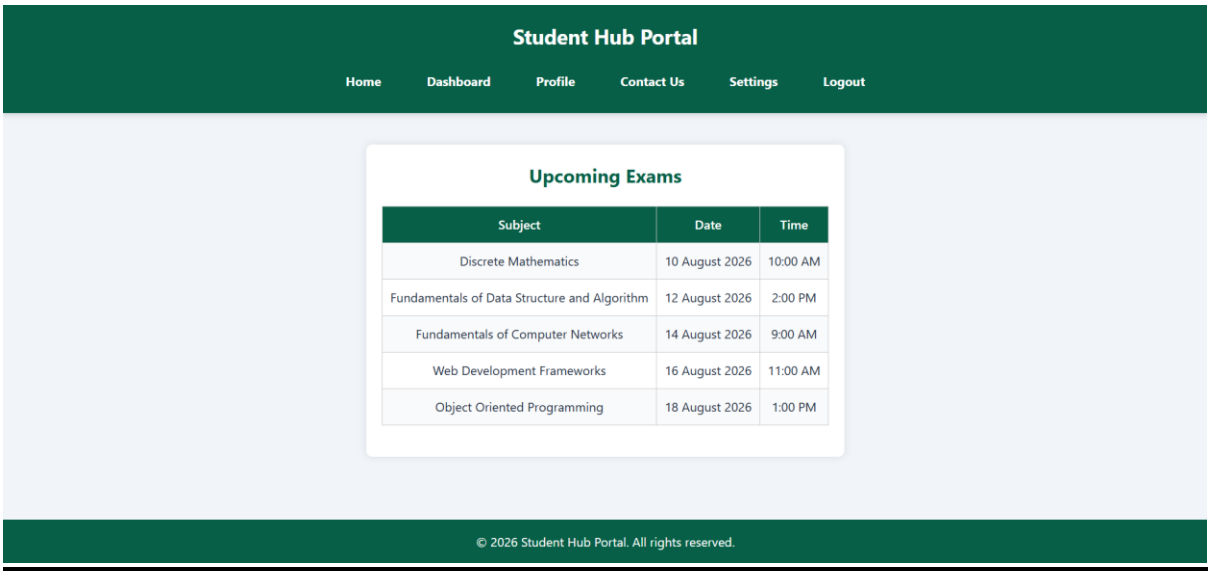
PREVIEW :



EXAM.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/exam.css

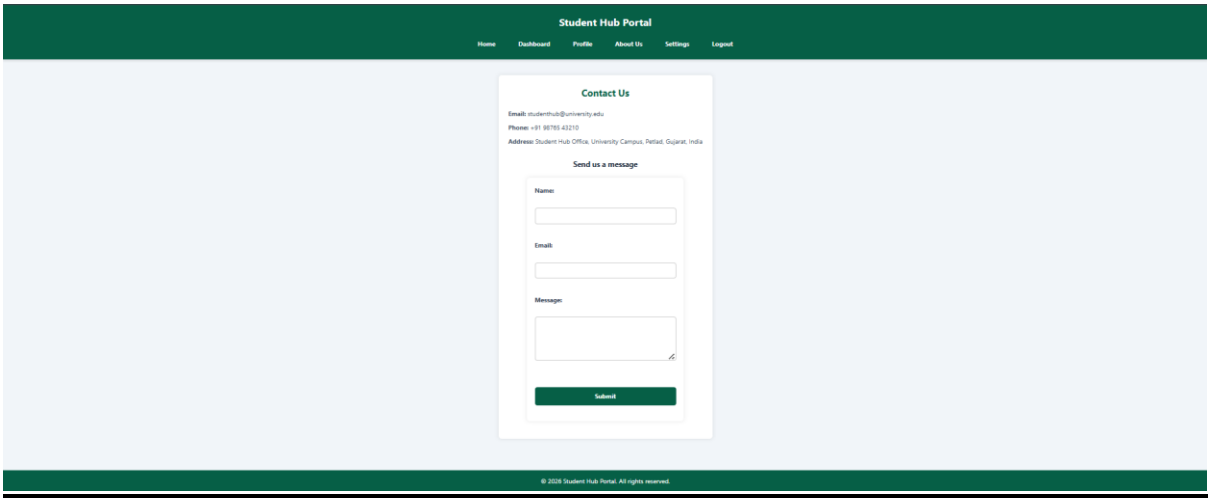
PREVIEW :



CONTACT.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/contact.css

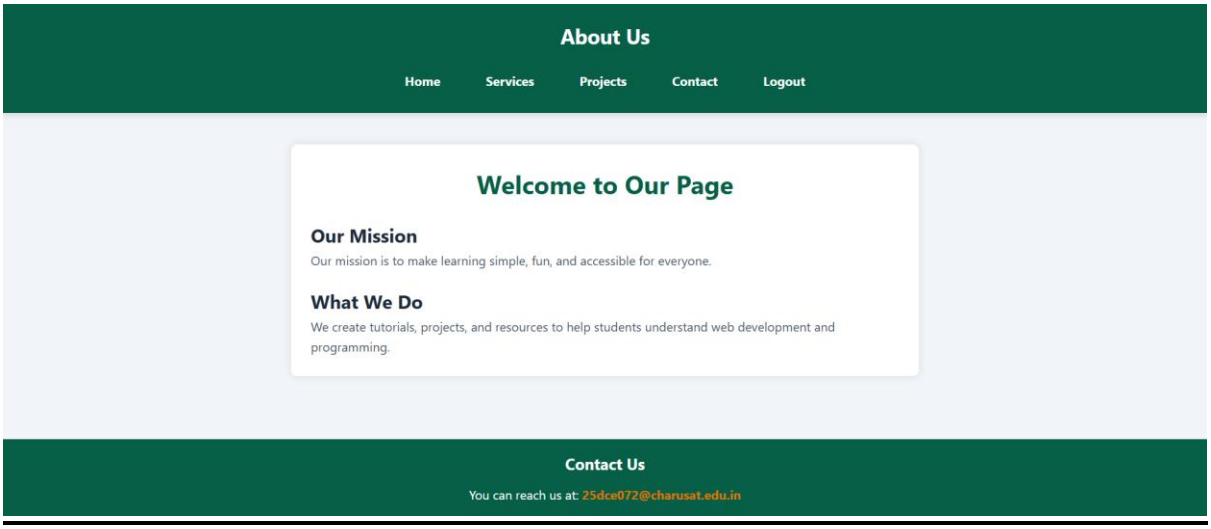
PREVIEW :



ABOUTUS.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/aboutus.css

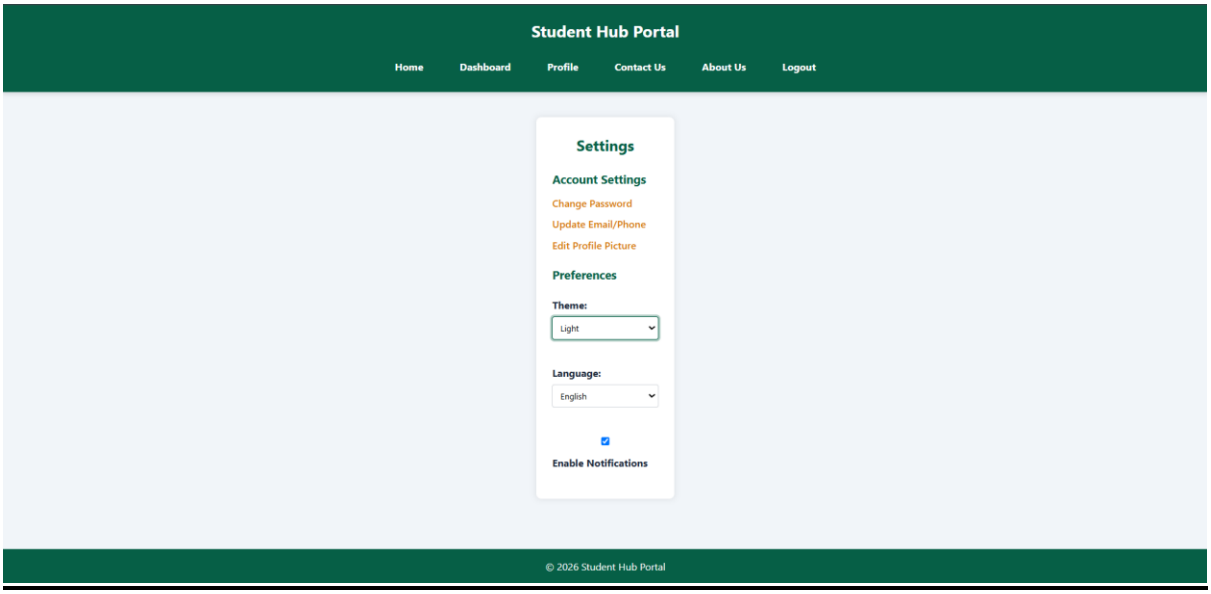
PREVIEW :



SETTINGS.CSS :

https://github.com/Aarav280907/WEB/blob/9a44b6229e079b0a26d7aa4ccb497a95c5fd2fae/Practical-1(Studenthub)/for_css/settings.css

PREVIEW :



KEY QUESTIONS :

1. How does the layout adapt for mobile, tablet, and desktop screens?

→The website uses responsive design. The layout changes according to the screen size.

- Mobile: Elements stack vertically and the content takes the available screen width.
- Tablet: Elements get more space and can be arranged in rows/columns.
- Desktop: More content can be displayed side-by-side.
- Bootstrap's responsive grid/classes help the website adjust automatically to different screen sizes.
- CSS properties such as width, max-width, display, flex, and responsive Bootstrap classes help maintain the layout.

2. Which layout technique is used and why?

→The project mainly uses Flexbox and Bootstrap's Grid System.

- Flexbox → useful for arranging items in a row or column and aligning them.
- Bootstrap Grid → divides the page into responsive columns.
- These techniques make the layout easier to manage and responsive.
- I had use **display:flex;** which is used for arrangement of elements of container flexibly.

3. Are typography, spacing, colors, and contrast readable?

→ **Yes.** The design uses CSS and Bootstrap styling to maintain readability.

- font-size → controls text size.
- font-family → controls the font.
- font-weight → controls text thickness.
- padding → provides internal spacing.
- margin → provides external spacing.
- color → controls text color.
- background-color → controls background.
- Good contrast between text and background makes the content easier to read.
- border-radius and box-shadow are used to make cards and sections visually clear.

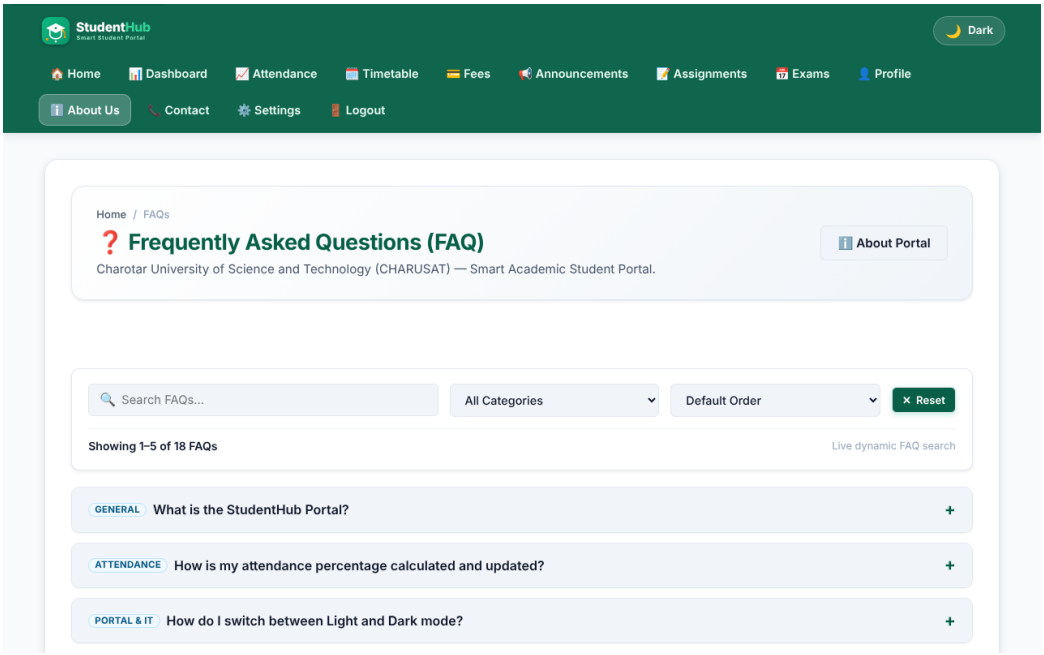
PRACTICAL-4

AIM: Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript.

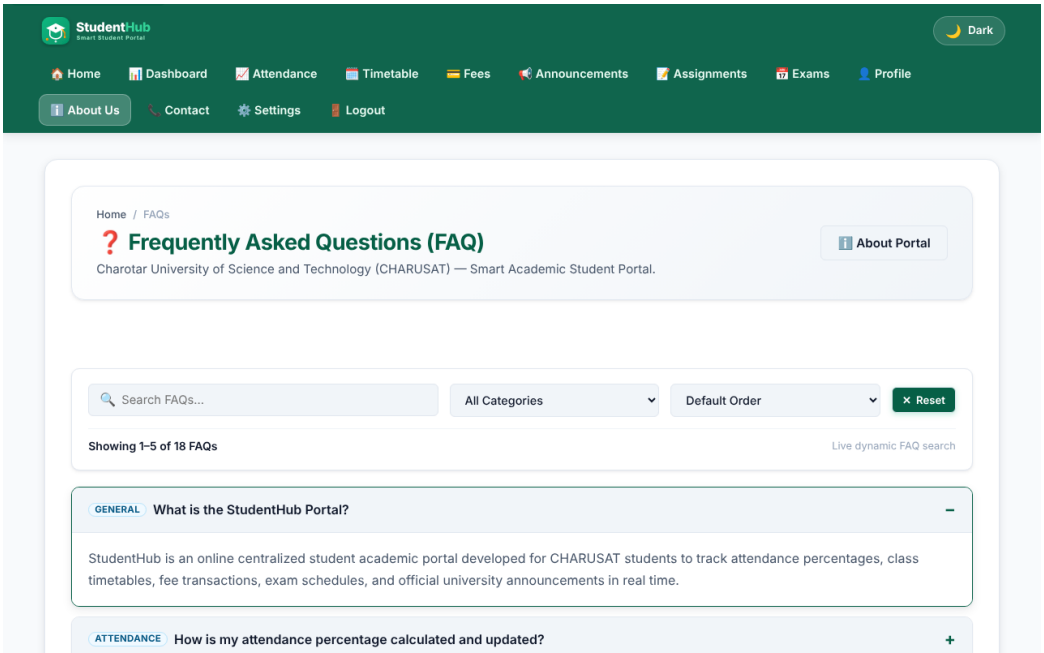
SOURCE CODE :

https://github.com/Aarav280907/WEB/tree/f711ece9a5c0a3d9163db60f7684acf5837073a8/Practical(1-4)(Studenthub)/javascript

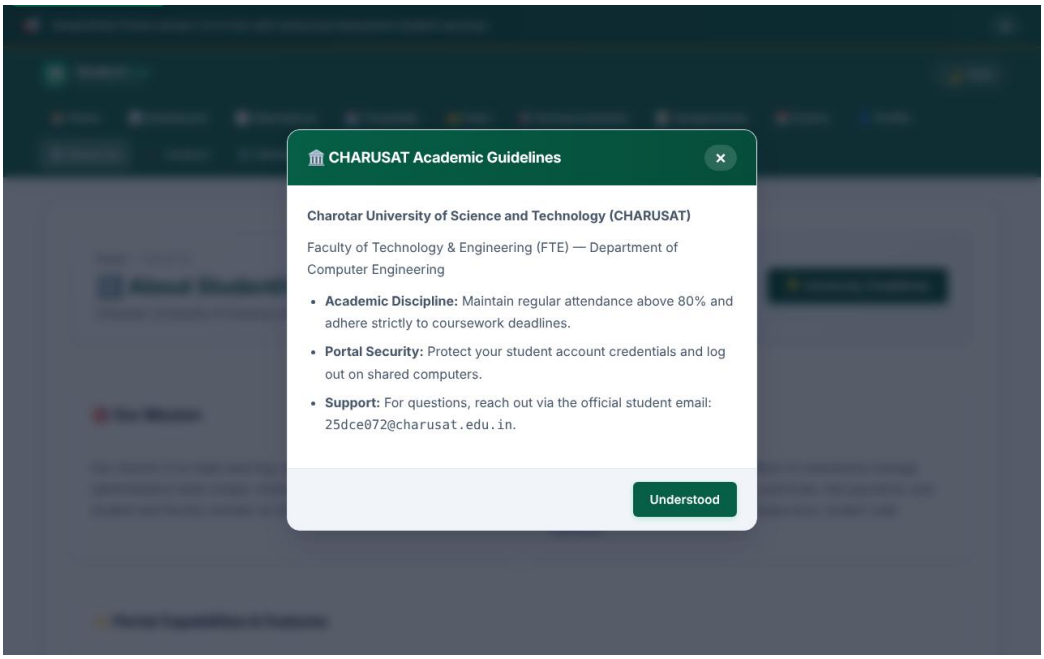
PREVIEW:



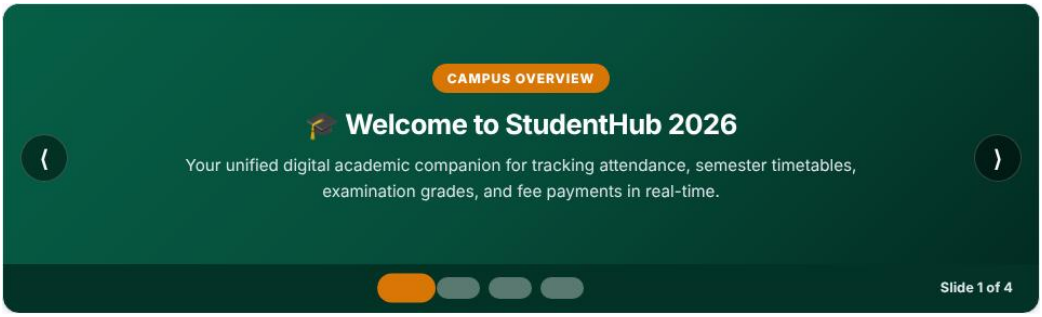
Collapsible FAQ - all answers collapsed



Collapsible FAQ - one answer expanded (aria-expanded = true)



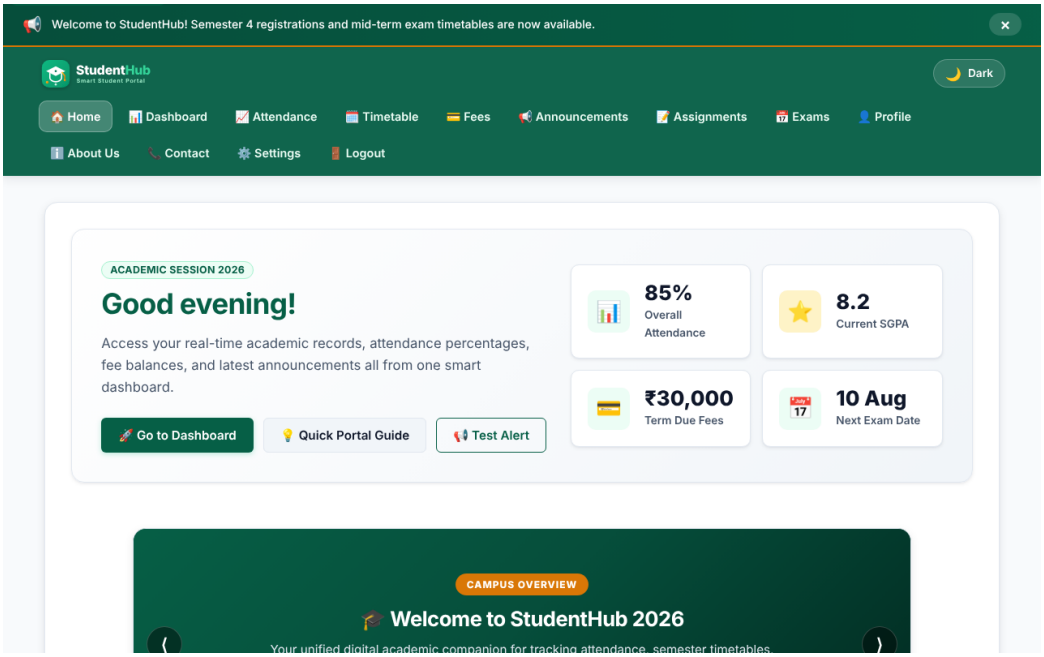
Modal popup opened with the backdrop; closes with the button, backdrop click or Escape



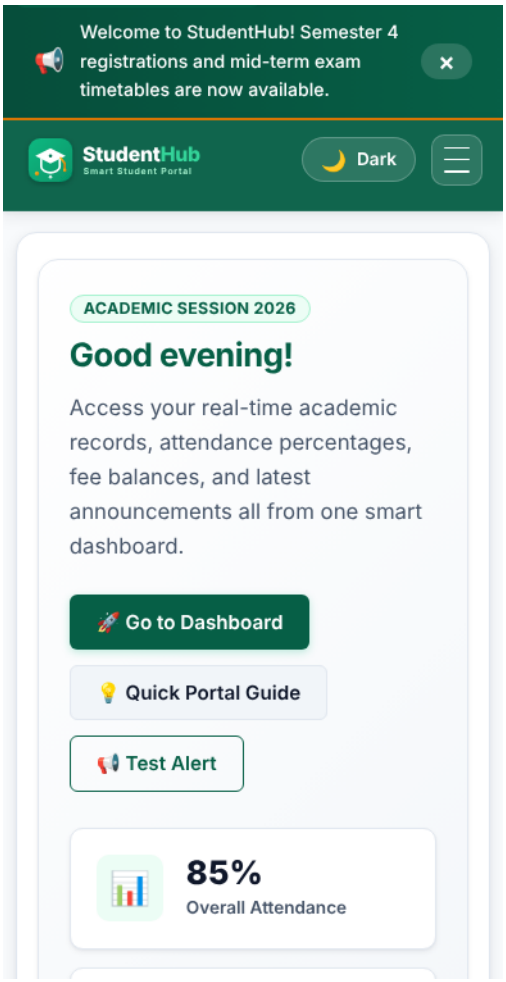
Image/content slider - slide 1 with previous/next buttons and dots



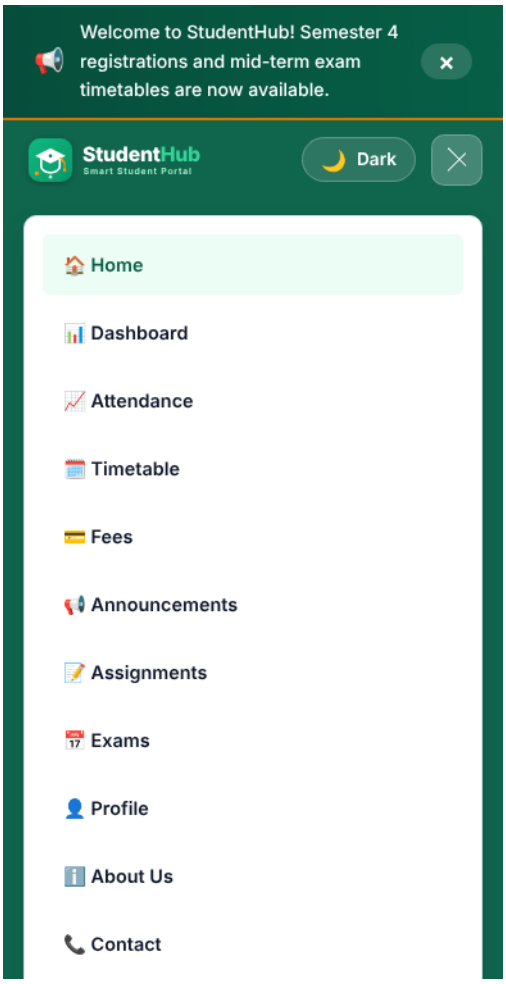
Image/content slider - slide 2 after pressing Next



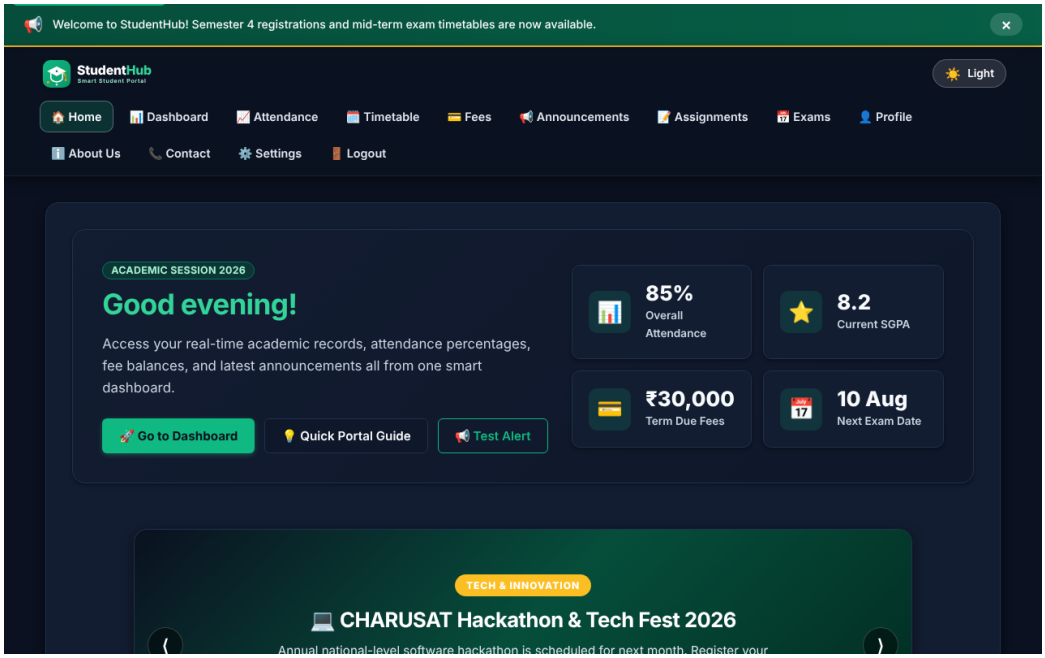
Notification banner at the top of the page with a close button



Mobile view (390 px) - navigation collapsed behind the hamburger button



Hamburger menu opened (aria-expanded = true)



Dark theme applied with the theme switcher (choice saved in localStorage)

KEY QUESTIONS :

1. How are DOM elements selected and modified?

DOM elements are selected using JavaScript methods such as:

- document.getElementById() — selects an element by its ID.
- document.querySelector() — selects the first matching element.
- document.querySelectorAll() — selects multiple matching elements.

For example, in the project:

const hamburgerBtn = document.getElementById('hamburger-btn');

const navElement = document.querySelector('header nav');

The selected elements are modified using:

- classList.add()
- classList.remove()
- classList.toggle()
- setAttribute()
- textContent

For example, when the hamburger menu is clicked:

const isOpen = navElement.classList.toggle('nav-active');

hamburgerBtn.setAttribute('aria-expanded', isOpen ? 'true' : 'false');

The project also dynamically changes the greeting using textContent and changes the theme by adding/removing the dark-theme class.

Answer: DOM elements are selected using getElementById(), querySelector(), and querySelectorAll(), and modified using classList, setAttribute(), and textContent.

2. Are event listeners attached correctly?

Yes. Event listeners are attached using addEventListener() for different interactive elements.

Examples from the project include:

- click → hamburger menu, theme button, FAQ, modal, notification
- keydown → closing the modal with the Escape key
- change → theme selection in settings
- resize → handling the responsive navigation
- DOMContentLoaded → initializing the JavaScript functionality after the page loads

For example:

```
hamburgerBtn.addEventListener('click', function () {
    // toggle navigation
});
```

The FAQ also uses:

```
button.addEventListener('click', function () {
    // open/close FAQ
});
```

The modal uses a keyboard event:

```
document.addEventListener('keydown', function (event) {
    if (event.key === 'Escape') {
        // close modal
    }
});
```

Answer: Yes, event listeners are correctly attached to the required interactive elements using addEventListener(), allowing the UI components to respond to user actions.

3. Is localStorage used for remembering theme choice?

Yes. Your project uses localStorage specifically for remembering the user's light/dark theme choice.

The selected theme is stored using:

```
localStorage.setItem(THEME_STORAGE_KEY, newTheme);
```

When the page loads, the saved theme is retrieved:

```
const savedTheme = localStorage.getItem(THEME_STORAGE_KEY) || 'light';
```

Then it is applied to the page:

applyTheme(savedTheme);

So, if the user selects Dark Mode, leaves the page, and opens another StudentHub page, the saved theme can be restored.

Answer: Yes. localStorage stores the selected light/dark theme and retrieves it when the page loads, so the user's theme preference is remembered.

4. Does interactivity improve usability without breaking accessibility?

Yes. The project uses JavaScript interactivity while also including several accessibility features.

Examples:

- The hamburger menu updates aria-expanded:

hamburgerBtn.setAttribute('aria-expanded', isOpen ? 'true' : 'false');

- The theme button has an accessible label:

aria-label="Toggle dark/light theme"

- The navigation has:

<nav aria-label="Primary Navigation">

- Images have meaningful alt text, such as:

alt="StudentHub Logo"

- The FAQ uses aria-expanded and updates it when the FAQ opens/closes.
- The modal uses role="dialog", aria-modal, and aria-hidden.
- The modal can also be closed using the Escape key.
- The modal moves focus to an interactive element when opened.
- Form fields use associated <label> elements.

Therefore, the JavaScript features improve usability while the ARIA attributes, keyboard interaction, labels, and focus handling help maintain accessibility.

Answer: Yes. Interactivity improves usability through menus, FAQ, modals, notifications, theme switching, etc., while accessibility is maintained using ARIA attributes, labels, keyboard support, focus management, and meaningful alt text.

PRACTICAL-5

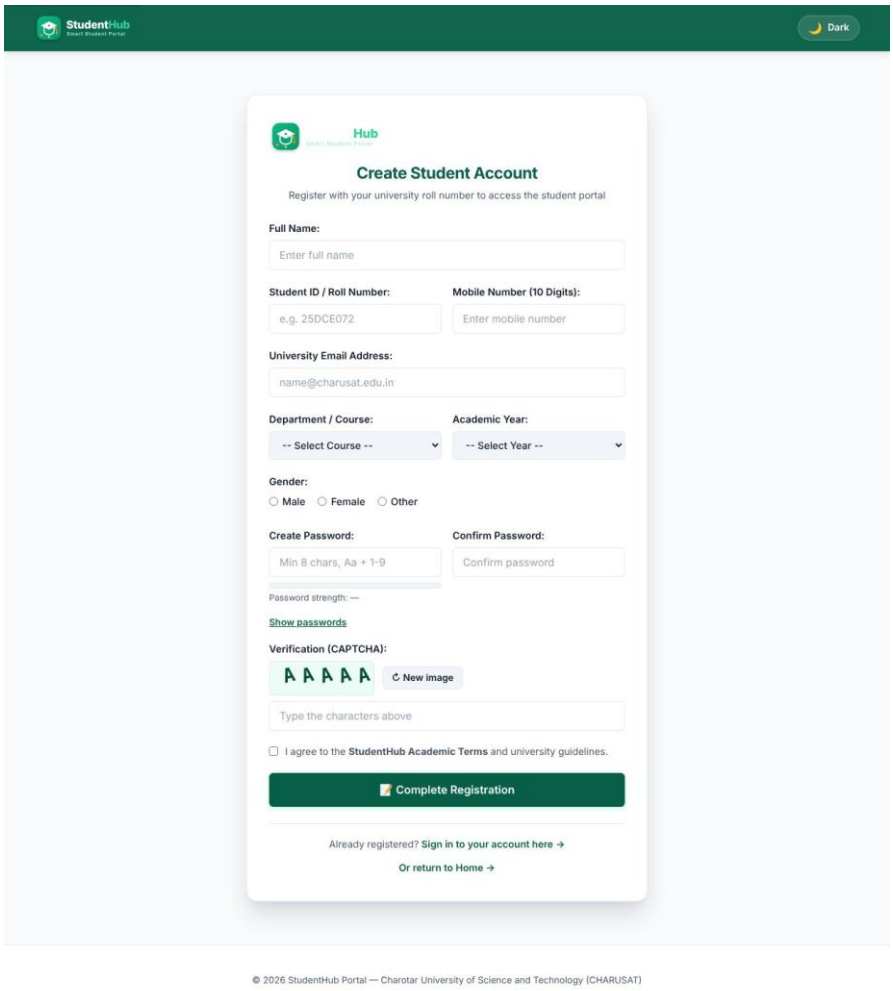
AIM: Build a registration form with HTML5 input types and attributes, add JavaScript validation (regex, inline error messages, password strength meter) and verify accessibility.

SOURCE CODE :

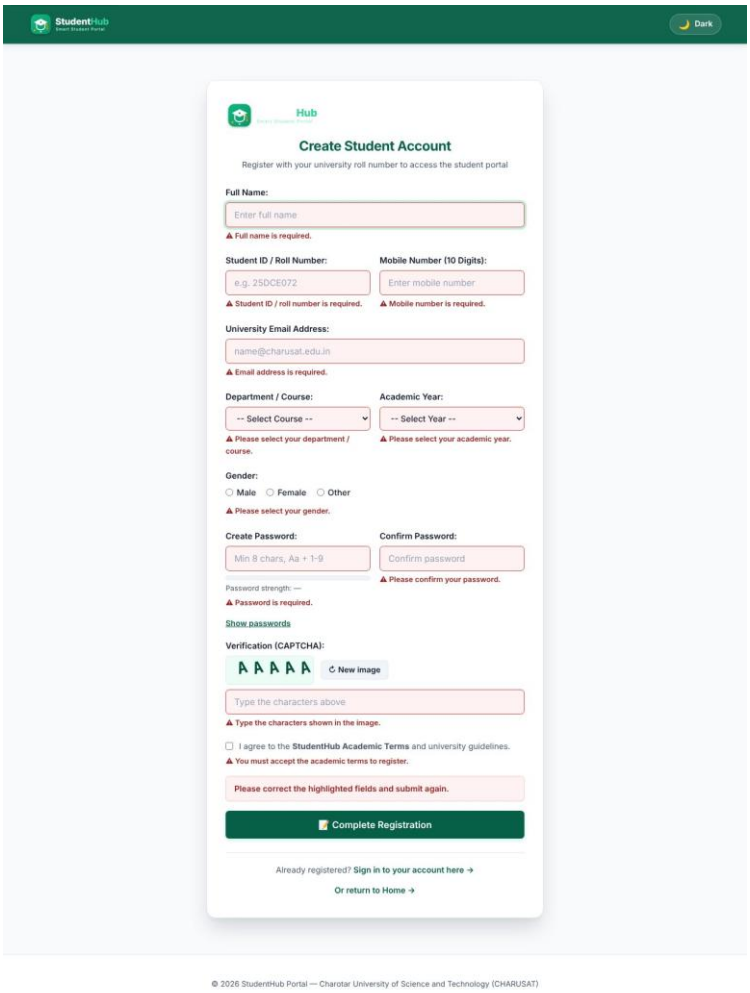
https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/javascript

GitHub repository: https://github.com/Aarav280907/WEB

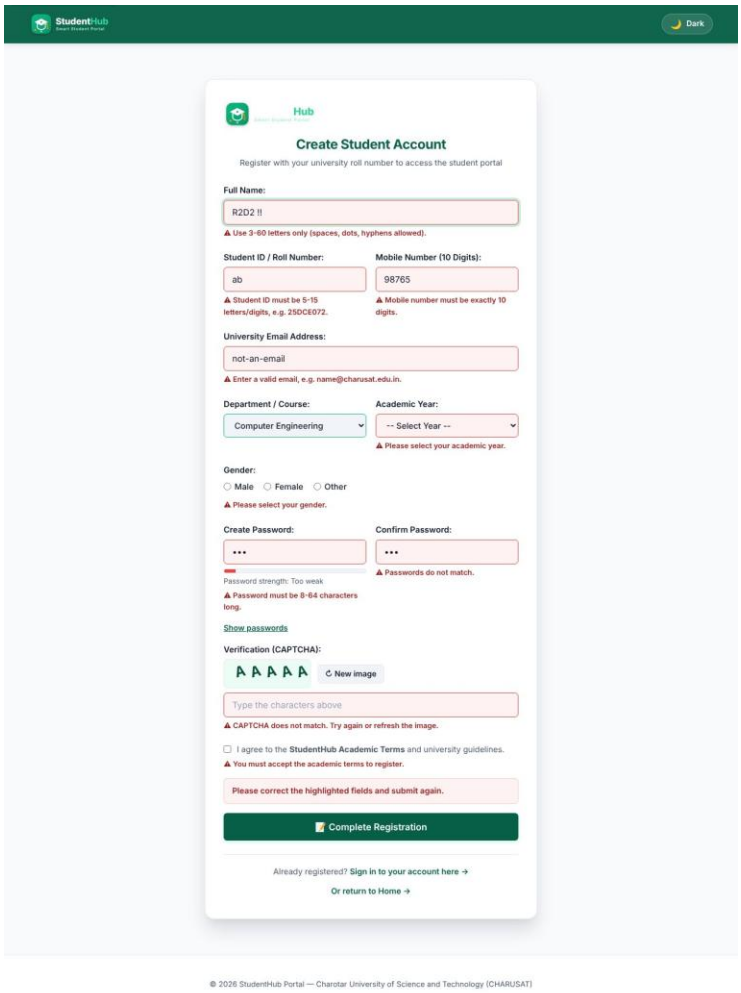
PREVIEW:



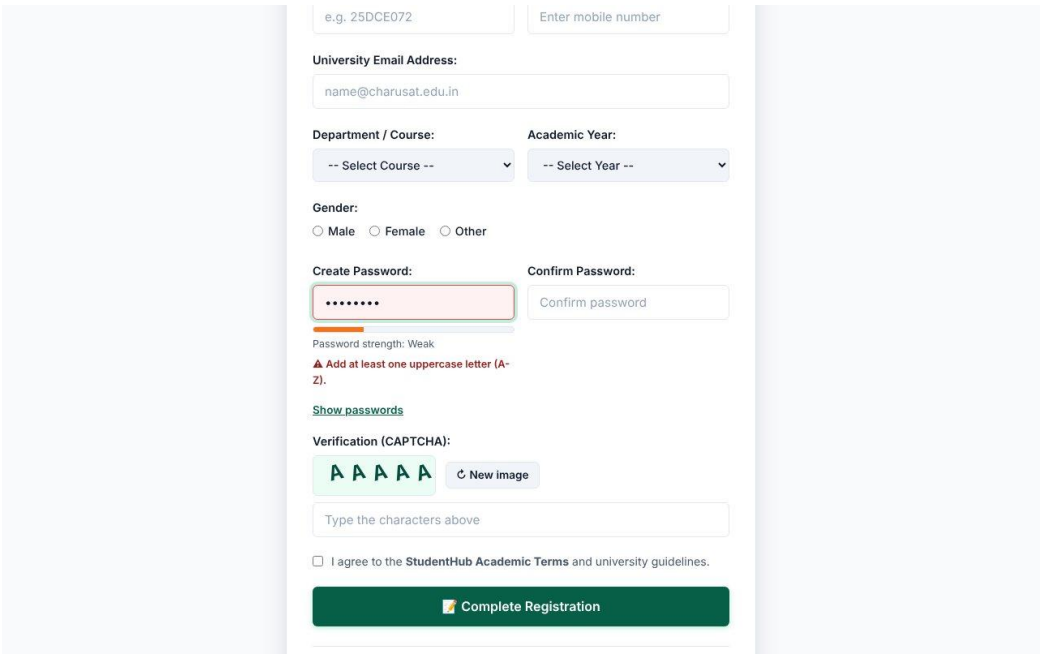
Registration form with HTML5 input types, labels and the Canvas CAPTCHA



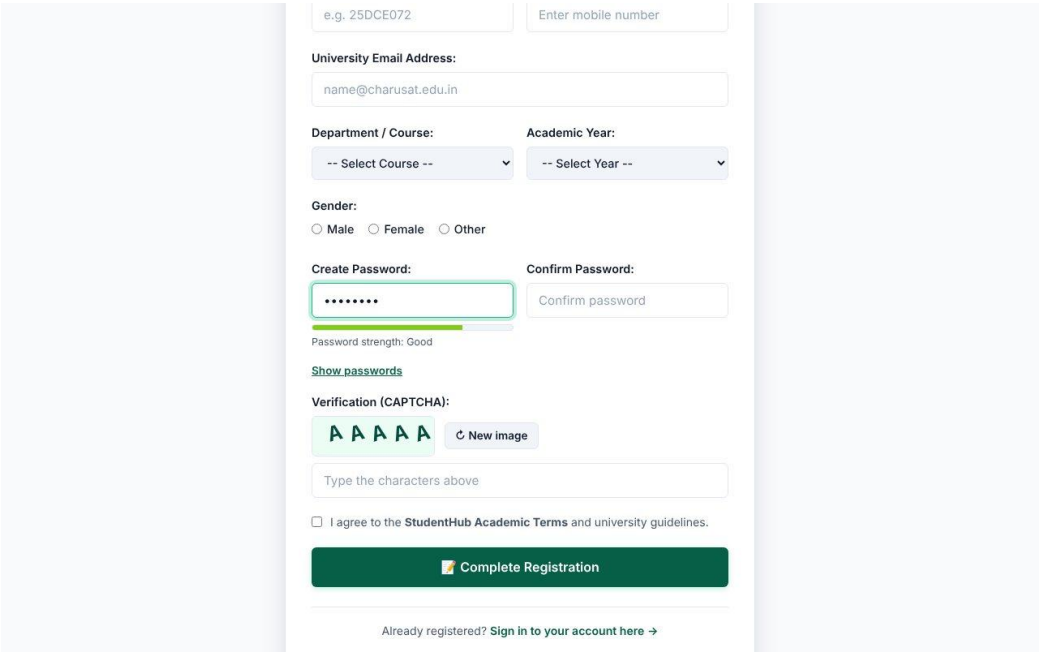
Submitting an empty form: an inline error appears next to every required field



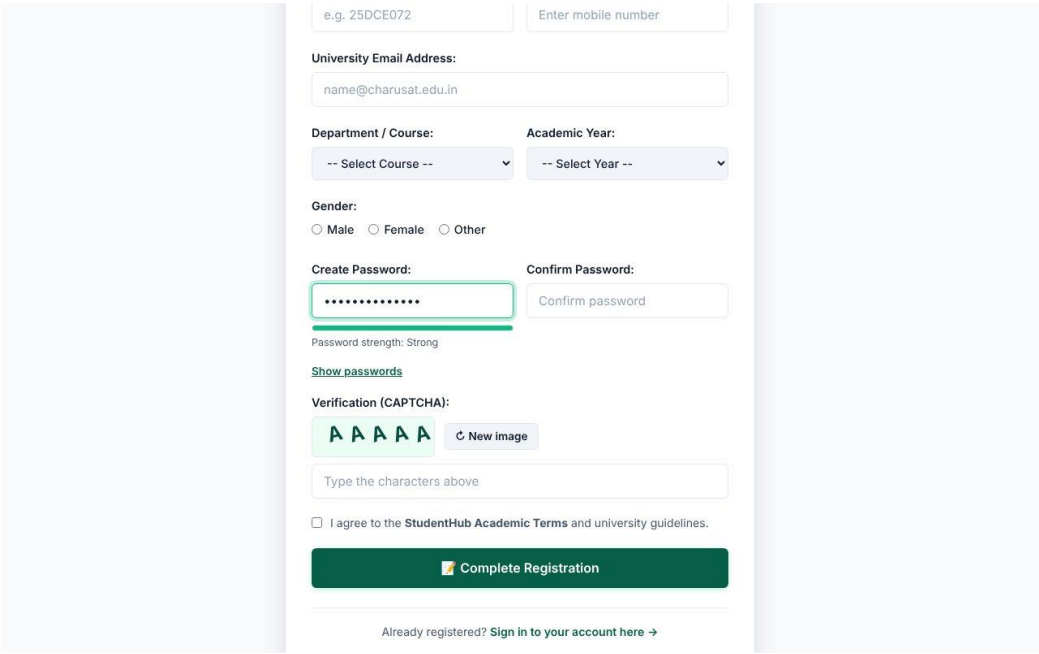
Invalid input: name with digits/symbols, short roll number, 5-digit mobile, bad e-mail, weak and mismatching passwords, wrong CAPTCHA



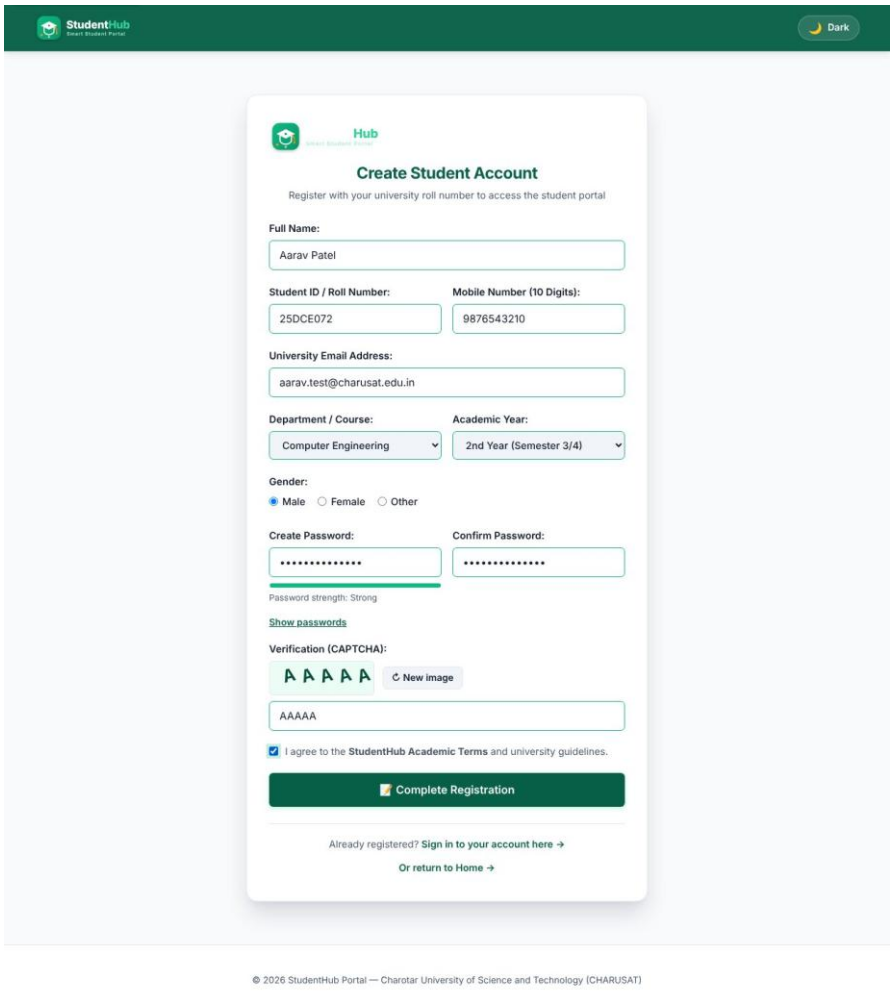
Password strength meter - "Too weak"



Password strength meter - medium



Password strength meter - strong



All fields valid: no error messages are shown

TEST CASES :

#	Input / action	Expected result	Result
1	Submit the empty form	Required-field error under every field; summary "Please correct the highlighted fields"; focus moves to the first invalid field	Pass
2	Full name = "R2D2 !!"	Error: letters, spaces, dots, hyphens only (3-60)	Pass
3	Student ID = "ab"	Error: must be 5-15 letters/digits	Pass
4	Mobile = "98765" (and typing letters)	Letters are filtered out while typing; 5 digits give "exactly 10 digits" error	Pass
5	E-mail = "not-an-email"	Error: enter a valid e-mail	Pass
6	Password = "abc"	Error: 8-64 characters; strength meter "Too weak"	Pass
7	Password = "abcdefgh" / "Abcdefg1" / "Strong@Pass123"	Meter moves weak -> medium -> strong; the first two still fail the upper/lower/digit rule where applicable	Pass
8	Confirm password differs	Error: Passwords do not match	Pass
9	Wrong CAPTCHA text	Error: CAPTCHA does not match; a new image is generated	Pass
10	Terms box not ticked	Error: you must accept the terms	Pass
11	All valid values	No errors; the form is submitted to the PHP processor (Practical 7)	Pass

KEY QUESTIONS :

1. Are correct input types and attributes used?

Yes. The form uses type="email", type="tel" (with inputmode="numeric" and maxlength="10"), type="password", select lists, radio buttons and a checkbox. Attributes such as required, minlength, maxlength, autocomplete and aria-describedby are set, so mobile keyboards and password managers behave correctly.

```
<input type="tel" id="mobile" name="mobile" maxlength="10" inputmode="numeric"
autocomplete="tel" required aria-describedby="mobile-error">
<span class="field-error" id="mobile-error" role="alert"></span>
```

The browser's own bubbles are switched off with novalidate so that all messages look the same and come from our JavaScript; the server repeats every check (Practical 7 and 9), so JavaScript is only a convenience, never the only protection.

2. Are errors displayed near the fields?

Yes. Each field has its own <span id="<name>-error" role="alert"> directly below it. The validator writes the message there, adds the class has-error to the field (red border) and sets aria-invalid="true". When the field becomes valid the message is cleared and is-valid is added. Validation also runs in real time on input, change and blur (intermediate extension), so the user sees the problem as soon as they leave the field.

```
const PATTERNS = {
  fullname:  /^[A-Za-z][A-Za-z .'-]{2,59}$/,
  student_id: /^[A-Za-z0-9]{5,15}$/,
  mobile:    /^[0-9]{10}$/,
  email:     /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/
};
```

3. Is password strength checked?

Yes. The password must be 8-64 characters and contain an upper-case letter, a lower-case letter and a digit. In addition scorePassword() gives 0-4 points (length >= 8, length >= 12, mixed case, digit, special character) and the coloured bar under the field shows "Too weak / Weak / Good / Strong" while typing.

```
function scorePassword(value) {
  let score = 0;
  if (value.length >= 8) score++;
  if (value.length >= 12) score++;
```

```
  if (r.upper.test(value) && r.lower.test(value)) score++;
  if (r.digit.test(value)) score++;
  if (r.special.test(value)) score++;
  return Math.min(score, 4);
}
```

4. Is the form keyboard and screen-reader accessible?

- Every input has a visible <label for="..."></label>; the radio group is wrapped with role="radiogroup" and aria-labelledby.
- Error spans use role="alert", so a screen reader announces a message when it appears; invalid fields get aria-invalid="true".
- The whole form can be completed with Tab / Shift+Tab / Space / Enter; the first invalid field receives focus after a failed submit; a visible focus outline is kept.
- The CAPTCHA canvas has role="img" and an aria-label, and a "New image" button; "Show passwords" is a real <button>.
- Colour is never the only signal: errors also have an icon and text.

5. Advanced extension - custom CAPTCHA on Canvas

A 5-character code (no confusing characters such as 0/O or 1/I) is drawn on a <canvas> with random rotation, noise lines and dots. The typed text is compared with the code in JavaScript. It is a demonstration only: a client-side CAPTCHA cannot stop bots, which is stated in the code comments; real protection is the server-side CSRF token and validation.

PRACTICAL-6

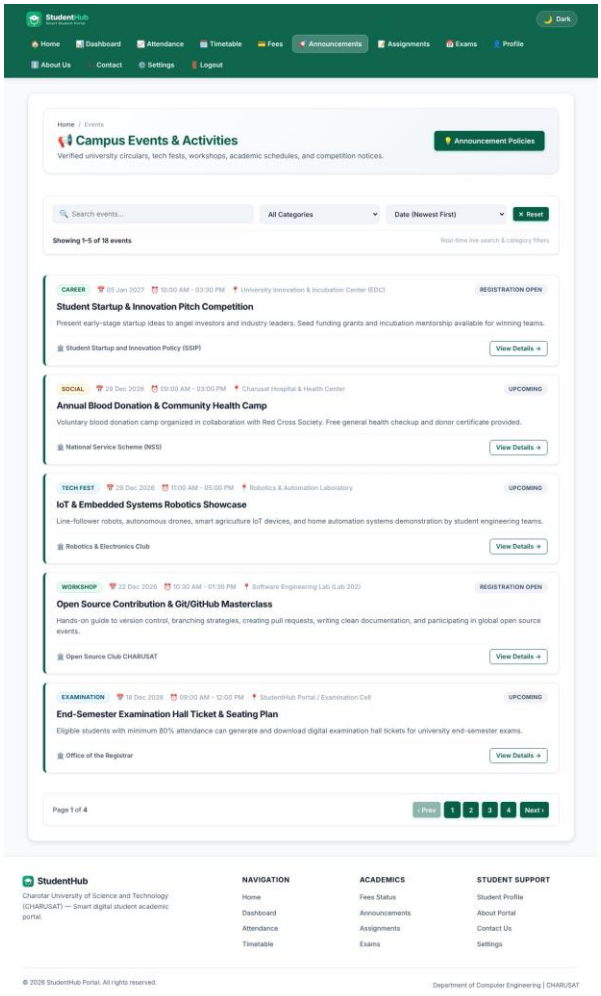
AIM: Load data from JSON files with the Fetch API and render it dynamically with search, filter, sort and pagination using modular JavaScript.

SOURCE CODE :

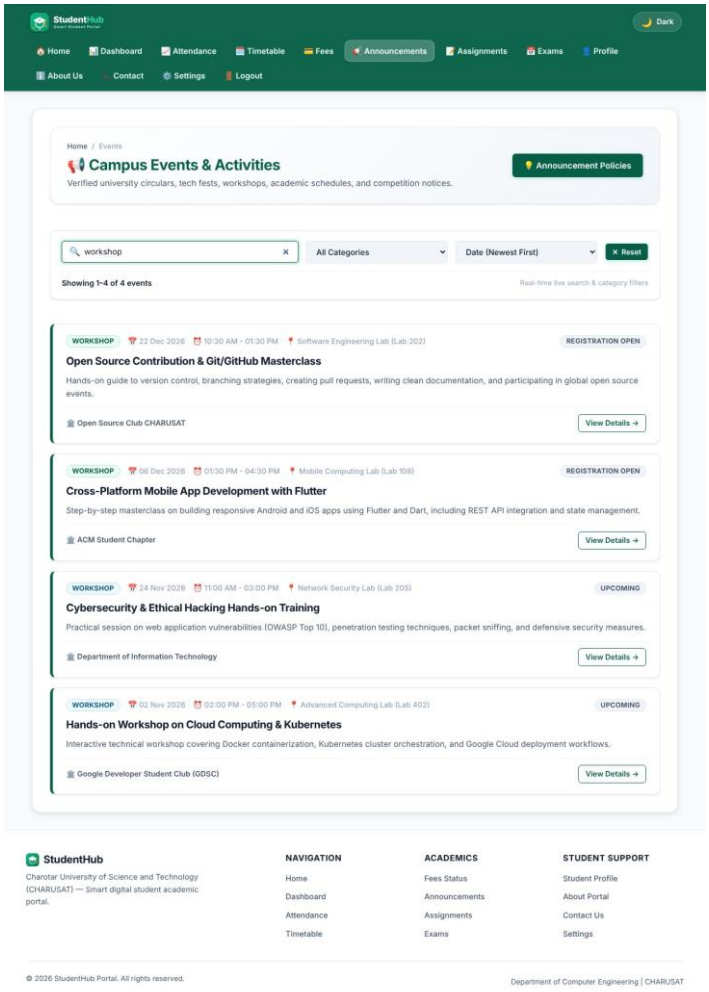
https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-6

GitHub repository: https://github.com/Aarav280907/WEB

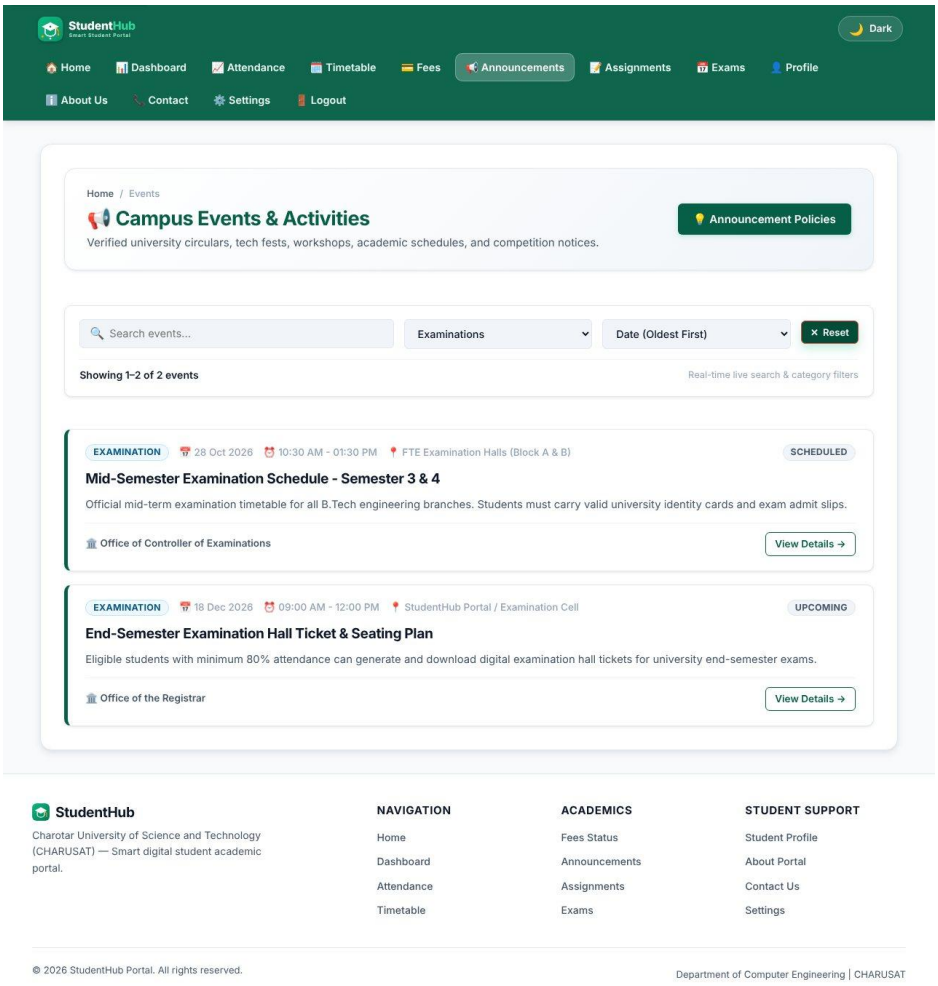
PREVIEW:



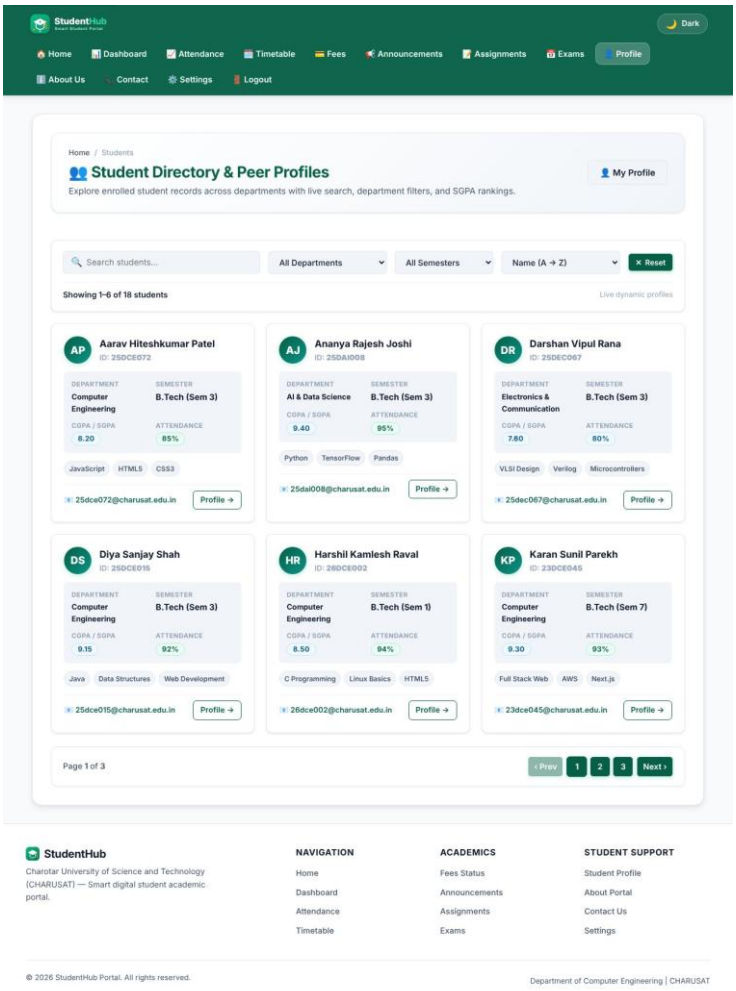
Events page - cards rendered from events.json



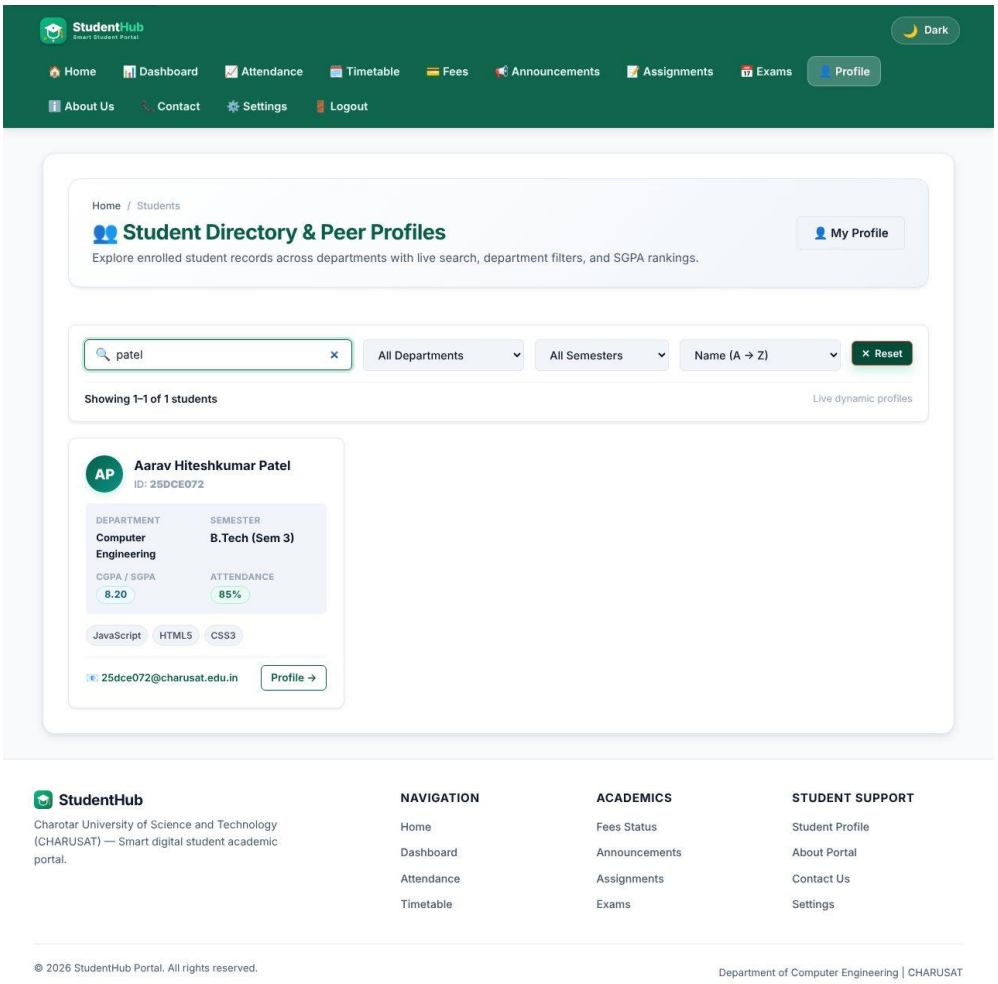
Search "workshop" - results filtered as the user types



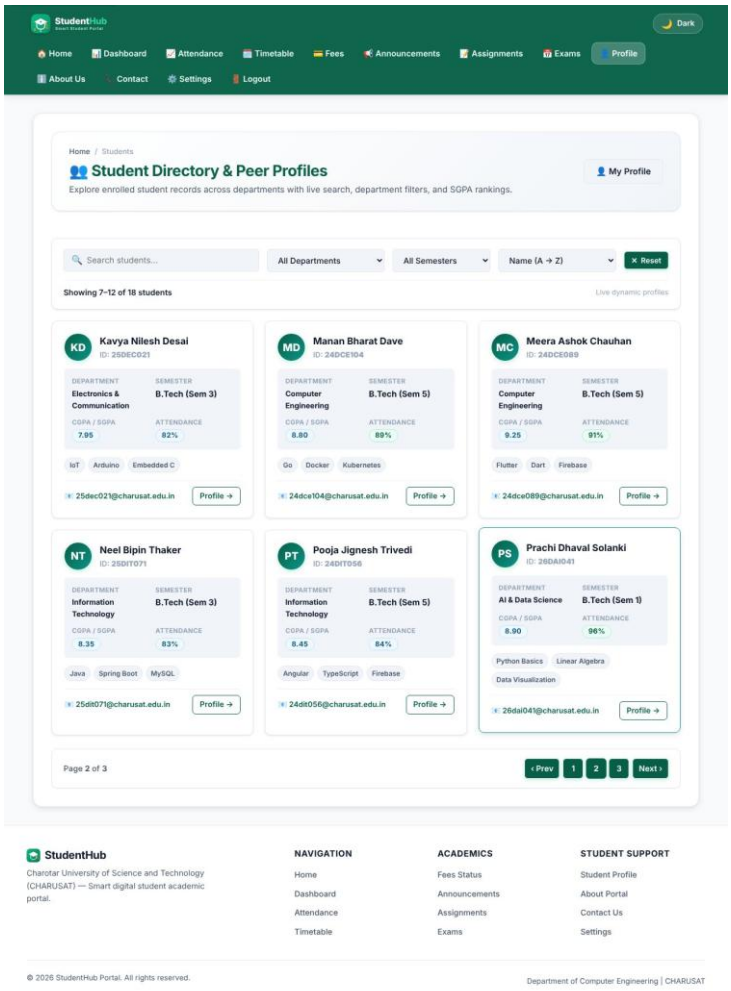
Category filter combined with a sort order



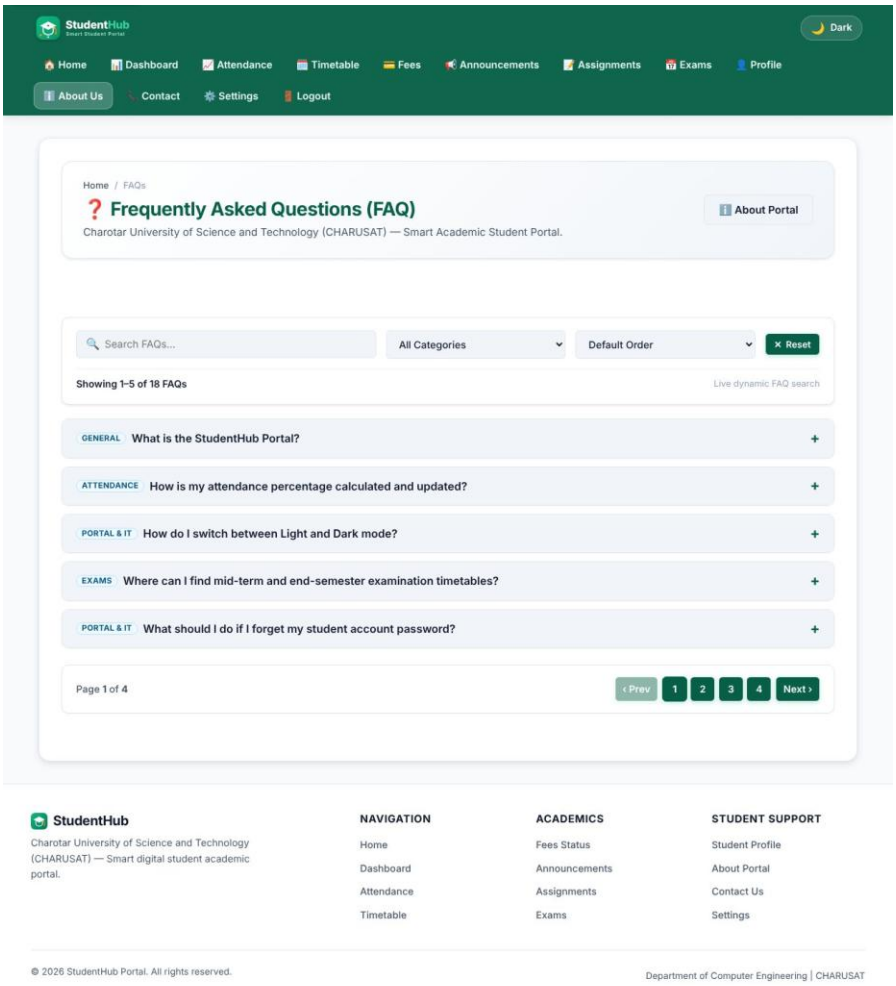
Student directory rendered from students.json



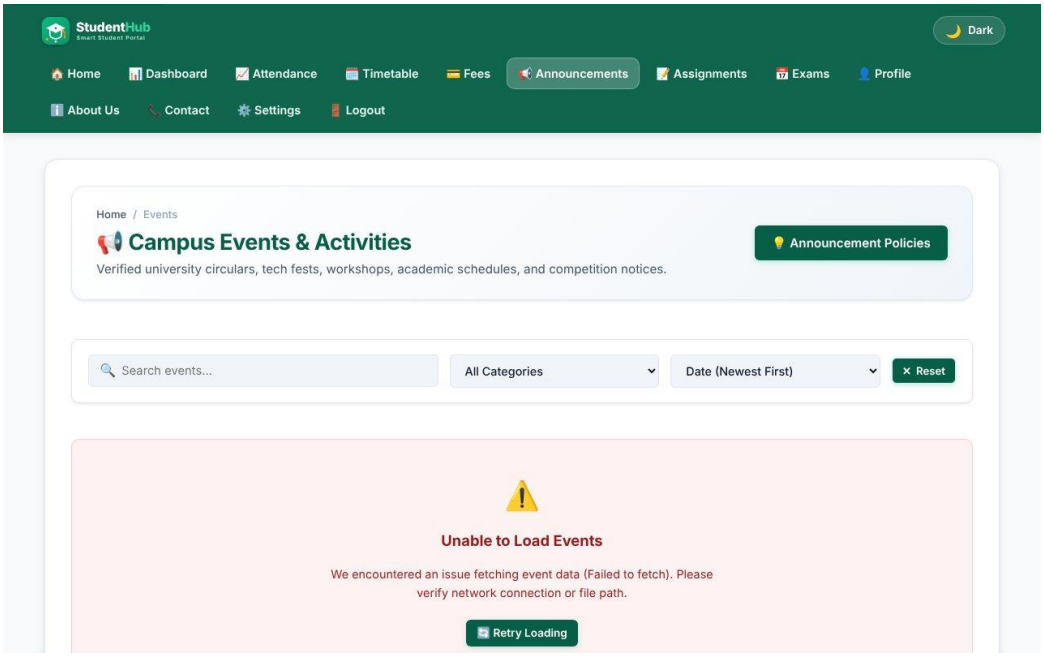
Student search "patel"



Pagination - second page of students



FAQ page rendered from faqs.json



Error state when the JSON request fails (network blocked) with a Retry button

TEST CASES :

#	Input / action	Expected result	Result
1	Open events page	18 events loaded from events.json, loading text shown first	Pass
2	Search "workshop"	Only matching events; count updates (5 events per page)	Pass
3	Category = Workshop + sort	Filter and sort work together	Pass
4	Search text with no match	Empty-state message with a "Clear all filters" button	Pass
5	Students: next page	Next 6 records shown, pagination buttons update	Pass
6	Block the .json request	Error card with a Retry button, no JavaScript error	Pass

KEY QUESTIONS :

1. How is JSON fetched, parsed and rendered?

fetch() is called inside an async function. The response is checked with response.ok, then response.json() parses the body to an array of objects. The array is stored once (allEvents) and every render is built from it with template strings and written with innerHTML after escaping the text.

```
const response = await fetch(url);
if (response.ok) {
  fetchedData = await response.json();
}
...
container.innerHTML = pageItems.map(renderCard).join("");
```

2. Which array methods are used for search, filter, sort and pagination?

- filter() - keeps the items that match the search text and the selected category;
- sort() - orders by date (newest/oldest) or title with localeCompare();
- slice(start, start + perPage) - cuts the current page;
- map() / join() - turns the page items into HTML; forEach() - attaches button listeners.

```
let filtered = allEvents.filter(item => matchesText(item) && matchesCategory(item));
filtered.sort((a, b) => new Date(b.date) - new Date(a.date));
const paginatedItems = filtered.slice(startIndex, startIndex + itemsPerPage);
```

3. How are loading and error states handled?

Before the request a "Fetching ..." block with role="status" and aria-live="polite" is shown. If the request fails or the status is not OK, renderErrorState() shows a message with role="alert" and a Retry button that runs the fetch again. Several relative paths for the JSON file are tried so the page works from different folders. An empty result shows a separate empty-state with a "Clear all filters" button.

4. How is modularity maintained?

Each page has its own file (events.js, students.js, faq.js) wrapped in an IIFE so no variables leak to the global scope; only one object (window.StudentHubEvents = { init }) is exported. Data (JSON), logic (JS), structure (HTML) and style (CSS) are separate files, and the three modules follow the same structure (fetch, process, render, init).

PRACTICAL-7

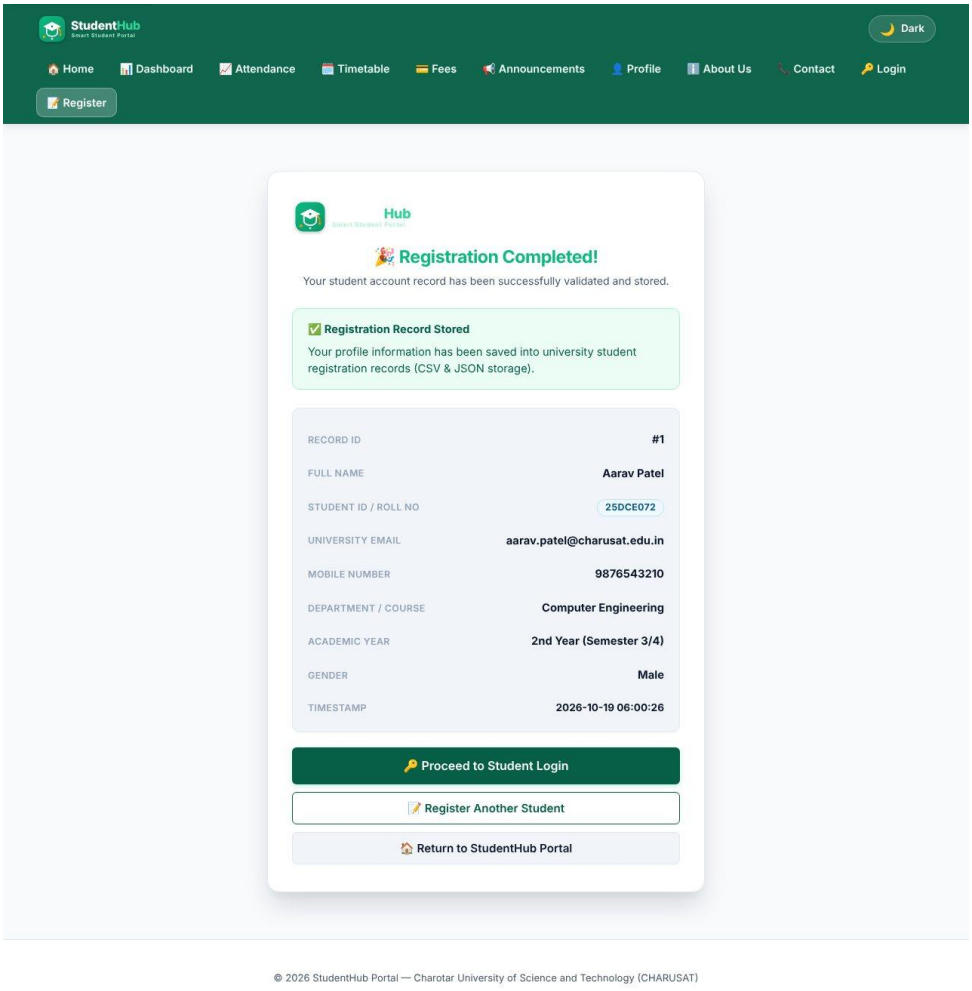
AIM: Process an HTML form with PHP using the POST method, validate and sanitize the input on the server and store the records in CSV and JSON files.

SOURCE CODE :

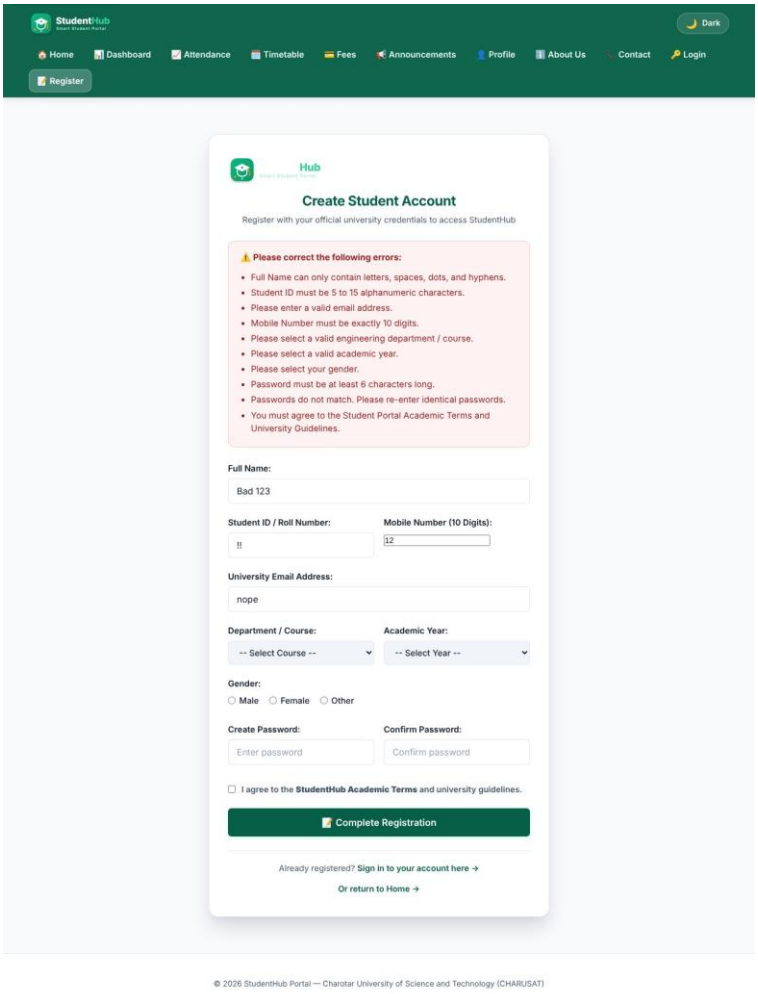
https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-7

GitHub repository: https://github.com/Aarav280907/WEB

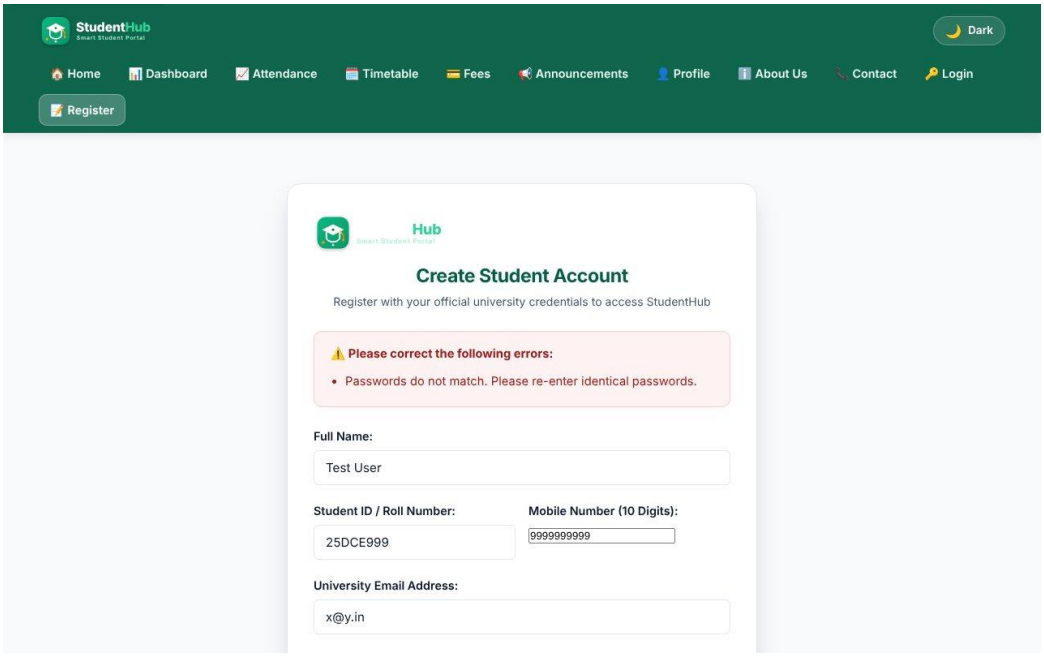
PREVIEW:



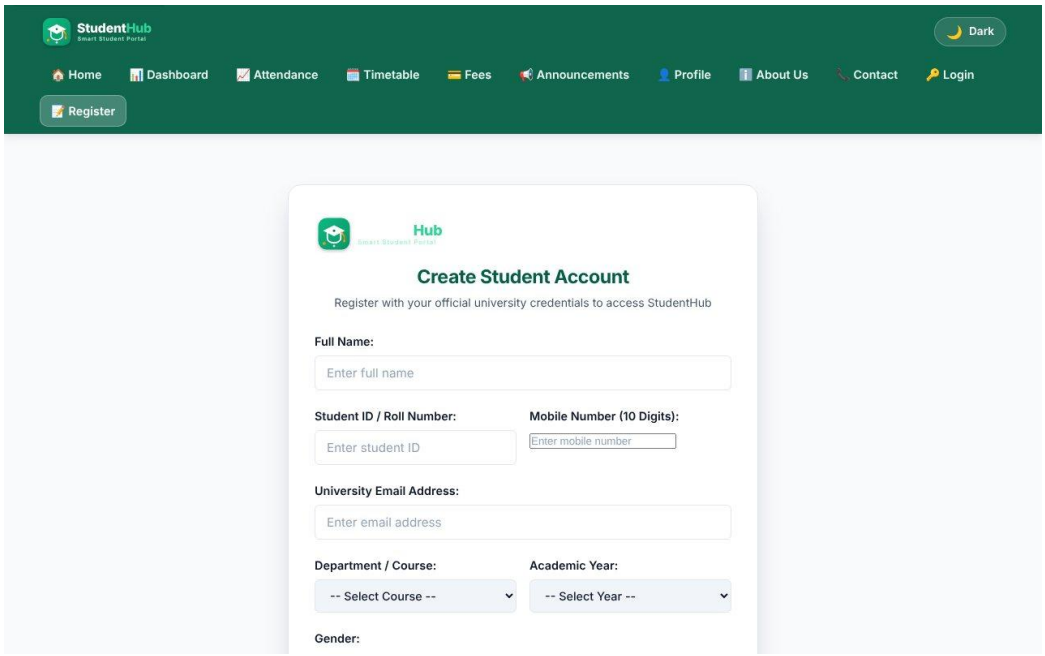
Valid registration - success page with the saved record



Server-side validation errors for invalid input (client JavaScript disabled)



Server-side check: passwords do not match



Opening the processor with GET shows the form again - nothing is written

Practical-7/registrations.csv (generated file)

```
id,fullname,student_id,email,mobile,course,year,gender,registration_date
1,"Aarav Patel",25DCE072,aarav.patel@charusat.edu.in,9876543210,"Computer Engineering","2nd Year (Semester 3/4)",Male,"2026-10-19 06:00:
26"
2,"Riya Shah",25DIT034,riya.shah@charusat.edu.in,9876543210,"Computer Engineering","2nd Year (Semester 3/4)",Male,"2026-10-19 06:00:28"
3,"Karan Mehta",25DAI011,karan.mehta@charusat.edu.in,9876543210,"Computer Engineering","2nd Year (Semester 3/4)",Male,"2026-10-19 06:00:
29"
```

Generated file registrations.csv

Practical-7/registrations.json (generated file)


```
[
    {
        "id": 1,
        "fullname": "Aarav Patel",
        "student_id": "25DCE072",
        "email": "aarav.patel@charusat.edu.in",
        "mobile": "9876543210",
        "course": "Computer Engineering",
        "year": "2nd Year (Semester 3/4)",
        "gender": "Male",
        "registration_date": "2026-10-19 06:00:26"
    },
    {
        "id": 2,
        "fullname": "Riya Shah",
        "student_id": "25DIT034",
        "email": "riya.shah@charusat.edu.in",
        "mobile": "9876543210",
        "course": "Computer Engineering",
        "year": "2nd Year (Semester 3/4)",
        "gender": "Male",
        "registration_date": "2026-10-19 06:00:28"
    },
    {
        "id": 3,
        "fullname": "Karan Mehta",
        "student_id": "25DAI011",
        "email": "karan.mehta@charusat.edu.in",
        "mobile": "9876543210",
        "course": "Computer Engineering",
        "year": "2nd Year (Semester 3/4)",
        "gender": "Male",
        "registration_date": "2026-10-19 06:00:29"
    }
]
```


Generated file registrations.json

TEST CASES :

#	Input / action	Expected result	Result
1	POST valid data (3 different students)	Success page; one row added to CSV and one object to JSON	Pass
2	Name = "Bad 123", ID = "!!", mobile = "12", e-mail = "nope", short password	List of error messages, nothing stored	Pass
3	Password and confirm password different	Error "passwords do not match"	Pass
4	GET request to process_signup.php	No record is created; empty form shown	Pass
5	HTML tags in name	Tags stripped by strip_tags and rejected by the name pattern	Pass
6	Passwords in generated files	Not written to CSV/JSON (only profile fields)	Pass

Note on production use:

Practical 7 stores data in files only to learn form processing. Passwords are deliberately not stored here; real registration with password_hash() and a database is done in Practical 9.

KEY QUESTIONS :

1. Is POST used?

Yes. The form has method="post" and the script starts with if ($_SERVER['REQUEST_METHOD'] === 'POST'). Data never appears in the URL or the browser history, and opening the page with GET only shows the form.

2. Are inputs validated and sanitized on the server side?

Every field is trimmed and sanitized (strip_tags, FILTER_SANITIZE_EMAIL, preg_replace to keep digits only), then validated with the same rules as the JavaScript: length and character patterns, allowed values for course / year / gender (white-lists), e-mail with filter_var, password rules, terms accepted. All error messages are collected in an array and shown together; output is escaped with htmlspecialchars().

```
$fullname = trim(strip_tags($_POST['fullname'] ?? ''));
$mobile   = preg_replace('/[^0-9]/', '', $_POST['mobile'] ?? '');
if (!in_array($course, $allowedCourses, true)) { $errors[] = 'Select a valid course.'; }
```

3. Is file writing safe?

Files are opened with a fixed path (never from user input), JSON is rewritten with json_encode after reading the old content, and the CSV is appended under flock(LOCK_EX) so two simultaneous users cannot corrupt the file. The CSV writer uses fputcsv, which quotes commas and quotes. The return values of file_put_contents / fopen are checked and a friendly error is shown if writing fails.

```
$fp = @fopen($csvFile, 'a');
if ($fp && flock($fp, LOCK_EX)) {
  fputcsv($fp, [$id, $fullname, $student_id, $email, $mobile, $course, $year, $gender,
$registration_date]);
  flock($fp, LOCK_UN);
}
```

4. Are success and error responses clear?

Yes. A green success card shows the stored data; a red box lists every problem with a bullet; the form keeps the values the user typed (except passwords). A storage failure gives a separate message so the user knows it is not their mistake.

PRACTICAL-8

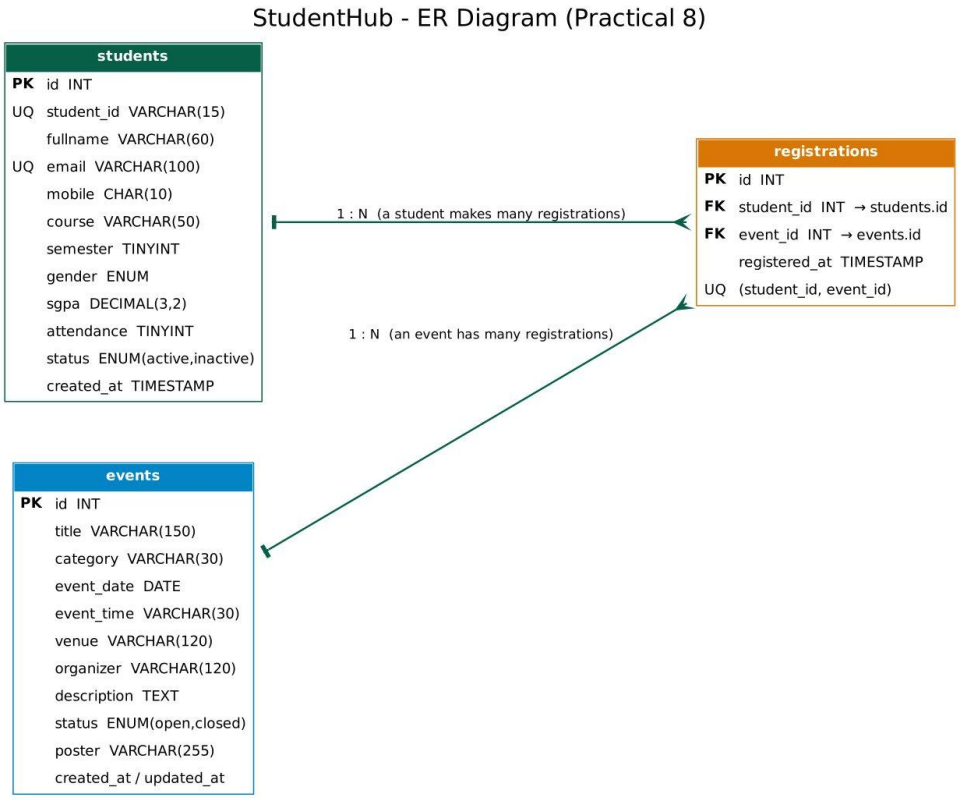
AIM: Create the MySQL schema and ER model for StudentHub, connect with PDO / MySQLi using prepared statements and test the connection.

SOURCE CODE :

https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-8

GitHub repository: https://github.com/Aarav280907/WEB

PREVIEW:



ER diagram: students 1-N registrations N-1 events

```
mysql> SHOW TABLES; DESCRIBE students; DESCRIBE events; DESCRIBE registrations;

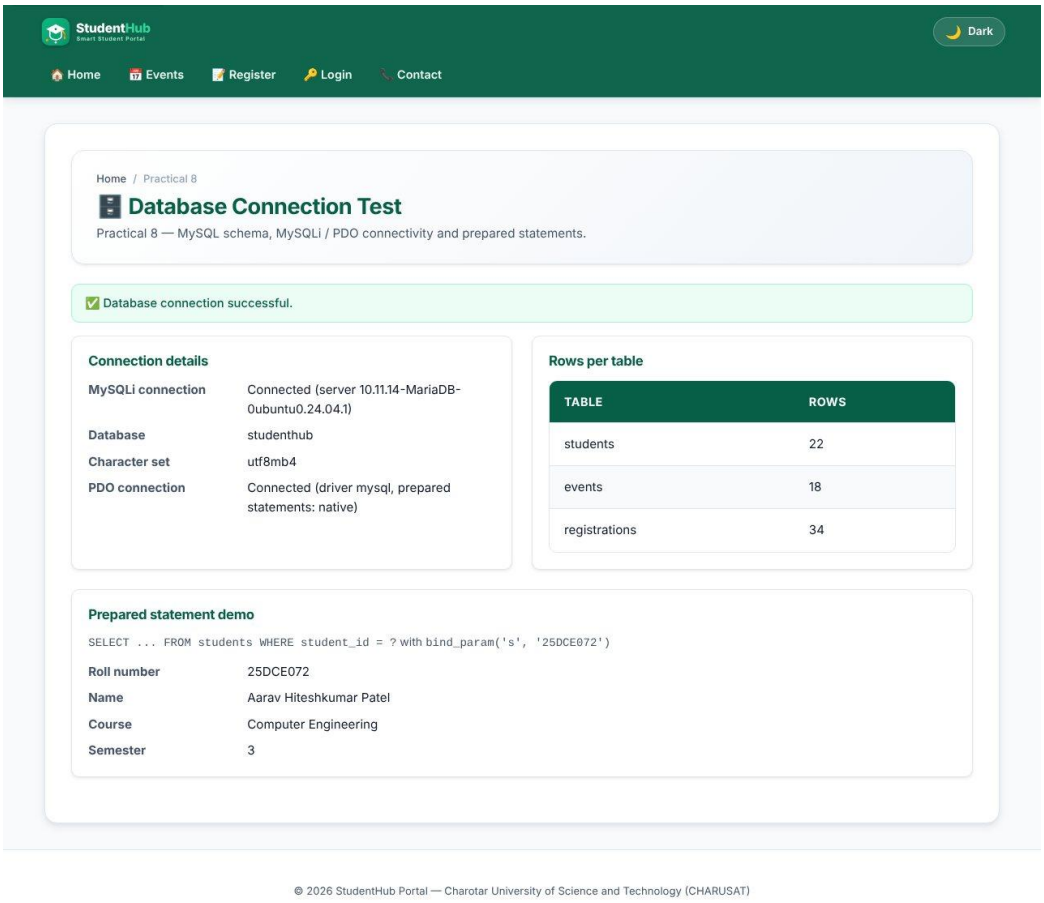
+---------------------+
| Tables_in_studenthub |
+---------------------+
| audit_logs          |
| events              |
| registrations       |
| remember_tokens     |
| students            |
| users               |
+---------------------+
+------------+-----------------------------+------+-----+--------------------+----------------+
| Field      | Type                        | Null | Key | Default            | Extra          |
+------------+-----------------------------+------+-----+--------------------+----------------+
| id         | int(10) unsigned            | NO   | PRI | NULL               | auto_increment |
| student_id | varchar(15)                 | NO   | UNI | NULL               |                |
| fullname   | varchar(60)                 | NO   |     | NULL               |                |
| email      | varchar(100)                | NO   | UNI | NULL               |                |
| mobile     | char(10)                    | NO   |     | NULL               |                |
| course     | varchar(50)                 | NO   | MUL | NULL               |                |
| semester   | tinyint(3) unsigned         | NO   |     | NULL               |                |
| gender     | enum('Male','Female','Other') | NO   |     | Other              |                |
| sgpa       | decimal(3,2)                | NO   |     | 0.00               |                |
| attendance | tinyint(3) unsigned         | NO   |     | 0                  |                |
| status     | enum('active','inactive')   | NO   |     | active             |                |
| created_at | timestamp                   | NO   |     | current_timestamp() |                |
+------------+-----------------------------+------+-----+--------------------+----------------+
+-------------+----------------------+------+-----+---------------------+-------------------------------+
| Field       | Type                 | Null | Key | Default             | Extra                         |
+-------------+----------------------+------+-----+---------------------+-------------------------------+
| id          | int(10) unsigned     | NO   | PRI | NULL                | auto_increment                |
| title       | varchar(150)         | NO   |     | NULL                |                               |
| category    | varchar(30)          | NO   | MUL | NULL                |                               |
| event_date  | date                 | NO   | MUL | NULL                |                               |
| event_time  | varchar(30)          | NO   |     |                     |                               |
| venue       | varchar(120)         | NO   |     |                     |                               |
| organizer   | varchar(120)         | NO   |     |                     |                               |
| description | text                 | YES  |     | NULL                |                               |
| status      | enum('open','closed') | NO   |     | open                |                               |
| poster      | varchar(255)         | YES  |     | NULL                |                               |
| created_at  | timestamp            | NO   |     | current_timestamp() |                               |
| updated_at  | timestamp            | NO   |     | current_timestamp() | on update current_timestamp() |
+-------------+----------------------+------+-----+---------------------+-------------------------------+
+--------------+-----------------+------+-----+---------------------+----------------+
| Field        | Type            | Null | Key | Default             | Extra          |
+--------------+-----------------+------+-----+---------------------+----------------+
| id           | int(10) unsigned | NO   | PRI | NULL                | auto_increment |
| student_id   | int(10) unsigned | NO   | MUL | NULL                |                |
| event_id     | int(10) unsigned | NO   | MUL | NULL                |                |
| registered_at | timestamp       | NO   |     | current_timestamp() |                |
+--------------+-----------------+------+-----+---------------------+----------------+
```

Tables created in the studenthub database (SHOW TABLES / DESCRIBE)

```
Foreign keys (information_schema)

+-----------------+-------------+-----------------+-----------------------+------------------------+
| TABLE_NAME      | COLUMN_NAME | CONSTRAINT_NAME | REFERENCED_TABLE_NAME | REFERENCED_COLUMN_NAME |
+-----------------+-------------+-----------------+-----------------------+------------------------+
| audit_logs      | user_id     | fk_audit_user   | users                 | id                     |
| registrations   | event_id    | fk_reg_event    | events                | id                     |
| registrations   | student_id  | fk_reg_student  | students              | id                     |
| remember_tokens | user_id     | fk_remember_user | users                | id                     |
+-----------------+-------------+-----------------+-----------------------+------------------------+
```

Foreign keys read from information_schema



test_connection.php - MySQLi and PDO connected, row counts and a prepared-statement demo

TEST CASES :

#	Input / action	Expected result	Result
1	Open test_connection.php with correct settings	Both connections successful; 22 / 18 / 34 rows	Pass
2	Wrong DB password (SH_DB_PASS env)	Friendly "database temporarily unavailable" message (HTTP 503); details only in the error log	Pass
3	Insert registration with unknown student_id	Rejected by the foreign key	Pass
4	Insert the same student/event twice	Rejected by UNIQUE (student_id, event_id)	Pass
5	SGPA 11 inserted directly	Rejected by the CHECK constraint	Pass

KEY QUESTIONS :

1. Is the schema normalized?

Yes, it is in third normal form. A student, an event and a registration are three separate entities. The many-to-many relation between students and events is resolved by the registrations table, so no event data is repeated in each student row and no student data in each event row. Each non-key column depends only on the primary key of its table.

students(id PK, student_id UNIQUE, fullname, email UNIQUE, mobile, course, semester, gender, sgpa, attendance, status)
events(id PK, title, category, event_date, event_time, venue, organizer, description, status, poster)
registrations(id PK, student_id FK, event_id FK, registered_at, UNIQUE(student_id, event_id))

2. Are primary and foreign keys correct?

Every table has an AUTO_INCREMENT INT UNSIGNED primary key. registrations.student_id references students(id) and registrations.event_id references events(id) with ON DELETE CASCADE, so deleting a student or event removes its registrations. UNIQUE keys on student_id and email prevent duplicate students, CHECK constraints keep sgpa between 0 and 10 and attendance between 0 and 100, and all tables are InnoDB (required for foreign keys) with utf8mb4.

3. Is the connection secure?

- Credentials are in one file (db.php) with environment-variable overrides, so they are not repeated in pages; the web server should not expose the SQL files.
- mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT) turns errors into exceptions that are caught; the user sees a generic message and the real error goes to error_log().
- Only prepared statements with bound parameters are used - values never get concatenated into SQL.
- The charset is set to utf8mb4 on the connection; PDO uses ATTR_EMULATE_PREPARES = false so the server does the binding.

$stmt = $conn->prepare('SELECT student_id, fullname, course, semester FROM students WHERE student_id = ?');
$stmt->bind_param('s', $id);
$stmt->execute();
$row = $stmt->get_result()->fetch_assoc();

4. Intermediate and advanced extensions

PDO with prepared statements is in db_pdo.php (pdo_connect()). The stored procedures sp_register_student_for_event (checks that the event is open and the student is not already registered, then inserts the registration) and sp_event_registration_count are in stored_procedures.sql. The full dump is in studenthub_dump.sql.

PRACTICAL-9

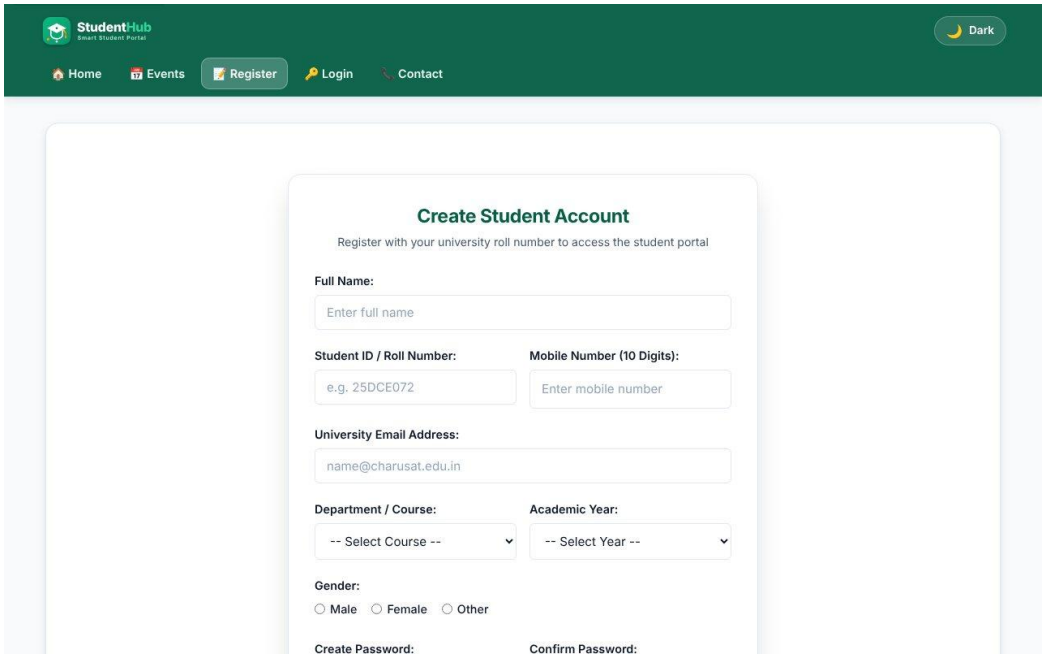
AIM: Implement secure user registration with MySQLi: prepared statements, password_hash(), duplicate checking and consistent frontend/backend validation.

SOURCE CODE :

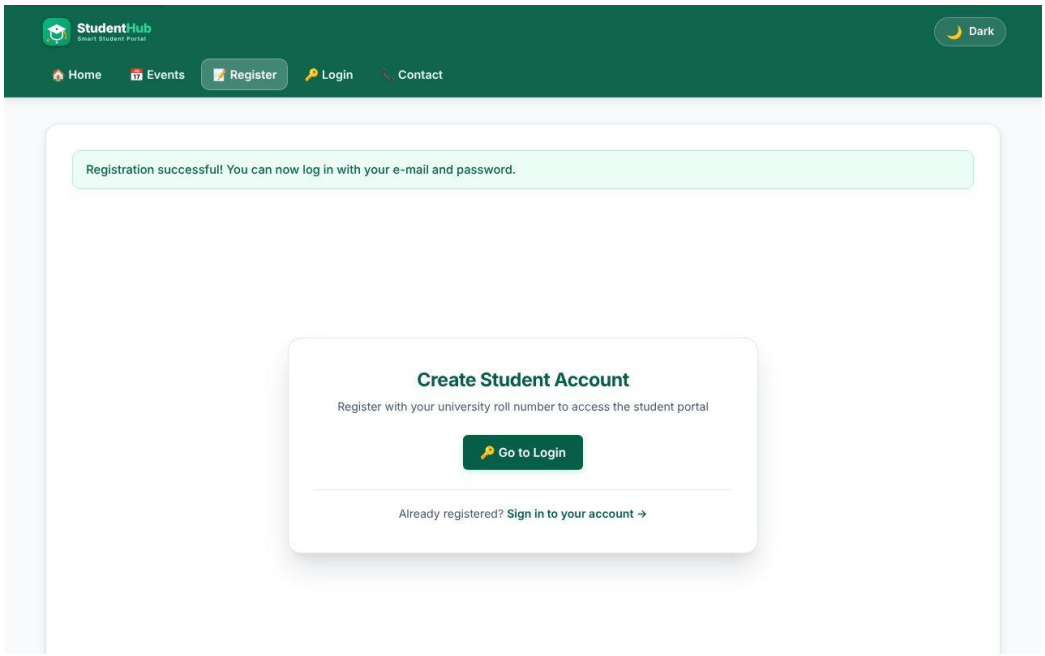
https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-9

GitHub repository: https://github.com/Aarav280907/WEB

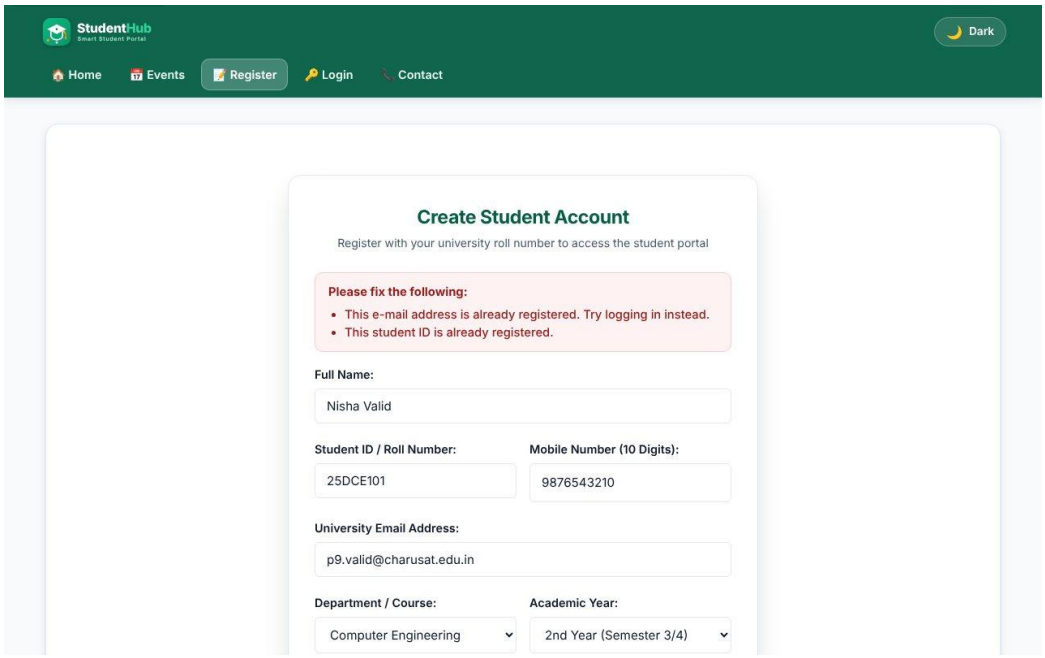
PREVIEW:



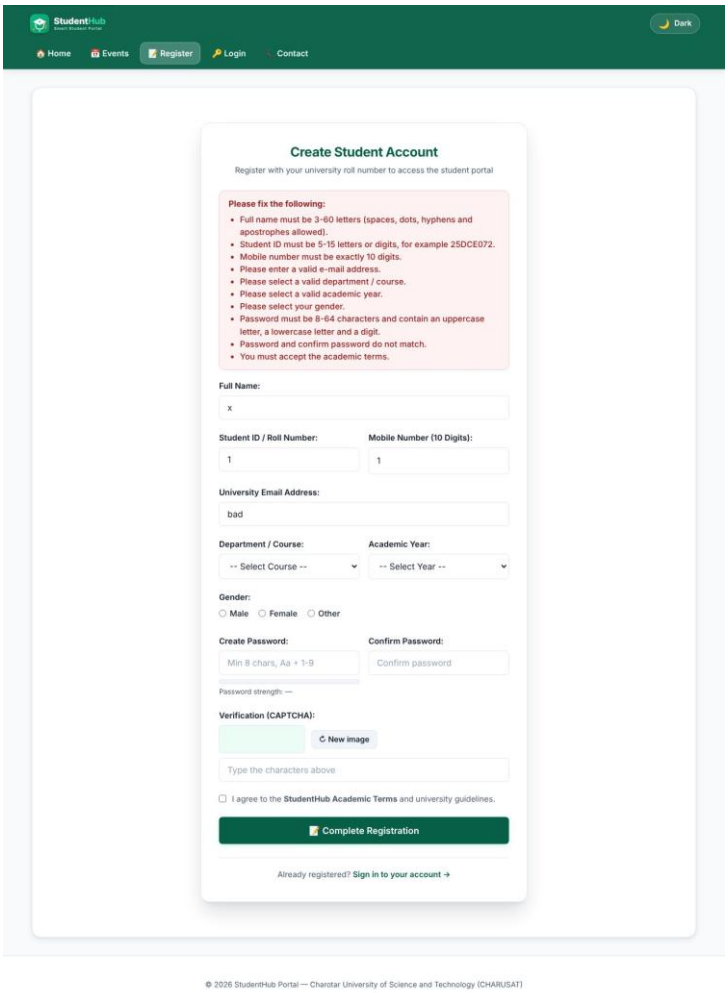
Registration form (same validated form as Practical 5, now saving to MySQL)



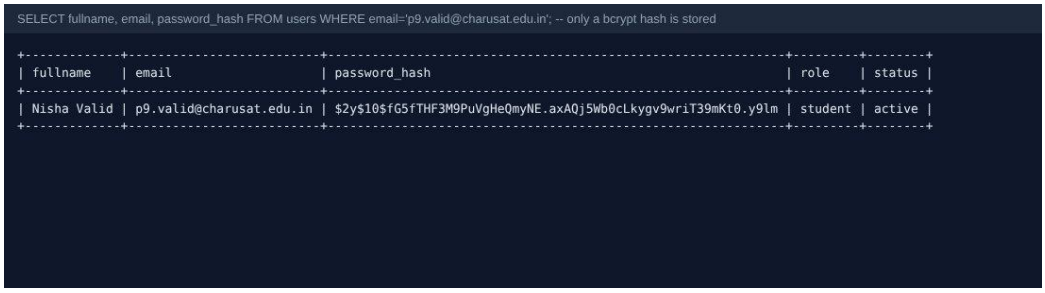
Valid registration - account created, redirected with a success message



Duplicate e-mail and student ID are rejected



Server-side validation errors (client scripts blocked for this test)



Only a bcrypt hash is stored in the users table

TEST CASES :

#	Input / action	Expected result	Result
1	Valid data, new e-mail and roll number	Row inserted; flash message; audit_logs row "user_registered"	Pass
2	Same e-mail and roll number again	Both "already registered" messages; no second row	Pass
3	Name "x", ID "1", e-mail "bad", mobile "1", short passwords that differ	All field errors listed	Pass
4	Name = Robert'); DROP TABLE users;--	Rejected by the name pattern; even if it were accepted it is only bound as a parameter	Pass

5	Name = <script>alert(1)</script>	Rejected; output is always escaped with e()	Pass
6	POST without CSRF token	Rejected: "form session expired"	Pass
7	password_verify(plain, stored hash)	true - the hash is valid and the plain text is not stored	Pass
8	Two simultaneous submissions of the same user	Second one caught by MySQL error 1062 and shown as duplicate	Pass

KEY QUESTIONS :

1. Is password_hash() used?

Yes. The password is hashed with PASSWORD_DEFAULT (bcrypt today, with a random salt and cost factor inside the hash) before the INSERT. The column is VARCHAR(255) to allow future algorithms. The plain password is never stored, logged or shown. Login later uses password_verify().

```
$hash = password_hash($password, PASSWORD_DEFAULT);
$stmt = $conn->prepare('INSERT INTO users (fullname, student_id, email, mobile, course, year, gender, password_hash)
                        VALUES (?, ?, ?, ?, ?, ?, ?, ?)');
$stmt->bind_param('ssssssss', $fullname, $student_id, $email, $mobile, $course, $year, $gender, $hash);
```

2. Is the duplicate check done?

Yes, twice. First a prepared SELECT looks for the same e-mail or student ID and tells the user which one exists. Second, the UNIQUE keys in the table are the final guard: if two requests race, MySQL error 1062 is caught and turned into the same friendly message.

```
$stmt = $conn->prepare('SELECT email, student_id FROM users WHERE email = ? OR student_id = ?');
...
catch (mysqli_sql_exception $ex) { if ($ex->getCode() === 1062) { /* duplicate key */ } }
```

3. Are frontend and backend validations consistent?

Yes. The PHP rules are a copy of the JavaScript patterns of Practical 5 (name, roll number, 10-digit mobile, e-mail, password 8-64 with upper, lower and digit, course / year / gender white-lists, terms). JavaScript gives quick feedback; PHP is the real protection, so a user who disables JavaScript or posts with a tool still gets the same errors (test 3).

4. Advanced extension - CSRF token and audit log

csrf_token() creates a random token in the session, csrf_field() puts it in a hidden input and csrf_valid() compares it with hash_equals(). A successful registration is written to audit_logs (user, action, IP, time) with audit_log(). After success the page redirects (Post/Redirect/Get) so a refresh does not resubmit the form.

PRACTICAL-10

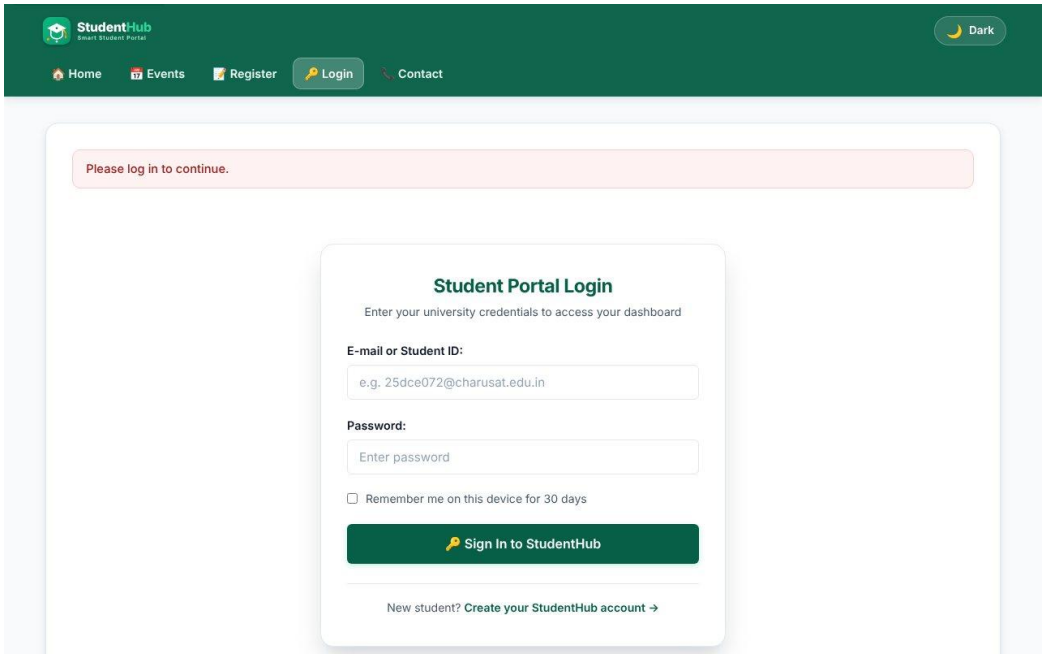
AIM: Implement login and logout with PHP sessions, role-based access (admin / student), idle timeout and a remember-me token.

SOURCE CODE :

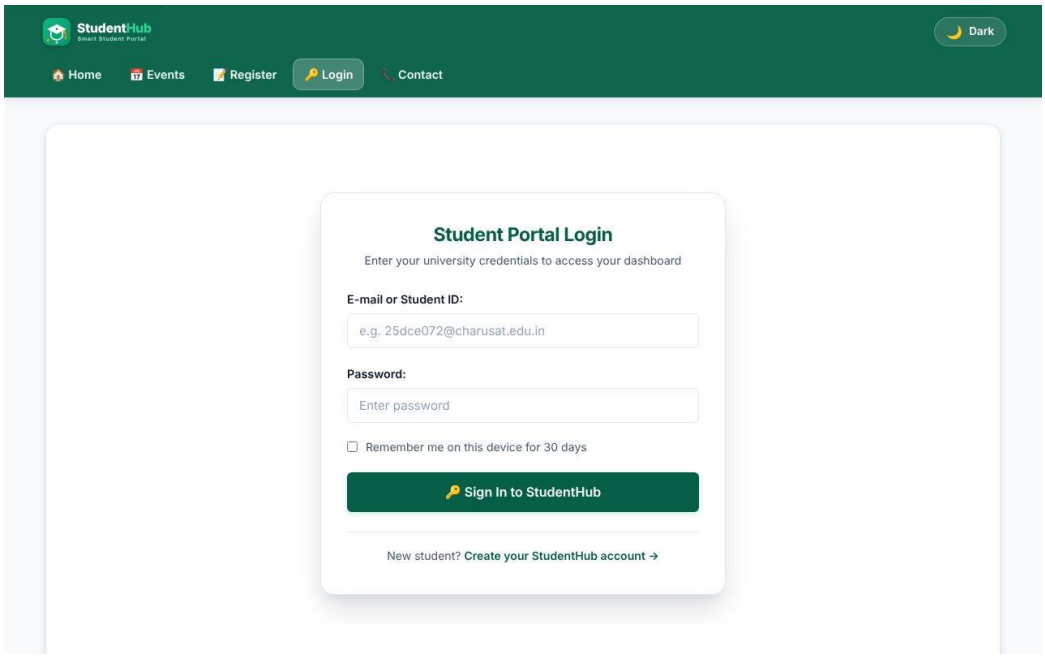
https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-10

GitHub repository: https://github.com/Aarav280907/WEB

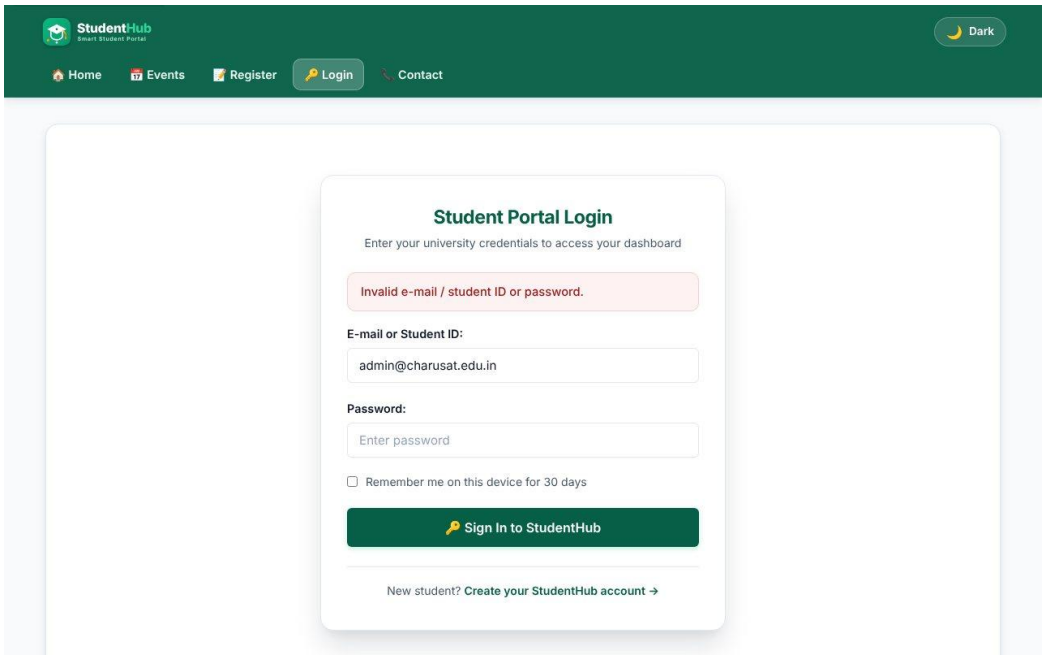
PREVIEW:



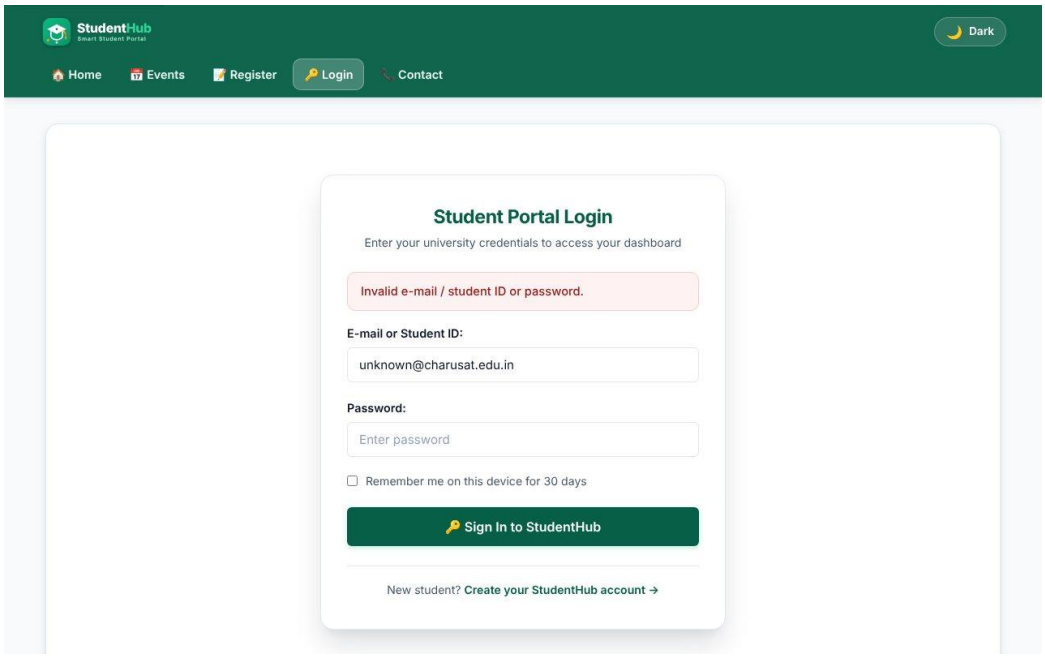
Opening a protected page without login redirects to the login page



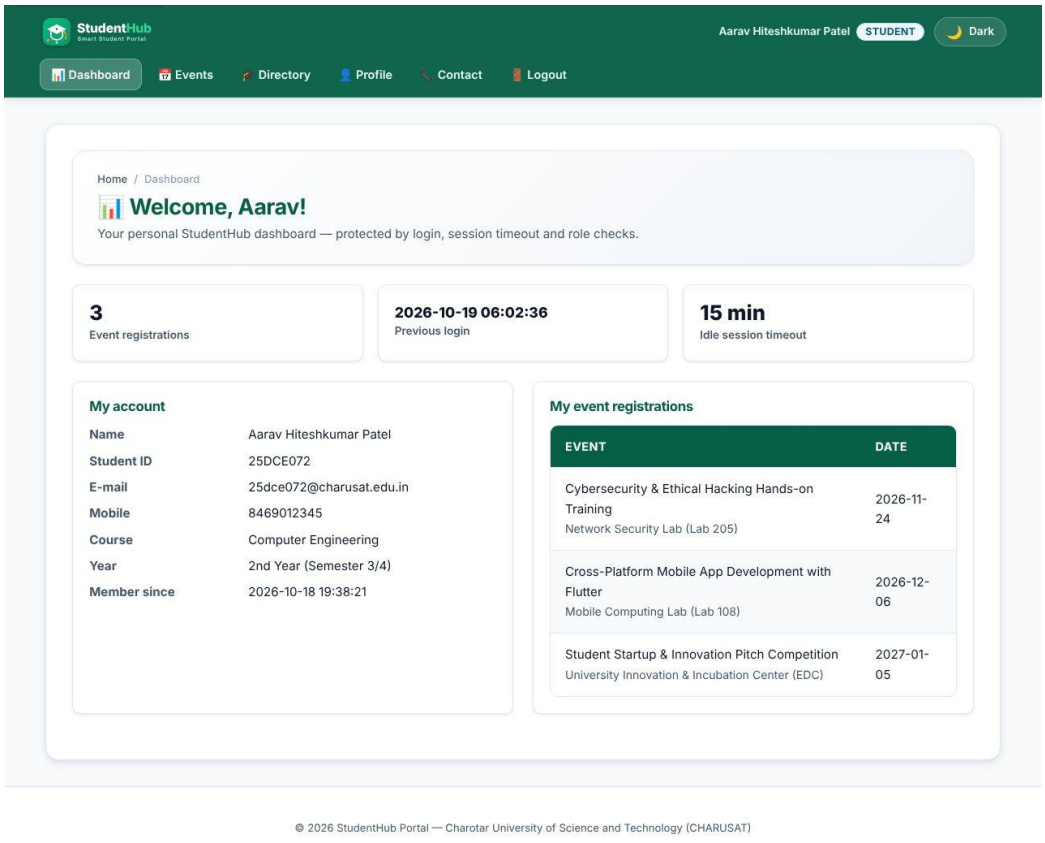
Login page (e-mail or roll number, password, remember me)



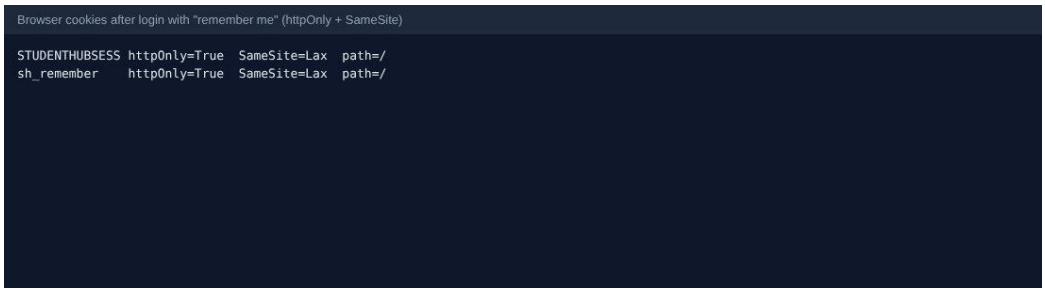
Failed login - wrong password



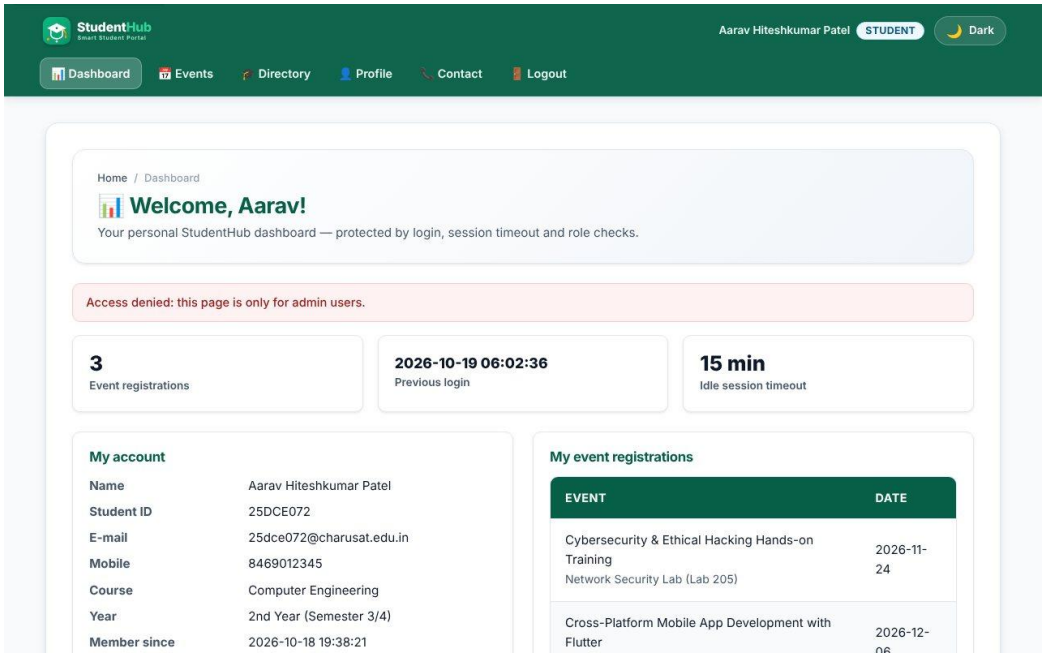
Unknown user gets the same generic message (no user enumeration)



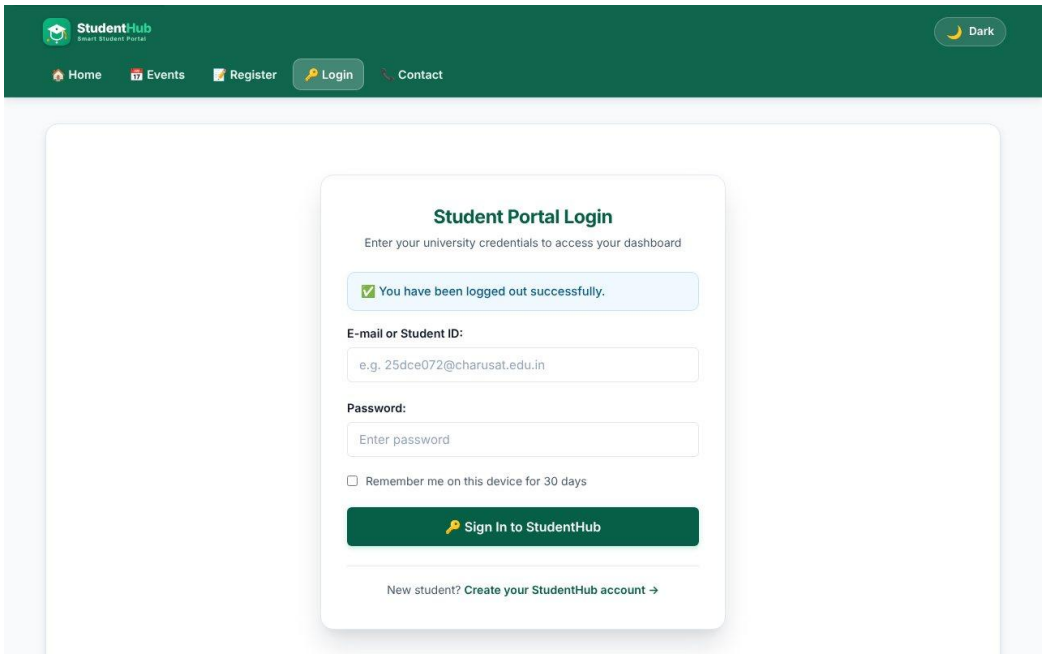
Student login success - student dashboard with previous-login time



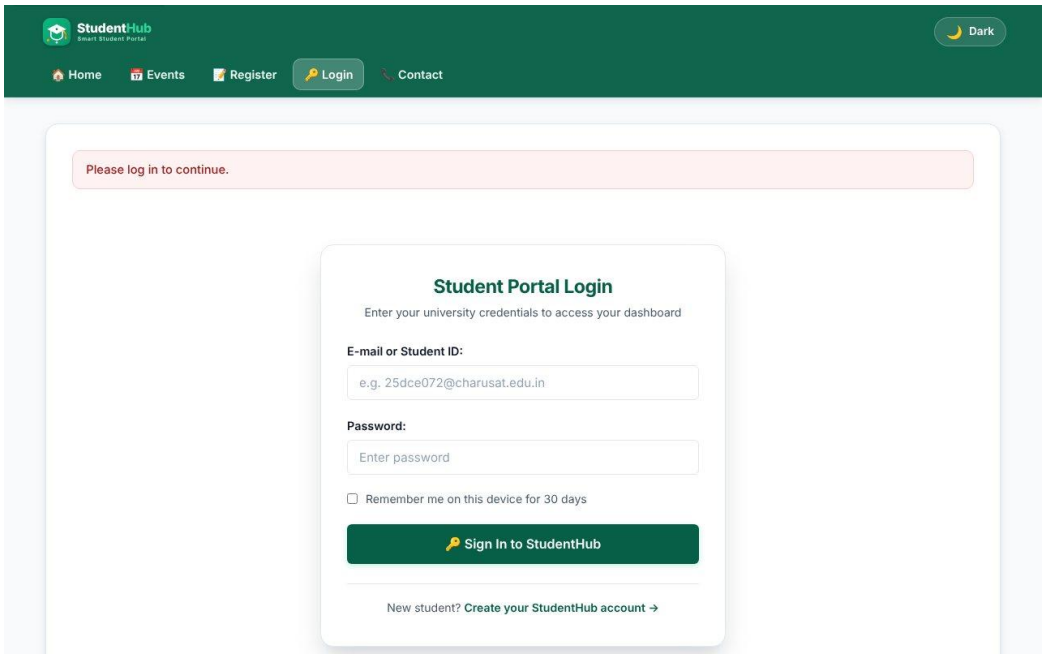
Cookie attributes: HttpOnly, SameSite; remember-me cookie



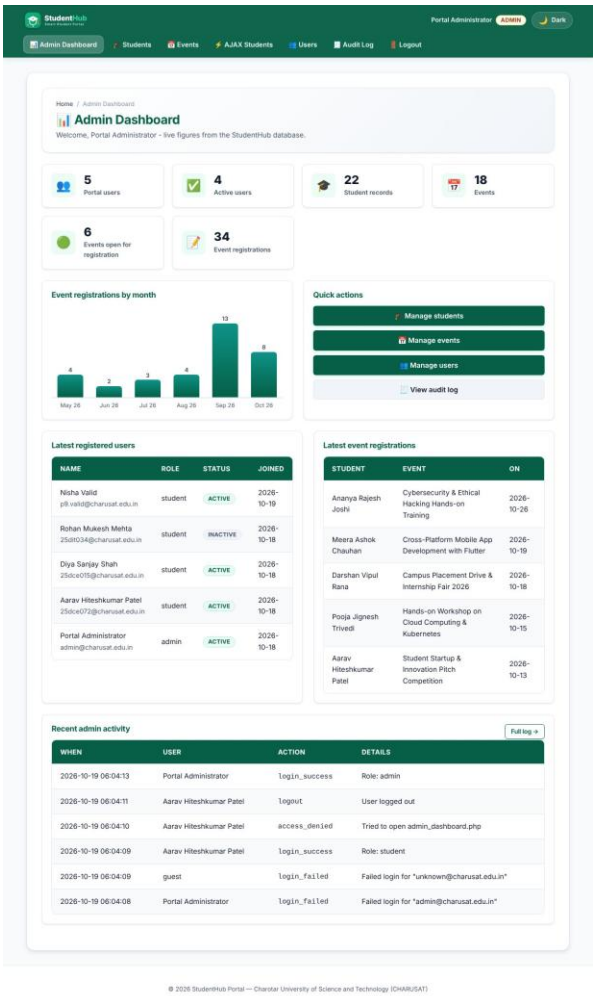
Unauthorized access: a student opening an admin page is sent back with "Access denied"



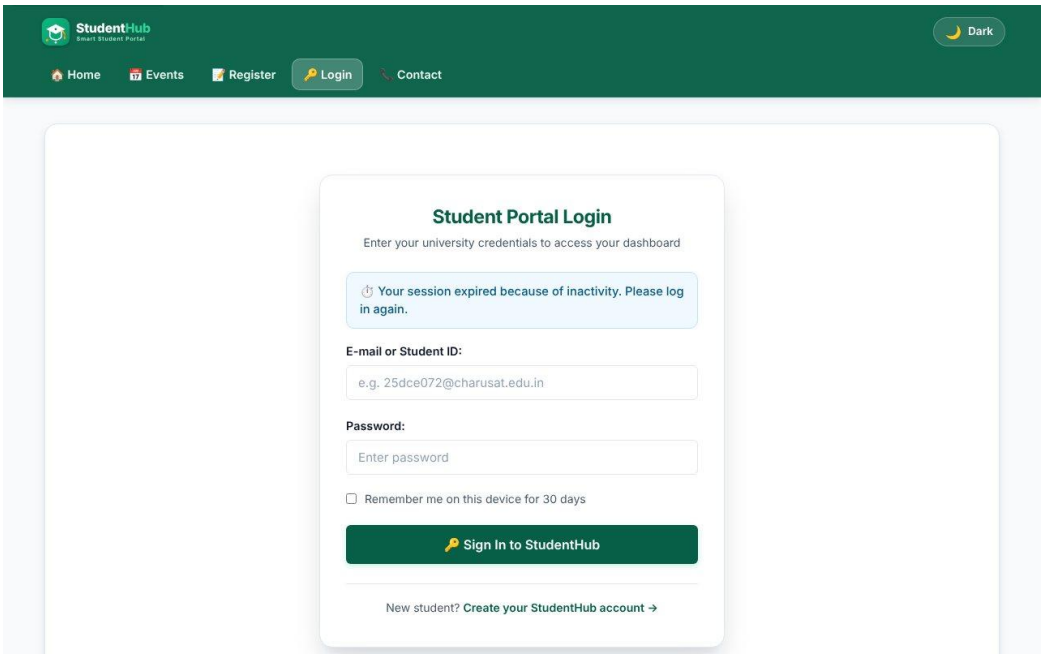
Logout - session destroyed, login page with message



After logout the dashboard is blocked again



Admin login goes to the admin dashboard



Idle timeout (test server set to 4 seconds): redirect to login with the timeout message

TEST CASES :

#	Input / action	Expected result	Result
1	Open dashboard.php as guest	Redirect to login.php	Pass
2	Correct e-mail + wrong password	"Invalid credentials" message, audit row login_failed	Pass
3	Unknown user	Same message as test 2	Pass
4	Student logs in (roll number)	Student dashboard; session ID changes after login	Pass
5	Admin logs in	Admin dashboard	Pass
6	Student opens admin page	Access denied, redirected to own dashboard	Pass
7	Logout	Session and cookie removed; back button cannot reopen the dashboard	Pass
8	Stay idle longer than SESSION_TIMEOUT	Next request redirects to login?timeout=1	Pass
9	Inactive account with correct password	Login blocked	Pass
10	5 wrong passwords in a row	Login locked for 30 seconds	Pass
11	Remember me, delete session cookie	Logged in again from the token; token rotated; old token rejected	Pass
12	Cookie flags	HttpOnly and SameSite=Lax set on the session cookie	Pass

KEY QUESTIONS :

1. Are sessions started and destroyed securely?

Sessions use a fixed name, an HttpOnly cookie (JavaScript cannot read it), SameSite=Lax, Secure when HTTPS is used and session.use_strict_mode so unknown IDs are rejected. session_regenerate_id(true) is called right after a successful login to prevent session fixation. Logout

empties $_SESSION, expires the cookie, calls session_destroy() and deletes the remember-me token from the database.

```
session_set_cookie_params(['lifetime' => 0, 'path' => '/', 'secure' => $https, 'httponly' => true, 'samesite' => 'Lax']);
ini_set('session.use_strict_mode', '1');
...
session_regenerate_id(true);
```

2. Is password_verify() used correctly?

Yes. The user row is loaded with a prepared statement and the typed password is compared with the stored hash only through password_verify(). The same error text is shown for "unknown user" and "wrong password", so an attacker cannot find out which e-mails exist.

```
if ($user && password_verify($password, $user['password_hash'])) {
    login_user($conn, $user);
    redirect($user['role'] === 'admin' ? ADMIN_HOME : STUDENT_HOME);
}
```

3. Is unauthorized dashboard access blocked?

Every protected page starts with require_login() or require_role('admin'). A guest is redirected to the login page; a logged-in student who requests an admin page gets an "Access denied" flash message and is sent to the student dashboard. The role is read from the session that was created from the database, never from a URL or form field.

```
$admin = require_role('admin');   // top of every admin page
```

4. Is session timeout implemented?

Yes. The time of the last request is stored in $_SESSION['last_activity']. On each request auth_boot() compares it with SESSION_TIMEOUT (15 minutes, changeable through config); if it is older, the timeout is written to the audit log, the session is destroyed and the user goes to login.php?timeout=1. For the screenshot a test server with a 4-second timeout was used.

```
if (time() - ($_SESSION['last_activity'] ?? 0) > SESSION_TIMEOUT) {
    destroy_session();
    redirect(LOGIN_URL . '?timeout=1');
}
$_SESSION['last_activity'] = time();
```

5. Are admin and student users redirected to the correct dashboards?

Yes, after login the role decides the target: admin -> Practical-14/admin_dashboard.php, student -> Practical-10/dashboard.php. Opening login.php while already logged in sends the user to the same place. The menu is also role based (guest, student, admin) so users only see links they may use.

6. Intermediate and advanced extensions

Last login: users.last_login is updated at each login and the previous value is shown on the dashboard. Remember me: a random selector:validator pair is placed in a cookie; only the SHA-256 hash of the validator is stored in remember_tokens, with an expiry. Every use issues a new token, and a stolen old token stops working. Deactivating a user (Practical 14) removes their tokens.

PRACTICAL-11

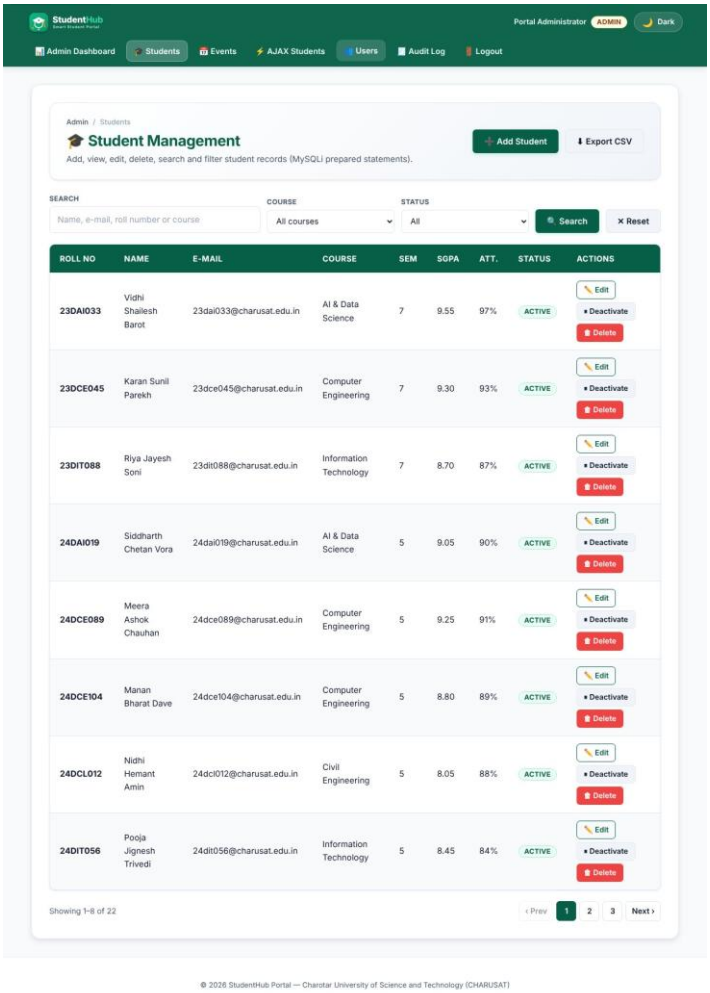
AIM: Build student management (CRUD) with PHP and MySQL: list, add, edit, delete, search and filter, with at least 20 seeded records.

SOURCE CODE :

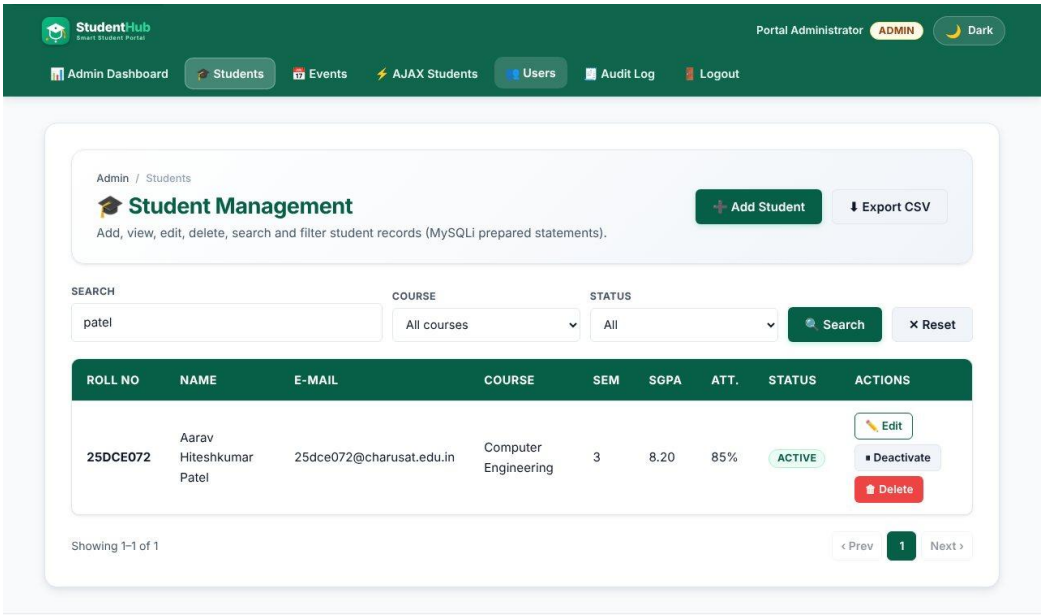
https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-11

GitHub repository: https://github.com/Aarav280907/WEB

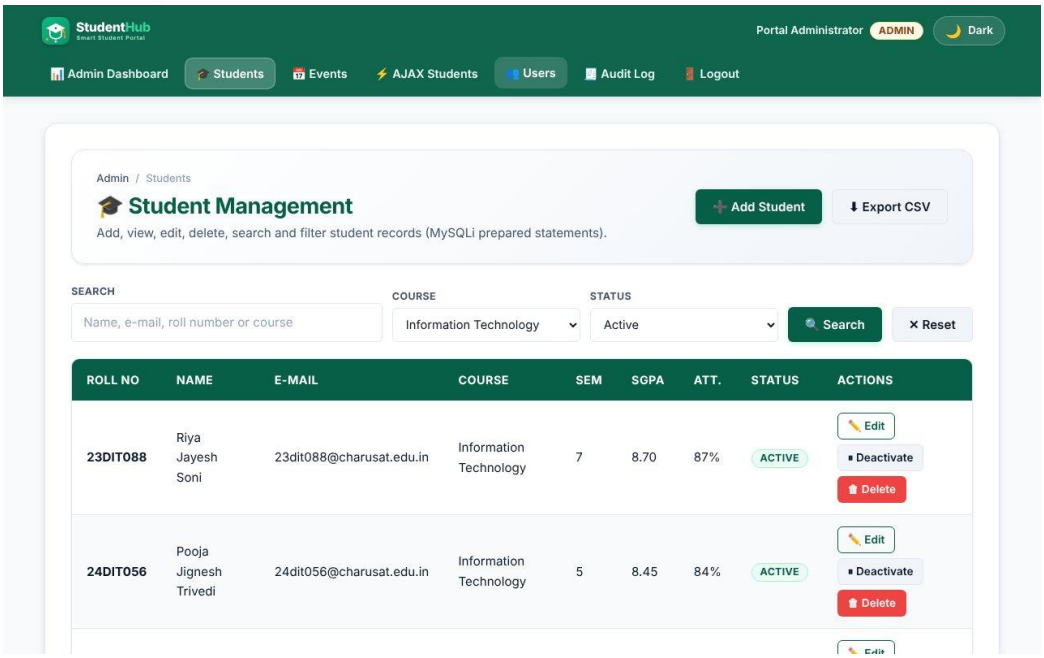
PREVIEW:



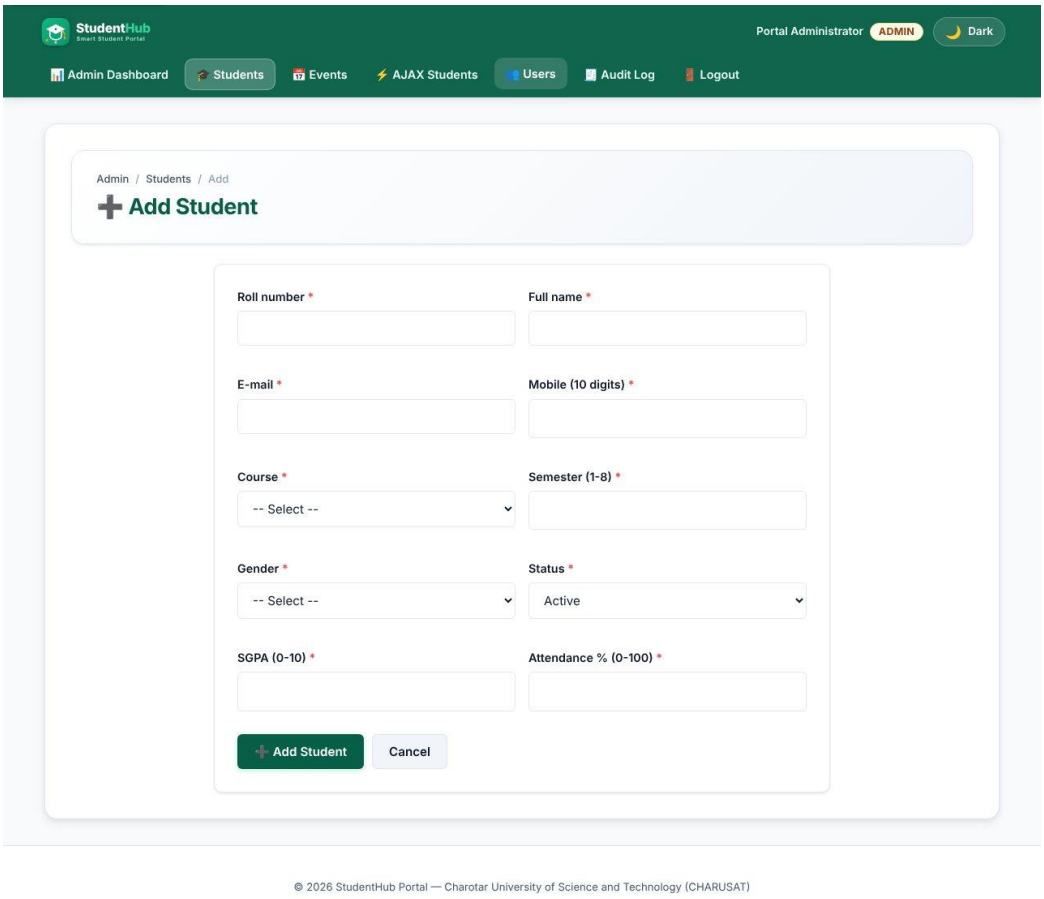
Student list: 22 seeded records, 8 per page, with pagination



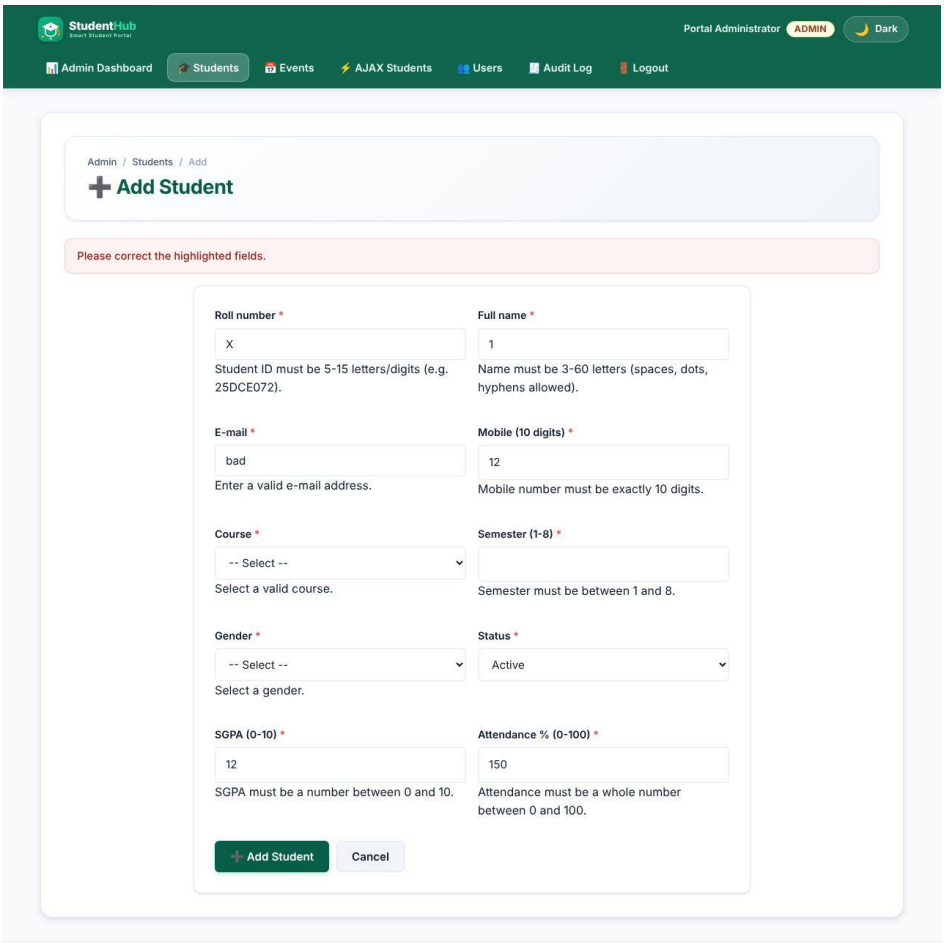
Search by name



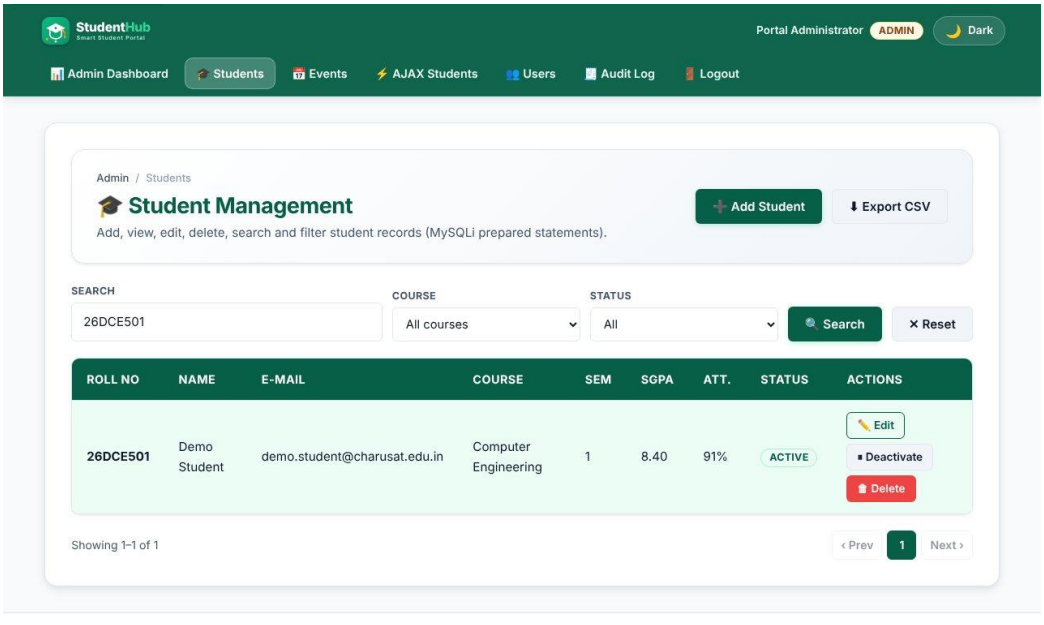
Course + status filter



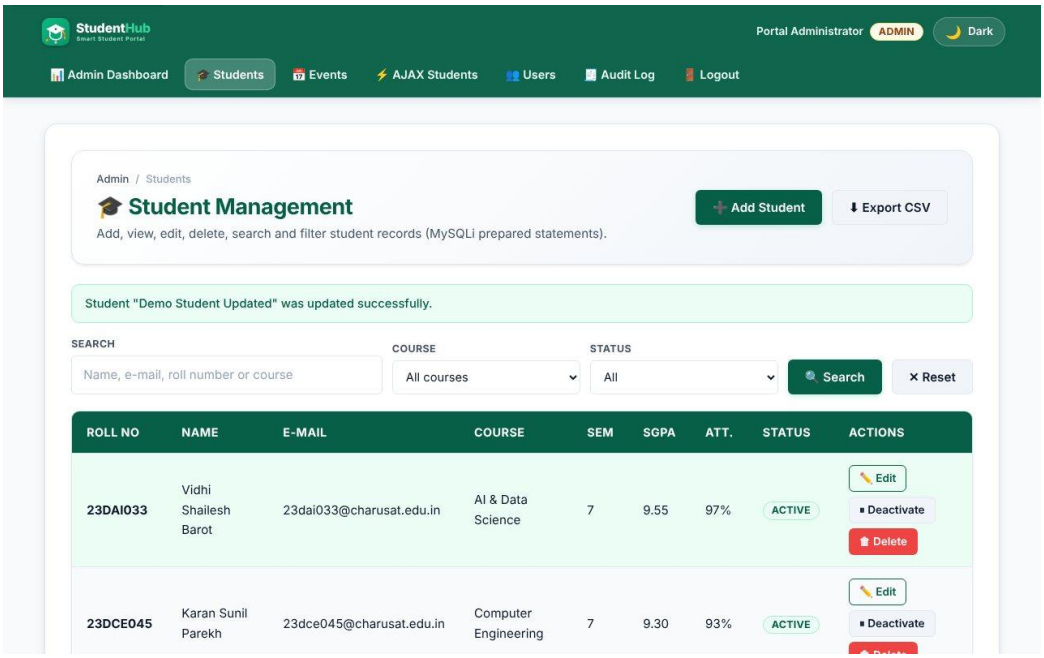
Add student form



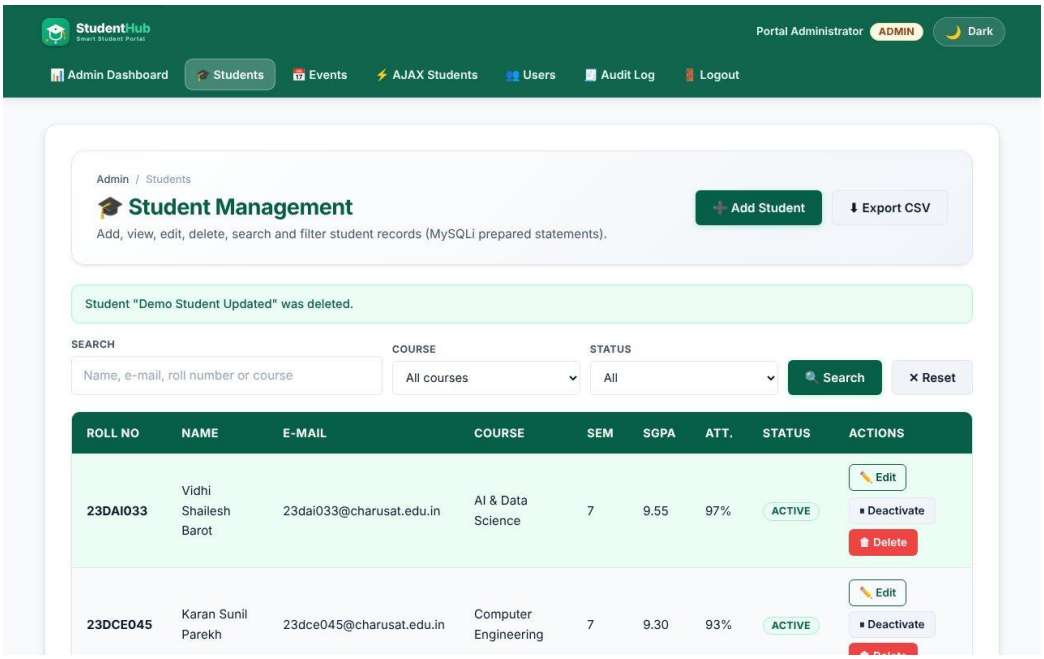
Server-side validation errors on the form



INSERT - new student shown after saving



UPDATE - edited student saved



DELETE - after the confirmation dialog the record is removed

students_export.php (downloaded CSV, first rows)

```
"Roll No",Name,Email,Mobile,Course,Semester,Gender,SGPA,"Attendance %",Status
23DAI033,"Vidhi Shailesh Barot",23dai033@charusat.edu.in,9714066554,"AI & Data Science",7,Female,9.55,97,active
23DCE045,"Karan Sunil Parekh",23dce045@charusat.edu.in,9824088990,"Computer Engineering",7,Male,9.30,93,active
23DIT088,"Riya Jayesh Soni",23dit088@charusat.edu.in,9727711335,"Information Technology",7,Female,8.70,87,active
24DAI019,"Siddharth Chetan Vora",24dai019@charusat.edu.in,9712344556,"AI & Data Science",5,Male,9.05,90,active
24DCE089,"Meera Ashok Chauhan",24dce089@charusat.edu.in,9687099887,"Computer Engineering",5,Female,9.25,91,active
24DCE104,"Manan Bharat Dave",24dce104@charusat.edu.in,9601255667,"Computer Engineering",5,Male,8.80,89,active
24DCL012,"Nidhi Hemant Amin",24dcl012@charusat.edu.in,9824011103,"Civil Engineering",5,Female,8.05,88,active
```

CSV export (intermediate extension)

TEST CASES :

#	Input / action	Expected result	Result
1	Open list	22 seeded students, page 1 shows 8	Pass
2	Search "patel"	Matching rows only (name, e-mail, roll number, course are searched)	Pass
3	Search "%"	Treated as normal text, not a wildcard (LIKE escaped)	Pass
4	Course + status filter	Filters combine with search and pagination	Pass
5	Add valid student	INSERT; success message; audit log row	Pass
6	Add duplicate roll number / e-mail	Error from the UNIQUE key (code 1062)	Pass
7	Invalid e-mail, 2-digit mobile, SGPA 12, attendance 150	Field errors, nothing saved	Pass
8	Edit name	UPDATE; success message	Pass
9	Delete	Browser confirm: "Delete ... permanently?"; row removed after OK	Pass
10	Deactivate / Activate (soft delete)	Status switches; row is kept	Pass
11	POST action with a bad CSRF token	"Invalid request" - nothing changes	Pass
12	Student role opens the list	Access denied	Pass
13	Export CSV	File with BOM, header row and the filtered rows	Pass

SQL dump:

Practical-11/students_dump.sql contains the students table and its 22 seeded rows.

KEY QUESTIONS :

1. Do SELECT, INSERT, UPDATE and DELETE work?

Yes, all four are used with prepared statements. SELECT: students_list.php (with COUNT(*) for pagination and LIMIT/OFFSET); INSERT and UPDATE: student_form.php (one form for add and edit, chosen by ?id=); DELETE and status change: student_action.php.

```
$stmt = $conn->prepare('INSERT INTO students (student_id, fullname, email, mobile, course, semester, gender, sgpa, attendance, status) VALUES (?,?,?,?,?,?,?,?,?,?)');
$stmt = $conn->prepare('UPDATE students SET fullname=?, ... WHERE id=?');
$stmt = $conn->prepare('DELETE FROM students WHERE id = ?');
```

2. Is search by name, e-mail and course implemented?

Yes. One text box searches name, e-mail, roll number and course with LIKE, and the Course and Status filters are added to the WHERE clause only when selected. The WHERE part is built from fixed text; the values are bound. like_escape() escapes % and _ so users cannot use them as wildcards.

$where[] = '(fullname LIKE ? OR email LIKE ? OR student_id LIKE ? OR course LIKE ?)';
$like = '%' . like_escape($q) . '%';

3. Are deletes confirmed?

Yes. The delete button is inside a form with data-confirm="Delete NAME permanently?"; portal-init.js shows window.confirm() and cancels the submit unless the user agrees. Deleting is also a POST request with a CSRF token (not a link), so another web page cannot trigger it.

4. Intermediate and advanced extensions

CSV export (students_export.php) writes the current search/filter result with a UTF-8 BOM for Excel and prefixes cells that start with =, +, - or @ to stop CSV formula injection. Soft delete: the Deactivate button sets status = 'inactive' instead of removing the row; the list can be filtered by status.

PRACTICAL-12

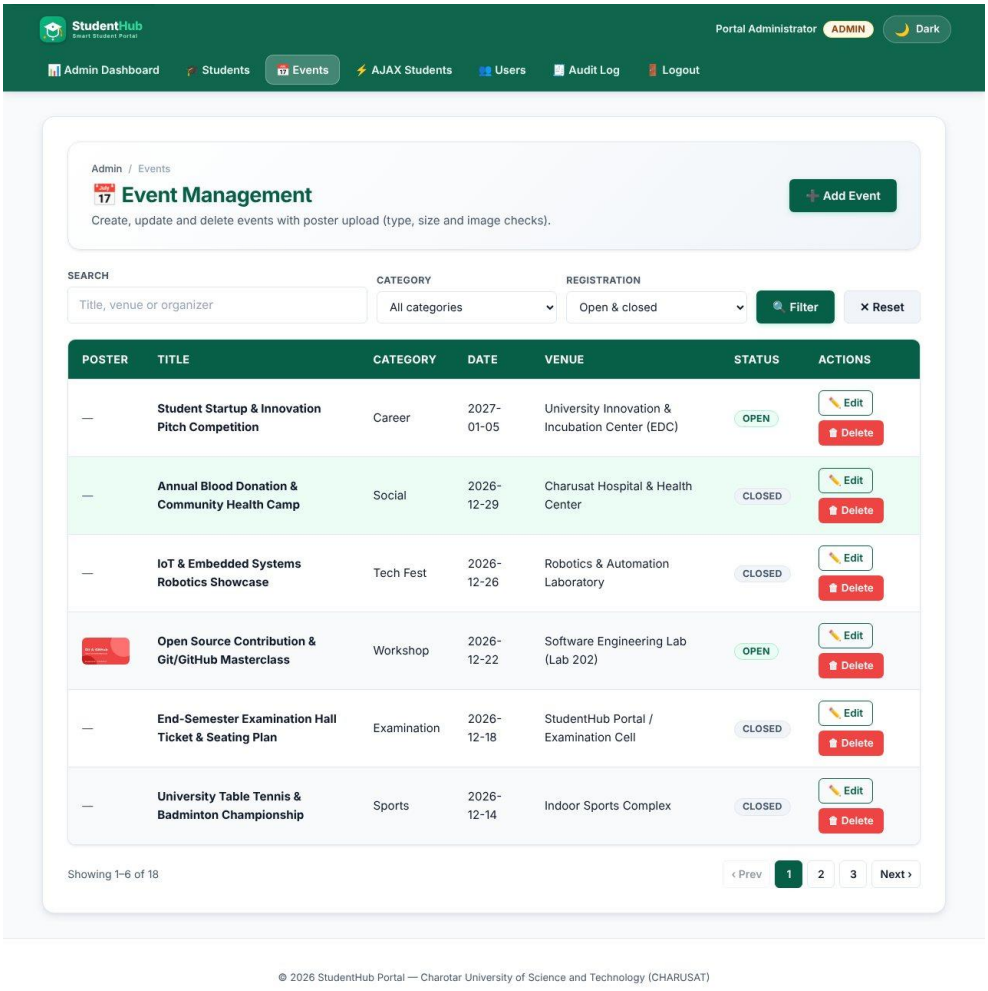
AIM: Build event management (CRUD) with poster upload, server-side upload validation and success/failure messages.

SOURCE CODE :

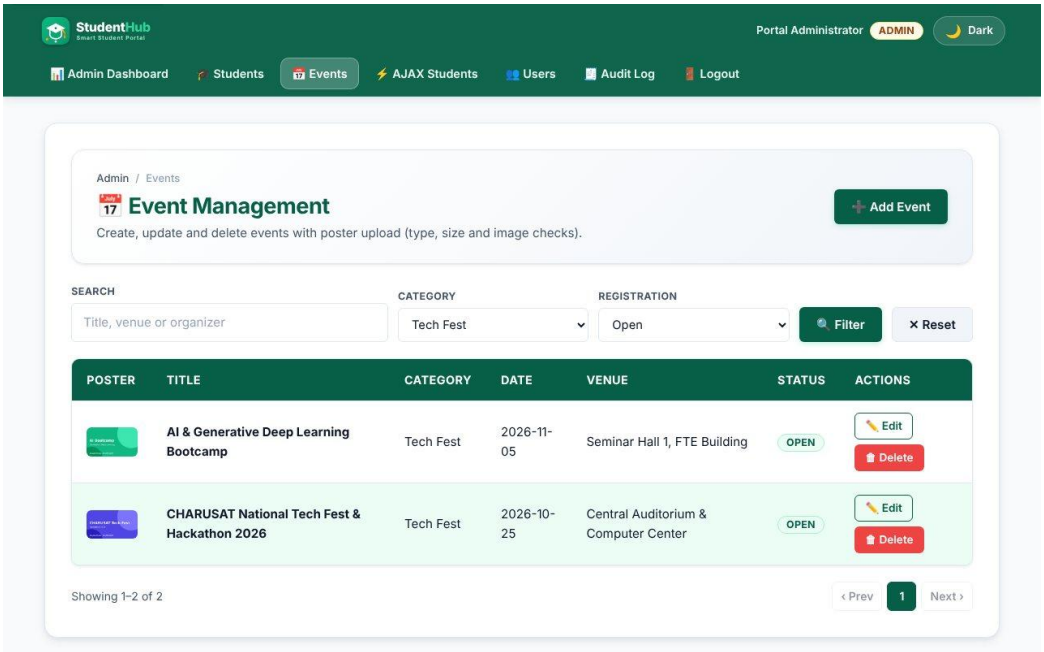
https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-12

GitHub repository: https://github.com/Aarav280907/WEB

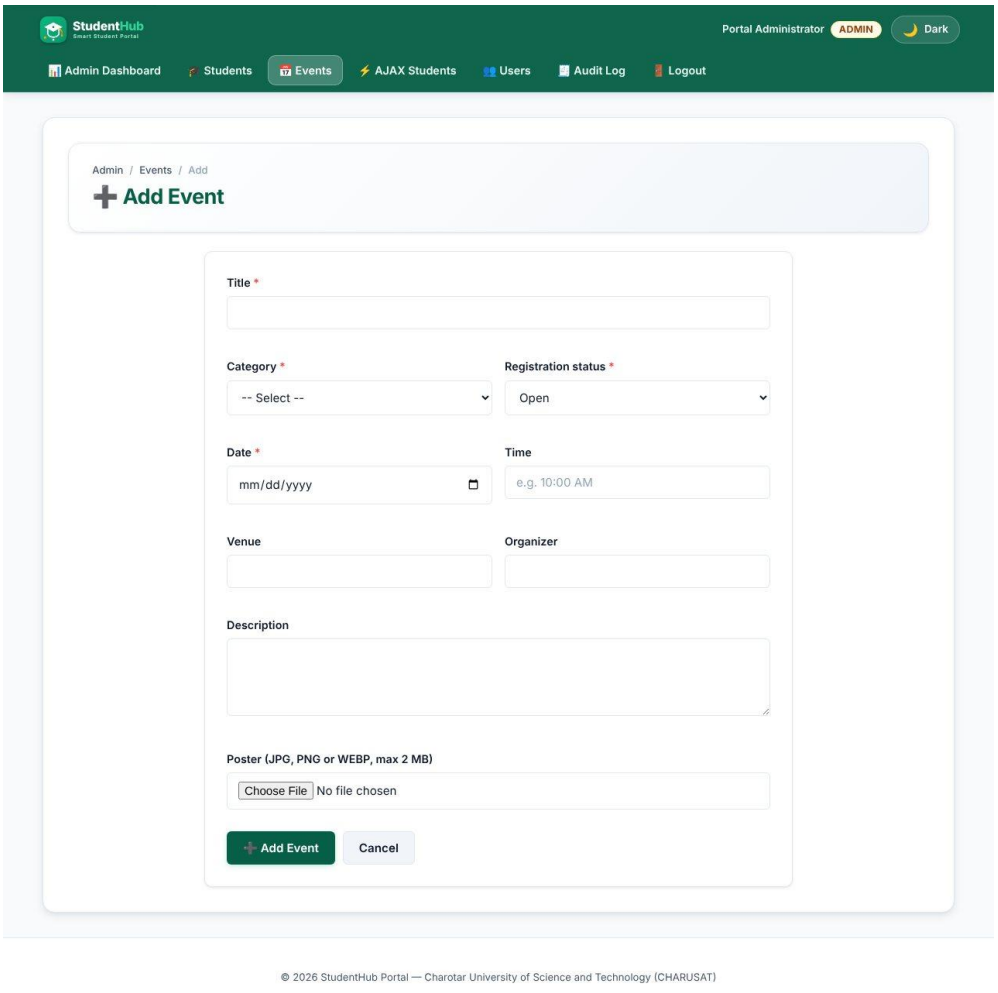
PREVIEW:



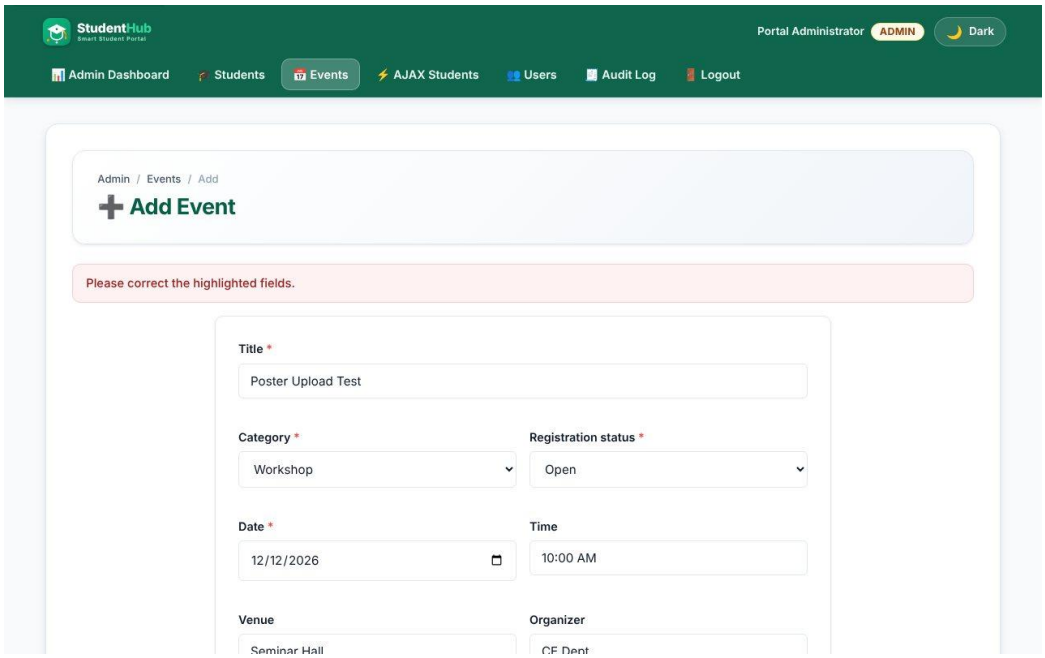
Event list with poster thumbnails



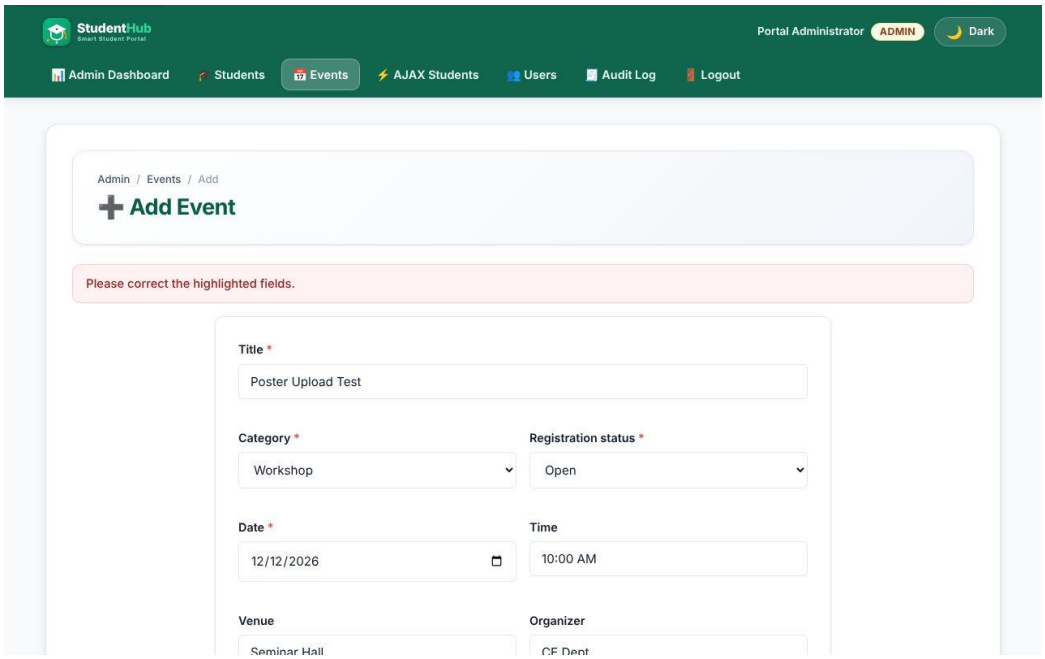
Category and open/closed filter (intermediate extension)



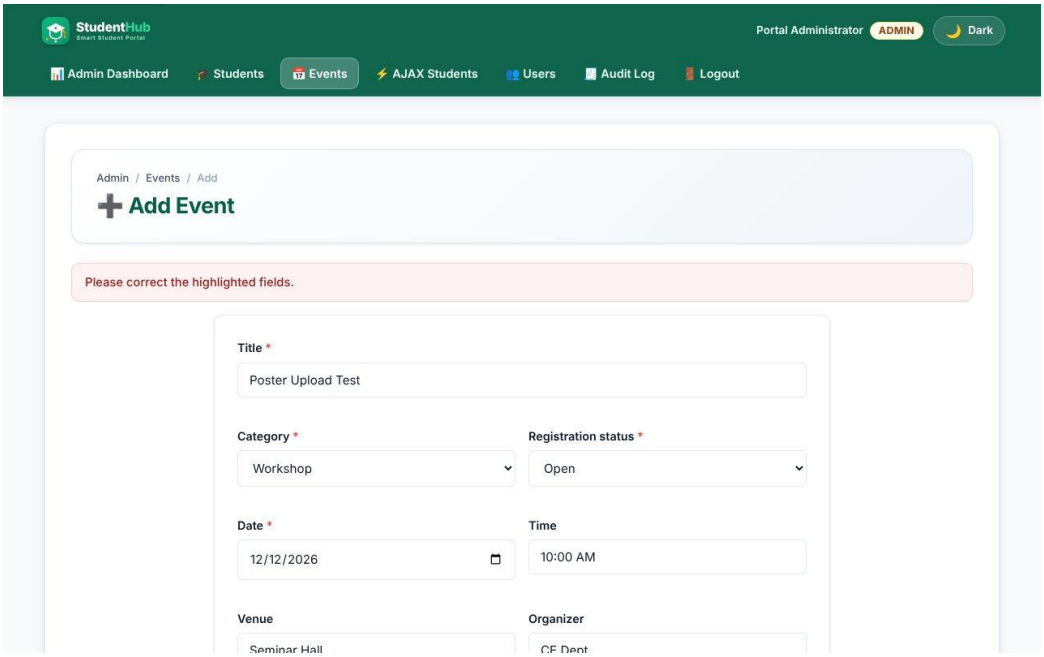
Add event form with poster field



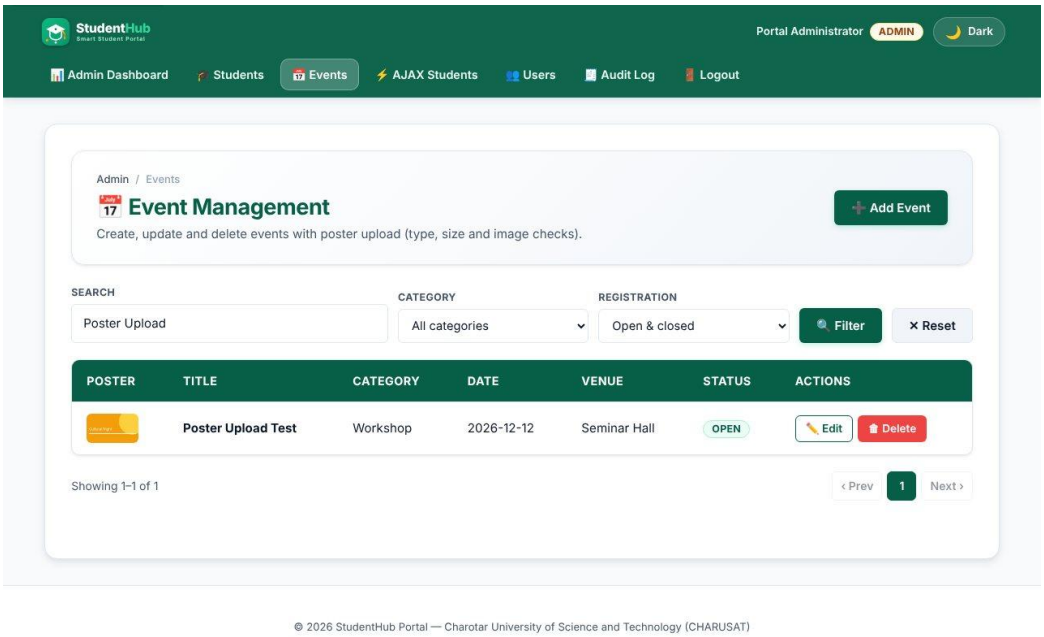
Upload validation: a PHP script renamed as a poster is rejected



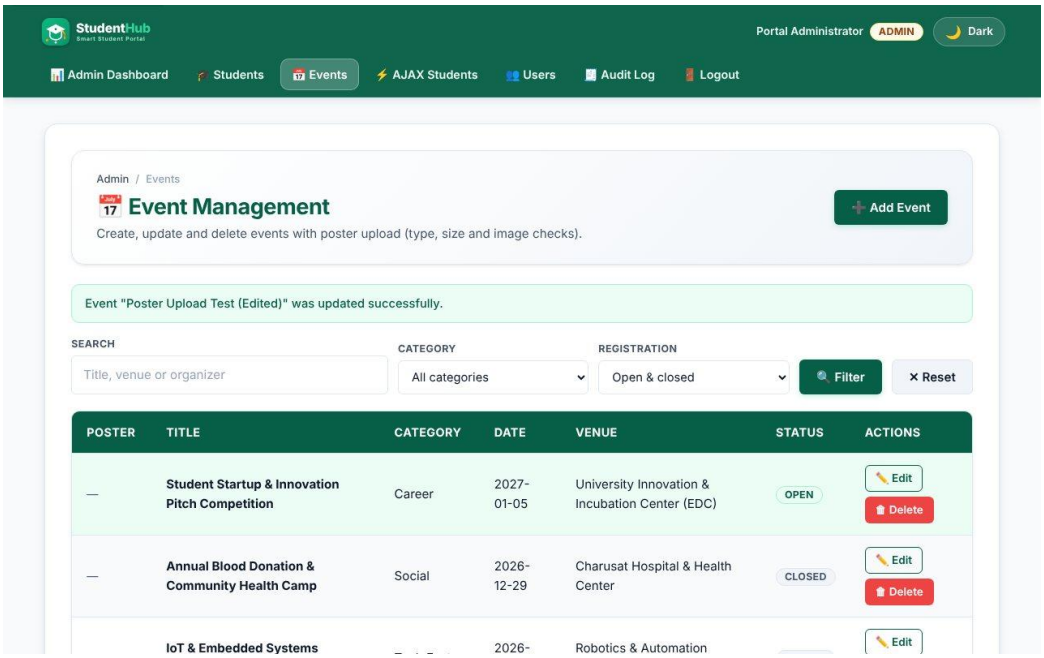
Upload validation: a text file is rejected



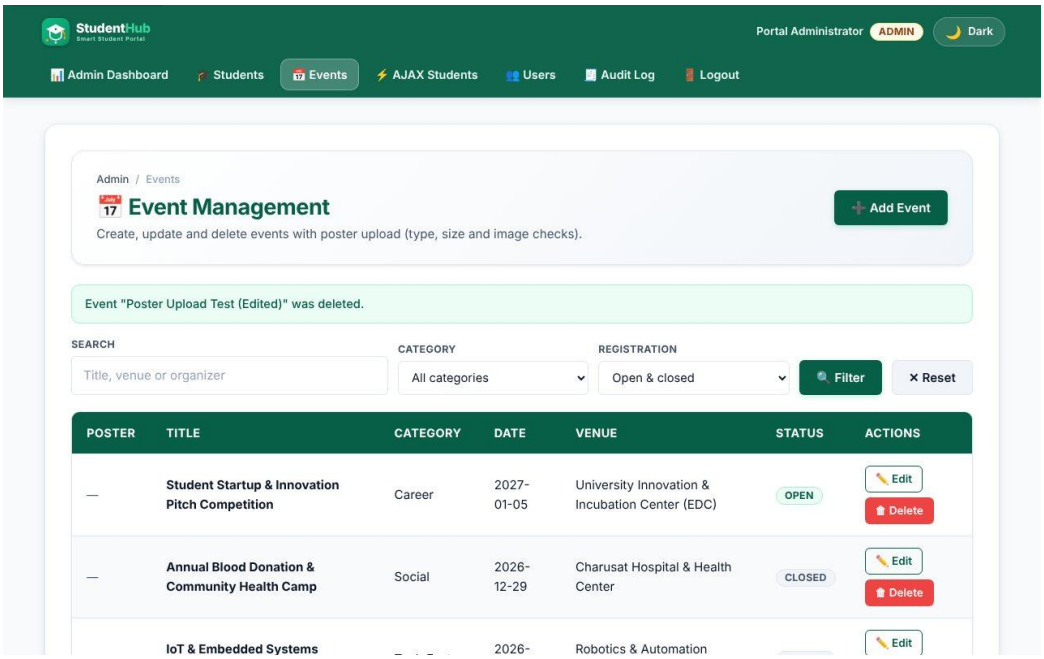
Upload validation: a file larger than 2 MB is rejected



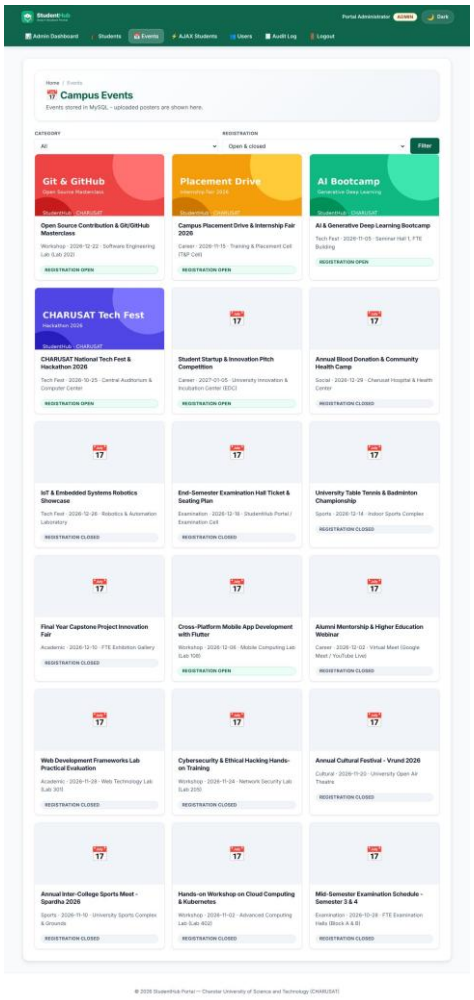
INSERT: event added with a valid JPG poster



UPDATE: event edited (status closed)



DELETE: event and its poster file removed



Public events page reading from MySQL with posters

Practical-12/uploads/ (random names; .htaccess blocks script execution)

```
total 132
drwxr-xr-x 2 root root  4096 Oct 19 06:04 .
drwxr-xr-x 3 root root  4096 Oct 18 19:35 ..
-rw-r--r-- 1 root root   299 Oct 18 19:35 .htaccess
-rw-r--r-- 1 root root 28824 Oct 18 19:35 849267c34b2312b0ca3433c6.jpg
-rw-r--r-- 1 root root 26838 Oct 18 19:35 9f034d34ab25b8d7cfef8045.jpg
-rw-r--r-- 1 root root 26187 Oct 18 19:35 f388e4787b4444de2cbf8636.jpg
-rw-r--r-- 1 root root 25129 Oct 18 19:35 fcaab2971793d7a1ee42bfb0.jpg
-rw-r--r-- 1 root root    31 Oct 18 19:35 index.php

# Practical 12 - upload directory protection.
# Uploaded files must never be executed as code, whatever their name says.
<FilesMatch "\.(php|phtml|php[0-9]|phar|pl|py|cgi|sh|html?|js)$">
  Require all denied
</FilesMatch>
php_flag engine off
Options -Indexes -ExecCGI
AddType text/plain .php .phtml
```

Upload folder: random file names and the protective .htaccess

TEST CASES :

#	Input / action	Expected result	Result
1	Add event with valid poster (JPG)	Saved; file stored with a random name; audit log row	Pass
2	evil.php as poster	Rejected - only JPG, PNG, WEBP allowed	Pass
3	PHP code saved as fake.jpg	Rejected - real MIME type is text, not an image	Pass

4	doc.txt	Rejected	Pass
5	4 MB random image	Rejected - larger than 2 MB	Pass
6	No poster	Event saved without a poster (optional)	Pass
7	Title "x"	Error: 5-150 characters	Pass
8	Edit event, keep poster	Old poster kept	Pass
9	Edit event, upload new poster	New file saved, old file deleted	Pass
10	Delete event	Row and poster file removed; audit log row	Pass
11	Filter category = Tech Fest, status = open	Only matching events	Pass

SQL dump:

Practical-12/events_dump.sql contains the events table with 18 rows; sample_posters.sql attaches the demo posters.

KEY QUESTIONS :

1. Do add, update and delete work?

Yes. events_admin.php lists and filters events, event_form.php inserts or updates (?id= decides), event_action.php deletes (POST + CSRF). Each change is written to audit_logs. If saving to the database fails after a file was stored, the new file is deleted again so no orphan files remain.

2. Is the UI linked to the database?

Yes. The admin list and the public gallery (events_public.php) are both read from the events table; the poster column stores only the file name and poster_url() builds a safe URL from it. Category and status come from white-lists (CATEGORIES, open/closed).

3. Are uploads validated for type and size?

Yes, in save_poster(): (1) the PHP upload error code is checked; (2) size <= 2 MB (also limited in the form with MAX_FILE_SIZE); (3) the real MIME type is read from the file content with finfo, not from the name the browser sends; only image/jpeg, image/png and image/webp pass; (4) getimagesize() must succeed, proving it is a real image; (5) the stored name is random (bin2hex(random_bytes(12))) with the extension taken from the detected MIME type, so a user can never choose the file name or extension.

```
$mime = (new finfo(FILEINFO_MIME_TYPE))->file($file['tmp_name']);
if (!isset(POSTER_TYPES[$mime])) return [null, 'Only JPG, PNG or WEBP images are allowed.'];
if (@getimagesize($file['tmp_name']) === false) return [null, 'The file is not a valid image.'];
$name = bin2hex(random_bytes(12)) . '.' . POSTER_TYPES[$mime];
```

4. Are success and failure messages shown?

Yes. A green flash message appears after add / update / delete (Post/Redirect/Get), and a red alert plus a message under the poster field appears for every failed validation or upload.

5. Advanced extension - image renaming, folder protection and audit log

Images are renamed randomly. Practical-12/uploads/.htaccess denies execution of .php/.phtml/.html/.js files and turns the PHP engine off in that folder, and an index.php returns 403 to avoid directory listing. Create, update and delete are recorded in audit_logs. Four sample posters are attached to seeded events by sample_posters.sql.

PRACTICAL-13

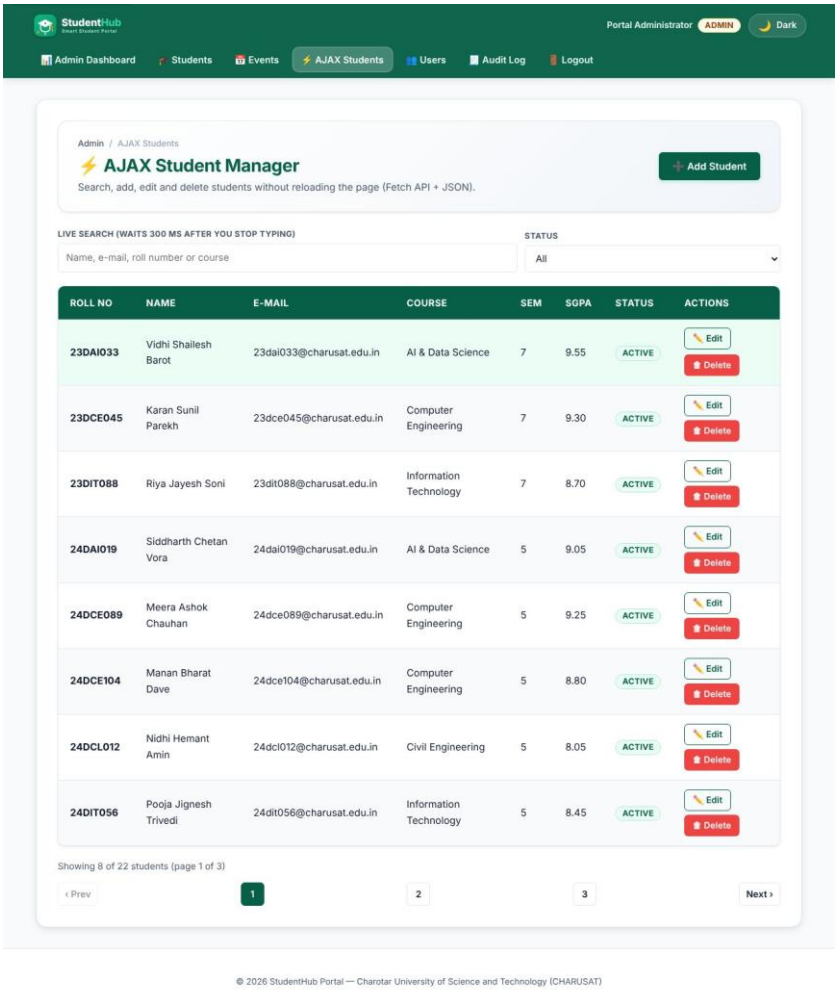
AIM: Create a PHP JSON API (GET / POST / PUT / DELETE) and use the Fetch API (AJAX) for CRUD without page reload, with loading, success and error states.

SOURCE CODE :

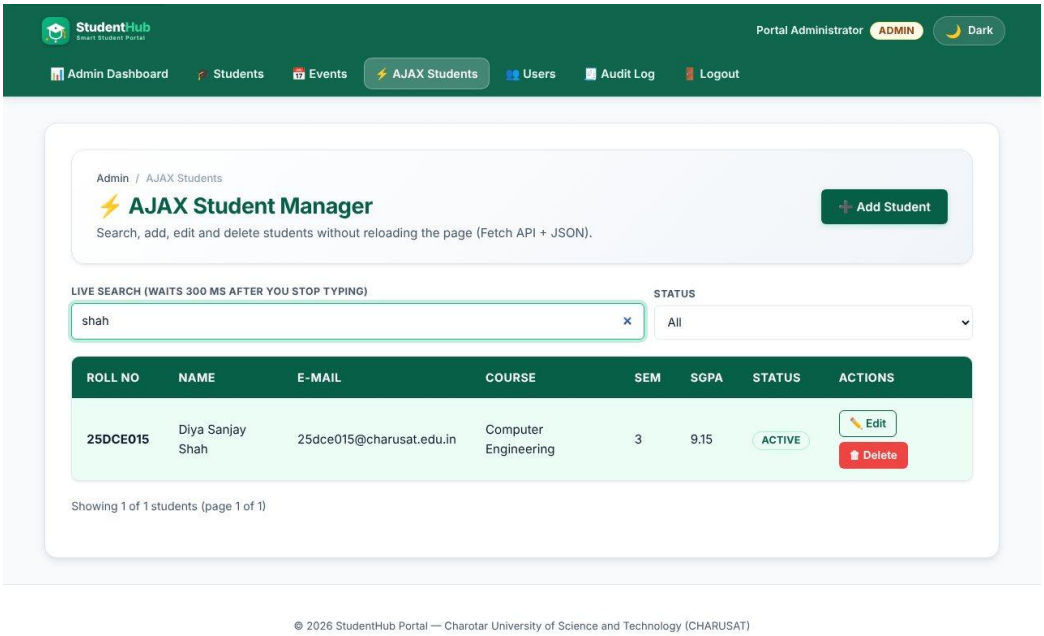
https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-13

GitHub repository: https://github.com/Aarav280907/WEB

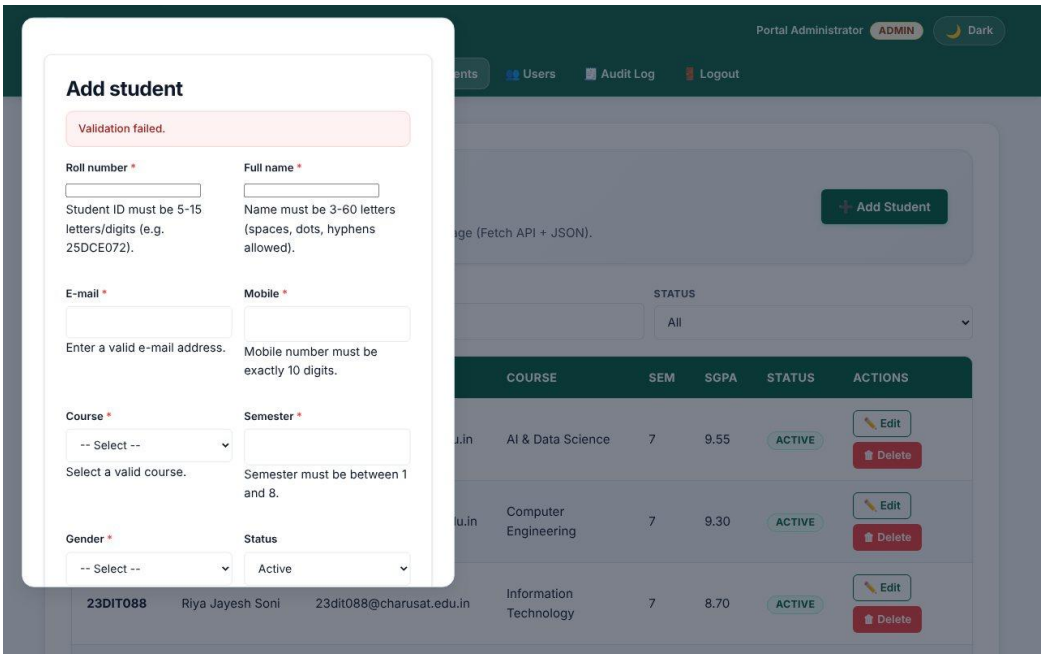
PREVIEW:



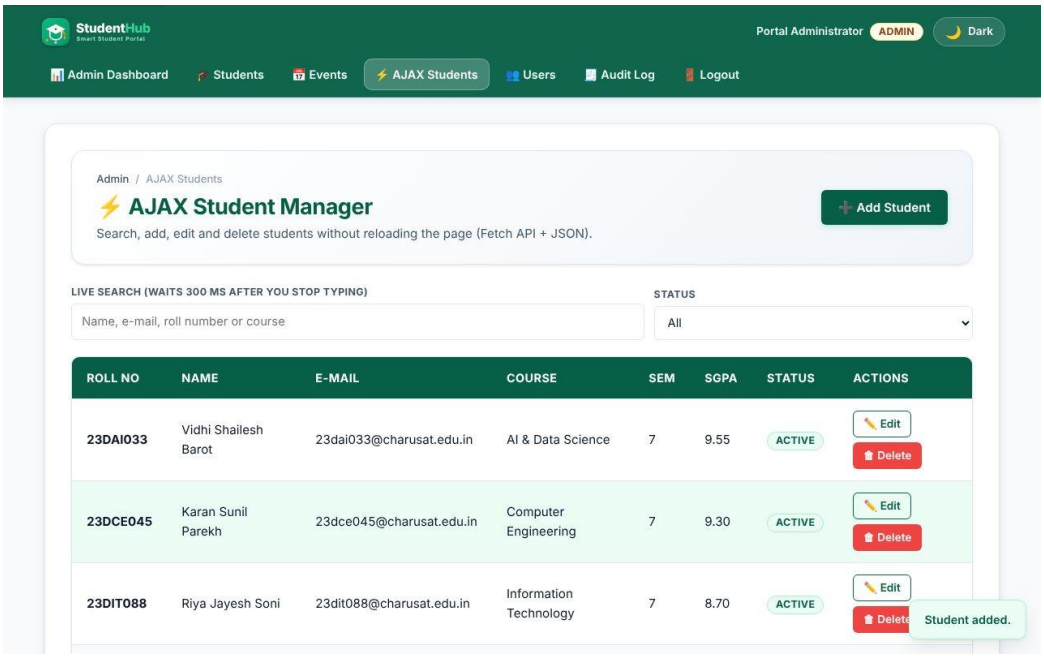
AJAX student manager - rows loaded with fetch() from api/students.php



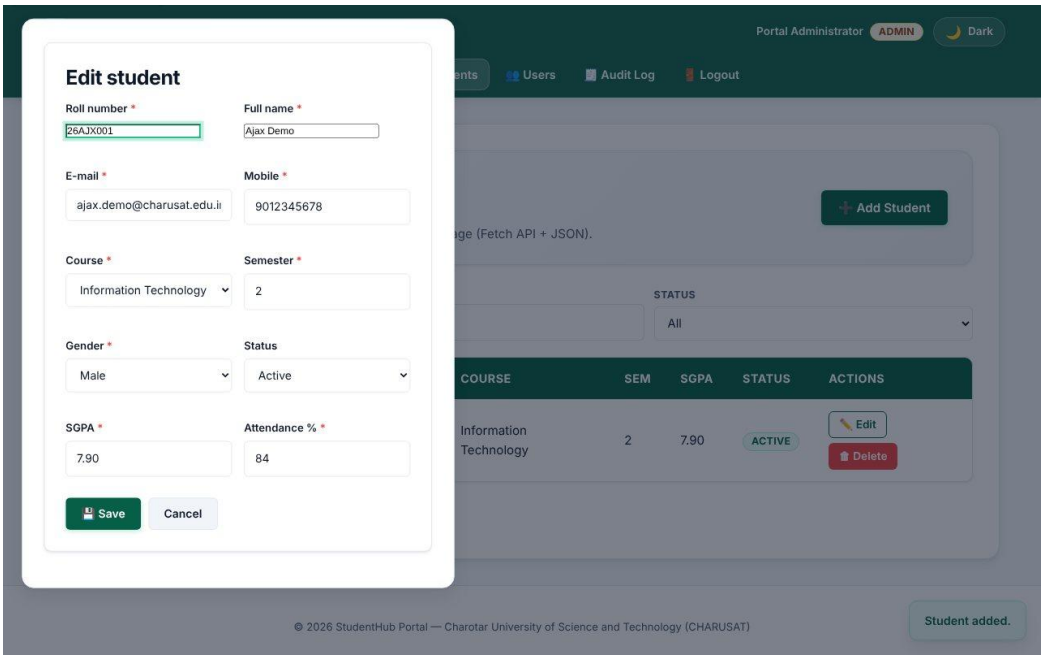
Live search with 300 ms debounce (intermediate extension)



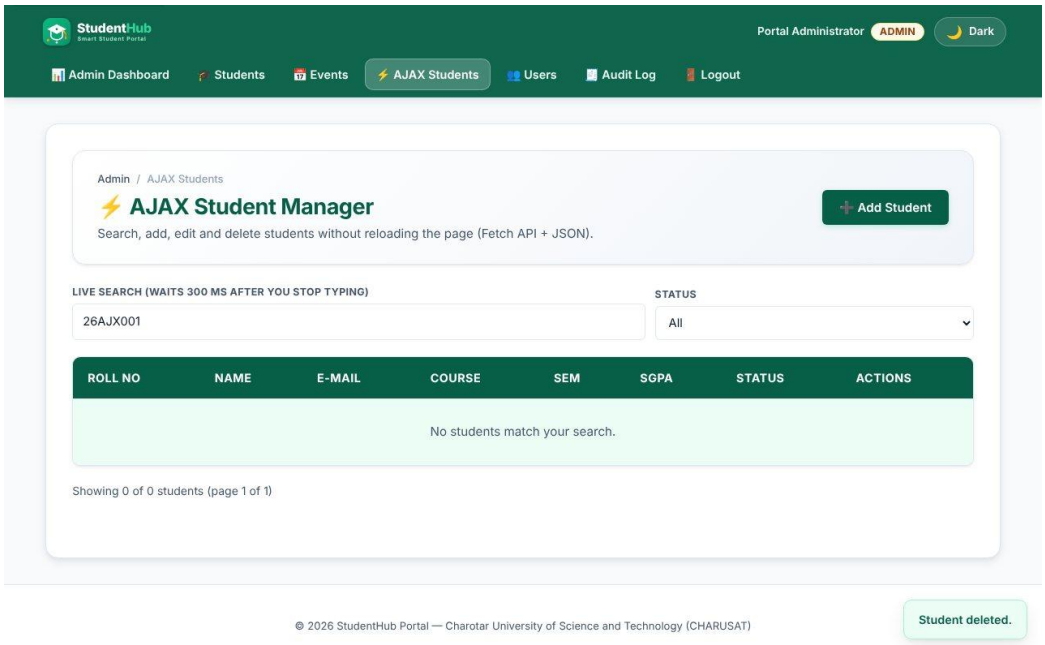
Server validation errors (HTTP 422) shown inside the dialog



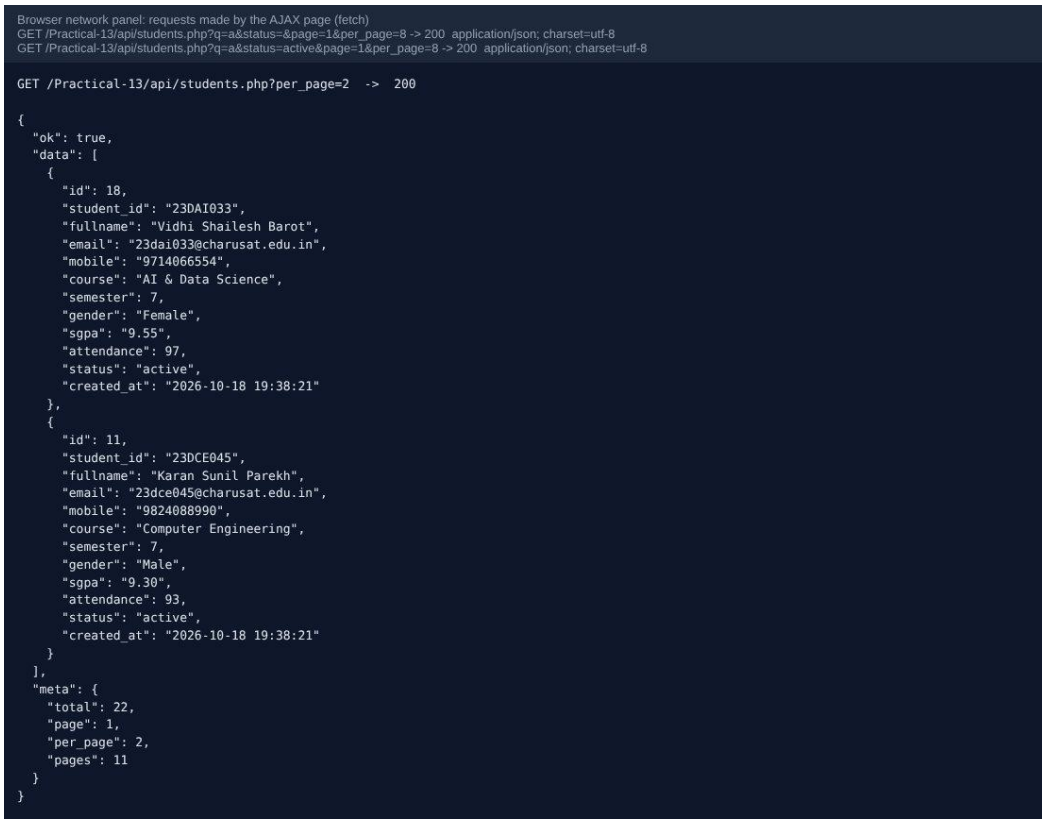
POST - student created, success toast, table refreshed



GET ?id= then PUT - edit dialog filled from JSON



DELETE - row removed without reloading the page



Browser network requests and the JSON response (application/json)



Centralized JSON error responses: 401, 422, 403, 405

TEST CASES :

#	Input / action	Expected result	Result
1	GET api/students.php without login	401 JSON {"ok":false,"error":{...}}	Pass
2	GET as student	403 Administrator access required	Pass
3	GET list as admin	200 JSON with data[] and meta{total,page,per_page,pages}	Pass
4	GET ?q=patel&per_page=3	Filtered, paginated result	Pass
5	GET ?status=zzz	400 Bad Request	Pass
6	GET ?id=99999	404 Not Found	Pass
7	PATCH	405 with Allow header	Pass
8	POST without X-CSRF-Token	403	Pass
9	POST malformed JSON	400	Pass
10	POST invalid fields	422 with a "fields" object	Pass
11	POST valid student	201 Created with the new row	Pass
12	POST duplicate roll number	409 Conflict	Pass
13	PUT ?id=N	200, row updated	Pass
14	DELETE ?id=N, then again	200, then 404	Pass
15	events.php GET (status=open) and POST	List 200; create 201 (admin only)	Pass
16	Server stopped / request blocked	Error banner "Network error - is the server running?"	Pass

KEY QUESTIONS :

1. Do endpoints return valid JSON?

Yes. bootstrap.php sets Content-Type: application/json for every endpoint and all responses go through json_out() / json_error(), which use json_encode. Even uncaught exceptions are turned into a JSON 500 by a central exception handler; the real message is only written to the error log.

{"ok":true,"data":[{...}],"meta":{"total":22,"page":1,"per_page":8,"pages":3}}
{"ok":false,"error":{"code":422,"message":"Validation failed.","fields":{"email":"Enter a valid e-mail address."}}}

2. Are GET and POST handled?

Yes, and also PUT and DELETE for students. api_methods([...]) allows only the listed methods and answers 405 with an Allow header for others. Writes read a JSON body (php://input) and need the X-CSRF-Token header; reads are open to admins only. events.php supports GET (list/one) and POST (create).

```
$method = api_methods(['GET', 'POST', 'PUT', 'DELETE']);
$admin = api_require_admin();
if ($method !== 'GET') api_csrf();
```

3. Are loading, success and error states shown?

Loading: a status message with aria-live and aria-busy on the table body. Success: a toast ("Student added."). Errors: field messages from the 422 response inside the dialog, a banner for list failures, a toast for edit/delete failures; the Save button is disabled and shows "Saving..." during a request. A request counter ignores stale answers when the user types quickly.

```
async function api(url, opts) {
  const res = await fetch(url, { ...opts, headers, credentials: "same-origin" });
  const body = await res.json();
  if (!res.ok || body.ok === false) throw { status: res.status, message: body.error.message, fields:
body.error.fields };
  return body;
}
```

4. Is input validated before database operations?

Yes. The same validate_student() / validate_event() functions used by the normal forms run on the decoded JSON before any SQL; on errors the API answers 422 and does not touch the database. SQL always uses prepared statements, and a duplicate key (MySQL 1062) becomes a 409 instead of a crash.

5. Advanced extension - status codes and centralized error handling

The API uses 200, 201, 400, 401, 403, 404, 405, 409, 422 and 500 consistently and all of them share one error envelope created in one place (json_error), so the JavaScript has a single way to read errors.

PRACTICAL-14

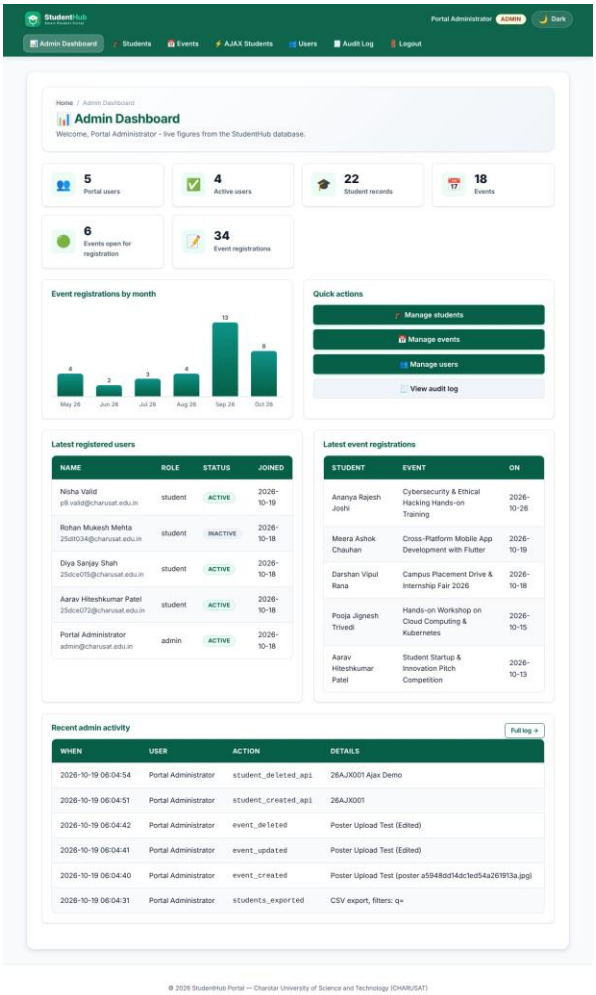
AIM: Build the admin dashboard with role-based access, dynamic menu, live statistics and an audit log of important actions.

SOURCE CODE :

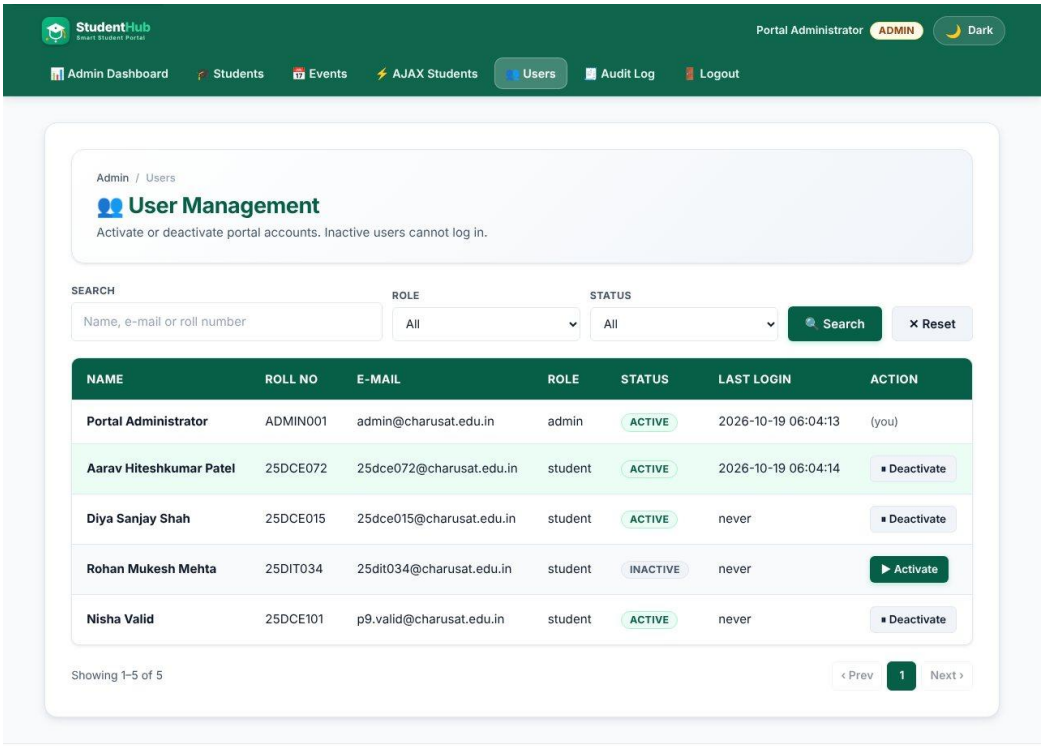
https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-14

GitHub repository: https://github.com/Aarav280907/WEB

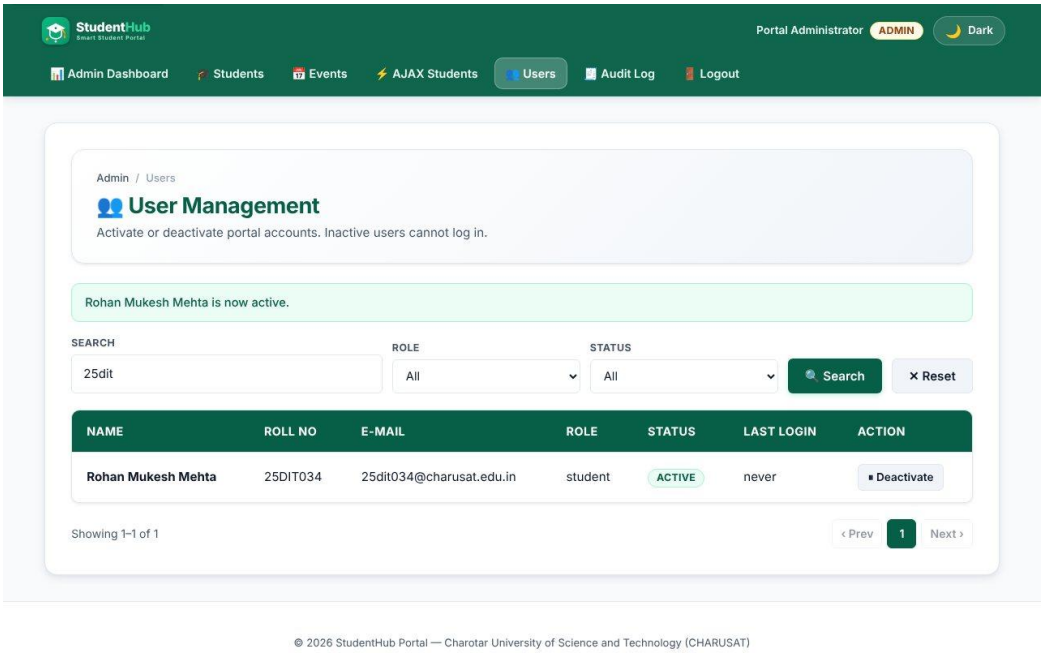
PREVIEW:



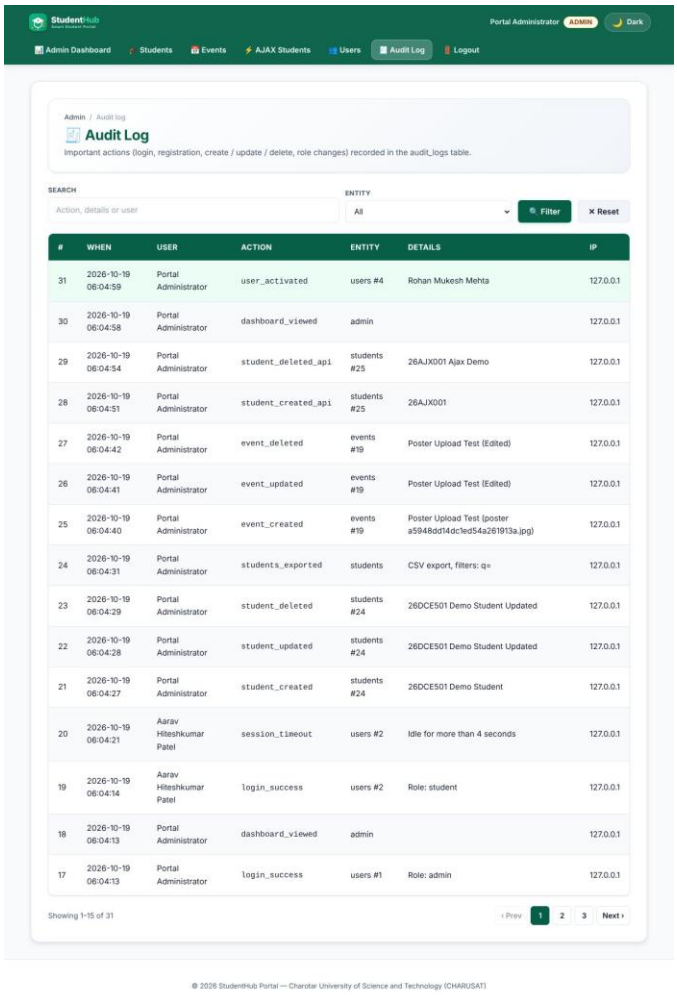
Admin dashboard: statistics from the database, registrations-by-month chart, latest users and registrations, recent audit rows



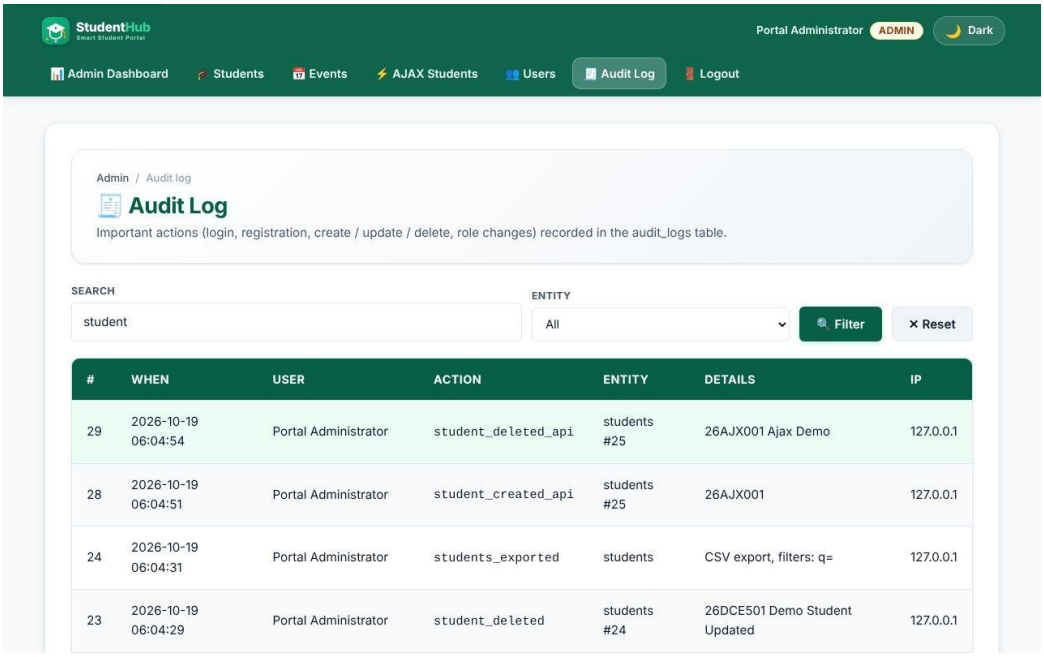
User management with active / inactive toggle



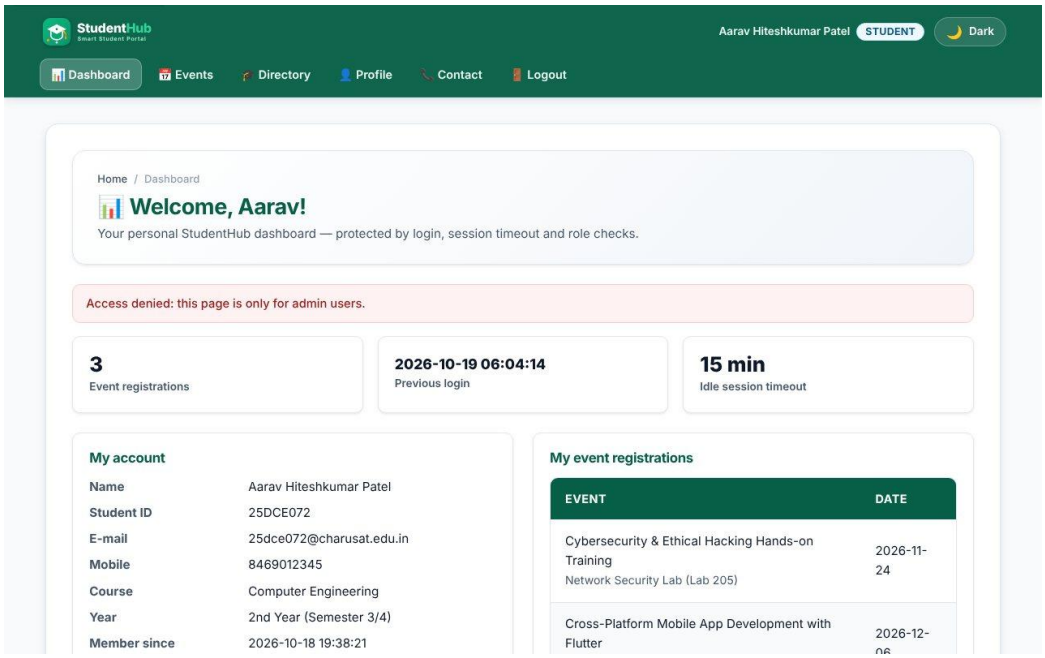
Toggling a user shows a message and is recorded in the audit log



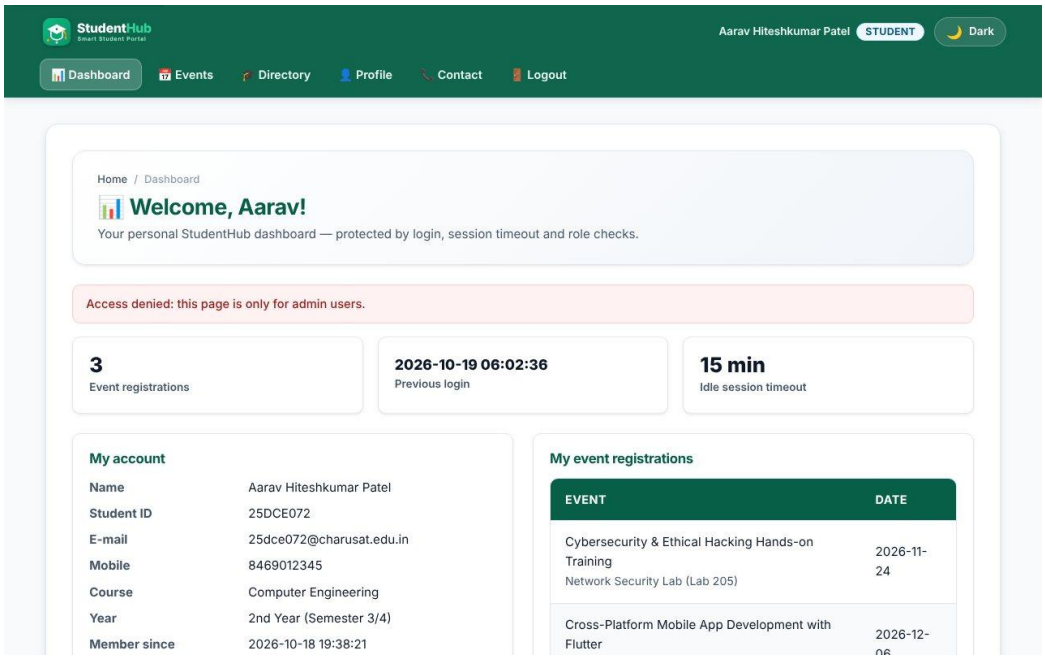
Audit log page



Audit log search filter



A student cannot open admin pages



Unauthorized access redirect (student on admin page)

TEST CASES :

#	Input / action	Expected result	Result
1	Guest opens admin_dashboard.php	Redirect to login	Pass
2	Student opens dashboard / users / audit log	Access denied, redirected	Pass
3	Admin opens dashboard	Numbers equal the database: 5 users, 22 students, 18 events, 34 registrations	Pass
4	Admin deactivates another user	Status inactive; user cannot log in; audit row user_deactivated	Pass

5	Admin deactivates himself	Refused: "You cannot deactivate your own account"	Pass
6	Activate again	Status active; audit row user_activated	Pass
7	POST toggle without CSRF token	Refused, nothing changes	Pass
8	Audit log search "user_"	Only matching actions shown, newest first	Pass
9	Create / update / delete student or event, login failures, timeouts	Each creates an audit_logs row (user, action, entity, details, IP, time)	Pass
10	Menu as guest / student / admin	Different links for each role	Pass

KEY QUESTIONS :

1. Are dashboard values loaded dynamically?

Yes. Every number is a COUNT(*) query run on each page load (users, active users, students, events, open events, registrations). The bar chart is built from GROUP BY DATE_FORMAT(registered_at, '%Y-%m') for the last six months, and the latest users, registrations and audit rows come from JOIN queries. Nothing is hard-coded.

```
$res = $conn->query("SELECT DATE_FORMAT(registered_at,'%Y-%m') m, COUNT(*) c FROM registrations
                     WHERE registered_at >= DATE_SUB(DATE_FORMAT(CURDATE(),'%Y-%m-01'), INTERVAL 5 MONTH) GROUP BY m");
```

2. Is access restricted to admin?

Yes. Every page in Practical 14 begins with require_role('admin'). The check is on the server for each request, so hiding the menu link is only cosmetic.

3. Do role-based menus work?

Yes. sh_menu() in layout.php returns a different list of links for a guest (Home, Events, Register, Login, Contact), a student (Dashboard, Events, Directory, Profile, Logout) and an admin (Admin Dashboard, Students, Events, AJAX Students, Users, Audit Log, Logout). The active page is marked with aria-current="page".

4. Are important admin actions recorded in audit_logs?

Yes. audit_log($conn, $userId, $action, $entity, $entityId, $details) stores the user, action, entity, details, IP address and time with a prepared INSERT. It is called for registration, login success / failure / blocked / timeout, student and event create / update / delete (forms and API), user activate / deactivate and dashboard views. audit_logs.php lets the admin search and filter the entries.

audit_log($conn, $admin['id'], $new === 'active' ? 'user_activated' : 'user_deactivated', 'users', $uid, $u['fullname']);

5. Intermediate and advanced extensions

Active / inactive toggle in users.php (not for your own account; the user's remember-me tokens are removed when deactivated). Registrations-by-month bar chart, drawn with CSS and an aria-label that lists the values for screen readers.

PRACTICAL-15

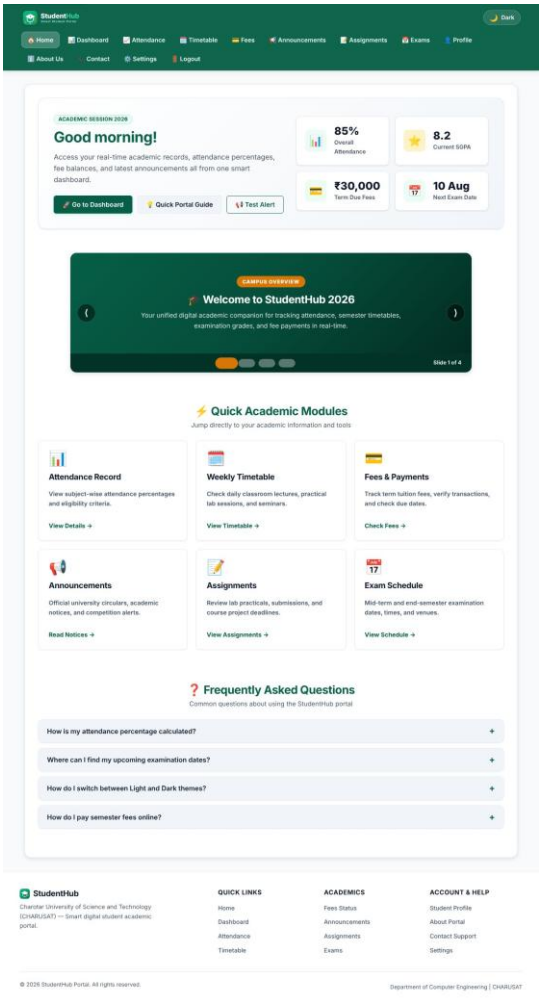
AIM: Integrate all modules of the StudentHub portal, test the complete system, finish the documentation and deploy it locally (Apache virtual host).

SOURCE CODE :

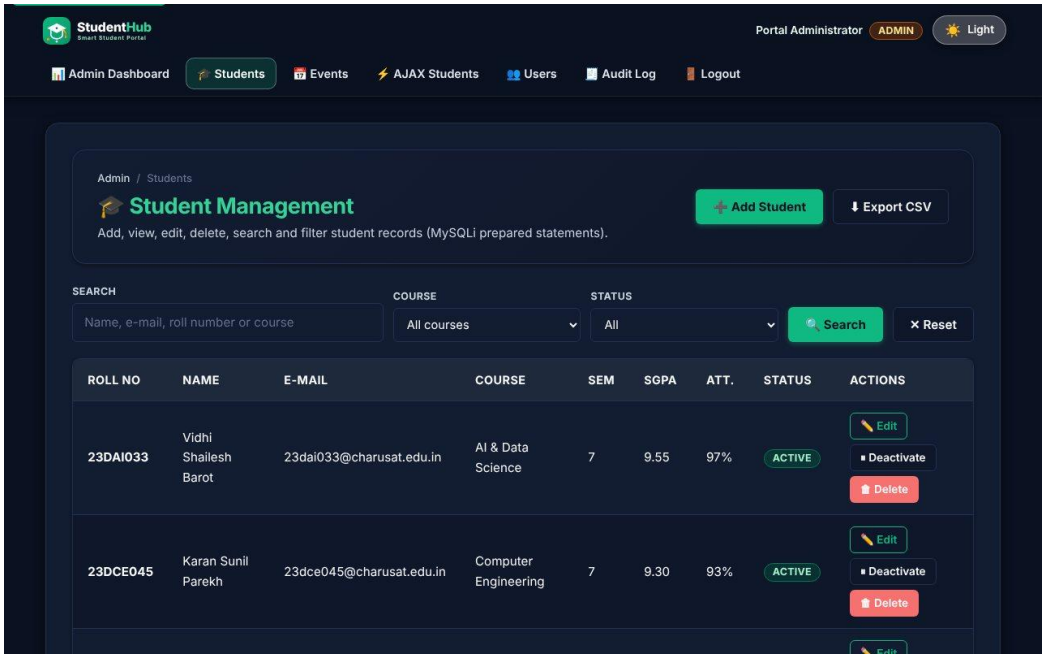
https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-15

GitHub repository: https://github.com/Aarav280907/WEB

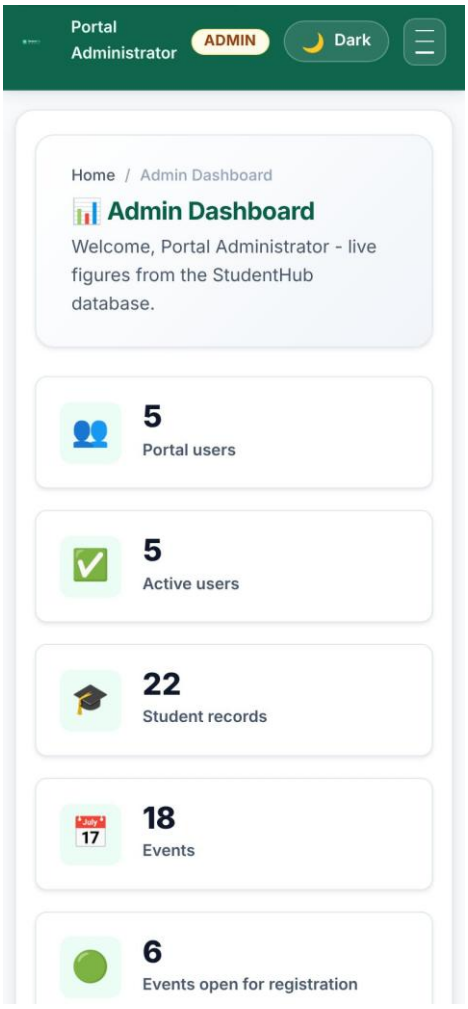
PREVIEW:



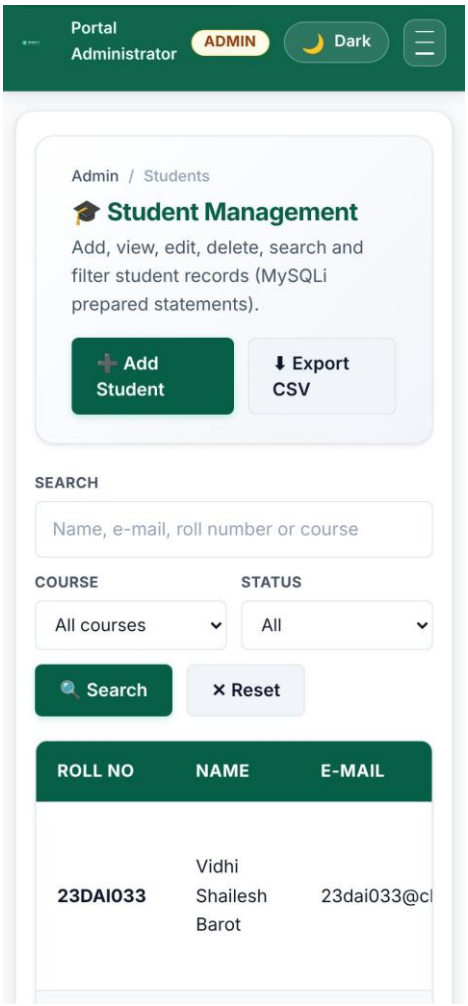
Home page - entry point of the portal



Dark theme on a PHP module page



Mobile layout of the admin dashboard (390 px)



Mobile layout of the student list

TEST CASES :

#	Input / action	Expected result	Result
1	P5 Validation	Inline errors, strength meter, CAPTCHA, aria-invalid	Pass
2	P6 JSON	Search, filter, sort, pagination, error state	Pass
3	P7 PHP form	POST validated; CSV + JSON files written	Pass
4	P8 Database	Schema, FK/UNIQUE/CHECK, PDO + MySQLi connection	Pass
5	P9 Registration	Hashed passwords, duplicates caught, CSRF	Pass
6	P10 Sessions	Login, logout, roles, timeout, lock-out, remember-me	Pass
7	P11 Students	CRUD, search, filter, CSV export, soft delete	Pass
8	P12 Events	CRUD, poster upload validation	Pass
9	P13 API	JSON endpoints, status codes, AJAX CRUD	Pass
10	P14 Admin	Dashboard, user toggle, audit log	Pass
11	Link check: crawl of 114 URLs as guest, student and admin	0 broken links (no 4xx/5xx)	Pass
12	PHP syntax check (php -l) on every .php file	No errors	Pass

KEY QUESTIONS :

1. Are all pages linked and navigable?

Yes. A crawler logged in as guest, student and admin and followed every link, image, stylesheet and script on the PHP pages and the HTML home page (114 URLs) - no response was 4xx or 5xx. Role-based menus link the modules to each other, and breadcrumbs show the position. The skip link, theme switch and hamburger menu are part of the shared header.

2. Do sessions, roles, validations and database operations work end to end?

Yes. The flow register (P9) -> login (P10) -> student dashboard, and login as admin -> dashboard (P14) -> students (P11) / events (P12) / AJAX (P13) / users / audit log was tested with scripted requests and with a real browser (Playwright); the screenshots in Practicals 5-14 were taken from that run. All write operations leave an audit_logs entry.

3. Are the security and accessibility checklists completed?

See the two tables below.

4. Is the documentation sufficient for installation and testing?

Yes. README.md in the project root explains requirements, installation (copy to htdocs, run Practical-15/reset_db.sh or import studenthub_full.sql, set DB credentials), demo logins, the folder structure, how to run each practical and how to test; Practical-15/TEST_REPORT.md lists every test case. Practical-15/studenthub.conf is a ready Apache virtual host.

5. Is the repository clean and properly committed?

Each practical is a separate folder so it can be committed separately with a clear message (for example "Practical 9: secure registration with password_hash"). The included .gitignore keeps logs, local settings and temporary files out of the repository; the repository contains only source, SQL, documents and screenshots. No real passwords or personal data are included - the seed data are demo records and the demo passwords are stored as bcrypt hashes.

SECURITY CHECKLIST :

Item	Done	Where

Prepared statements for every query	Yes	all PHP modules (MySQLi; PDO demo in P8)
Passwords hashed with password_hash / verified with password_verify	Yes	P9 register.php, P10 login.php
Output escaped with htmlspecialchars (e())	Yes	includes/helpers.php
CSRF token on every POST form and on API writes (header)	Yes	helpers.php, api/bootstrap.php
Session cookie HttpOnly + SameSite, strict mode, ID regenerated on login	Yes	helpers.php, auth.php
Idle session timeout	Yes	auth_boot()
Role check on the server for every admin page / endpoint	Yes	require_role(), api_require_admin()
Brute-force slow-down on login (5 attempts / 30 s)	Yes	P10 login.php
Same error for unknown user and wrong password	Yes	P10 login.php
Upload: size, real MIME, getimagesize, random name, execution blocked	Yes	includes/upload.php, uploads/.htaccess
Remember-me token stored only as SHA-256 hash, rotated on use	Yes	auth.php, remember_tokens
Generic error messages; details only in the error log	Yes	db.php, api/bootstrap.php
CSV export protected against formula injection	Yes	P11 students_export.php
Audit log of important actions	Yes	audit_log(), P14
HTTPS and production DB user with limited rights	To do on a real server	README (deployment notes)

ACCESSIBILITY CHECKLIST :

Item	Done	Where
Semantic landmarks (header, nav, main, footer) and a skip link	Yes	layout.php, HTML pages
One h1 per page, headings in order	Yes	all pages
Every form control has a label	Yes	all forms
Errors announced with role="alert" and aria-invalid	Yes	forms, validation.js
Status messages use role="status" / aria-live	Yes	AJAX page, JSON pages
Current page marked with aria-current	Yes	menu
Images have alt text (posters: "Poster of ...")	Yes	P12 pages
Full keyboard operation, visible focus	Yes	all pages
Dialog is a native <dialog> (focus trap, Esc to close)	Yes	P13
Light/dark themes with sufficient contrast	Yes	common.css
Responsive layout down to 390 px	Yes	screenshots above

DEPLOYMENT :

Local deployment: copy the project to the web root (XAMPP htdocs or /var/www/studenthub), import Practical-15/studenthub_full.sql (or run Practical-15/reset_db.sh), check the credentials in Practical-8/db.php and open Practical-10/login.php. Intermediate extension: Practical-15/studenthub.conf is an Apache virtual host (studenthub.local) with directory-listing off, SQL/shell files denied and security headers. Advanced extension: the static part (Html folder, for_css, javascript, Practical-6 JSON) works without PHP and can be published on GitHub Pages; the PHP modules need a PHP host.